A sparsity-sensitive decremental all-pairs distance tradeoff with near-unit stretch and linear additive error

Michal Dory, Sebastian Forster, Yasamin Nazari, Tijn de Vos

arxiv:2211.01152v4; CC BY 4.0

Editorial note. Only thm:main_add is this task primary claim. Entire original mathematical body and supporting algorithm proofs, scale assumptions, diagrams, references and appendix retained. Inputs/bbl mechanically expanded; original publisher titlepage closing boundary replaced by a wrapper comment.

Editorial note. Author-name ogonek accent macros are represented by their equivalent precomposed Unicode letters to avoid missing combining glyphs in the installed text font.

Editorial note. Existing runtime Noto Serif text fonts and their license are bundled for portable rendering of author-name ogonek accents; no dependency is installed and mathematical fonts/content are unchanged.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

We provide new tradeoffs between approximation and running time for the decremental all-pairs shortest paths (APSP) problem. For undirected graphs with m edges and n nodes undergoing edge deletions, we provide four new approximate decremental APSP algorithms, two for weighted and two for unweighted graphs. Our first result is $(2+\epsilon)$-APSP with total update time $\tilde{O}(m^{1/2}n^{3/2})$ (when $m = n^{1+c}$ for any constant $0 < c < 1$). Prior to our work the fastest algorithm for weighted graphs with approximation at most 3 had total $\tilde{O}(mn)$ update time for $(1+\epsilon)$-APSP [Bernstein, SICOMP 2016]. Our second result is $(2+\epsilon, W_{u,v})$-APSP with total update time $\tilde{O}(nm^{3/4})$, where the second term is an additive stretch with respect to $W_{u,v}$, the maximum weight on the shortest path from u to v.

Our third result is $(2+\epsilon)$-APSP for unweighted graphs in $\tilde{O}(m^{7/4})$ update time, which for sparse graphs ($m = o(n^{8/7})$) is the first subquadratic $(2+\epsilon)$-approximation. Our last result for unweighted graphs is $(1+\epsilon, 2(k-1))$-APSP, for $k \geq 2$, with $\tilde{O}(n^{2-1/k}m^{1/k})$ total update time (when $m = n^{1+c}$ for any constant $c > 0$). For comparison, in the special case of $(1+\epsilon, 2)$-approximation, this improves over the state-of-the-art algorithm by [Henzinger, Krinninger, Nanongkai, SICOMP 2016] with total update time of $\tilde{O}(n^{2.5})$. All of our results are randomized, work against an oblivious adversary, and have constant query time.

Contents

1 Introduction **2**
- 1.1 Our Results . . . 3
- 1.2 Related Work . . . 5

2 High-Level Overview **6**
- 2.1 $(2+\epsilon)$-APSP for Weighted Graphs . . . 6
  - 2.1.1 Static 2-APSP . . . 7
  - 2.1.2 Dynamic Challenges . . . 8
- 2.2 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs . . . 10
  - 2.2.1 $(2+\epsilon)$-APSP for Unweighted Graphs . . . 12
- 2.3 $(1+\epsilon, 2(k-1))$-APSP for Unweighted Graphs . . . 12
  - 2.3.1 $(\mathbf{1}+\epsilon, \mathbf{2})$-APSP . . . 12
  - 2.3.2 General k . . . 14
  - 2.3.3 Limitations of previous approaches . . . 14

3 Preliminaries **15**
- 3.1 Notation and Terminology . . . 15
- 3.2 Decremental Tools . . . 16

4 $(2+\epsilon)$-Approximate APSP for Weighted Graphs **16**
- 4.1 Decremental Data Structures . . . 17
- 4.2 Further Details and Time Analysis . . . 19
- 4.3 Stretch Analysis . . . 25
- 4.4 Putting Everything Together . . . 26

5 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs and Unweighted 2-APSP **26**
- 5.1 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs . . . 27
- 5.2 Reduction and Unweighted $(2+\epsilon)$-APSP . . . 34

6 Approximate APSP with Additive Factors **36**
- 6.1 Decremental Tools . . . 36
- 6.2 Warm-up: $(1+\epsilon, 2)$-APSP . . . 37
  - 6.2.1 Dynamic Data Structure . . . 37
  - 6.2.2 Stretch Analysis . . . 38
  - 6.2.3 Time Complexity . . . 40
- 6.3 $(1+\epsilon, 2k-2)$-APSP . . . 42
  - 6.3.1 Dynamic Data Structure . . . 42
  - 6.3.2 Stretch Analysis . . . 43
  - 6.3.3 Time Complexity . . . 44
  - 6.3.4 Putting Everything Together . . . 48

A Monotone ES-tree **55**

B Maintaining Bunches and Clusters **56**

1 Introduction

The dynamic algorithms paradigm is becoming increasingly popular for studying algorithmic questions in the presence of gradually changing inputs. A natural goal in this area is to design algorithms that process each change to the input as fast as possible to adapt the algorithm's output (or a data structure for querying the output) to the current state of the input. The time spent after an update to perform these computations is called the *update time* of the algorithm. In many cases, bounds on the update time are obtained in an *amortized* sense as an average over a long enough sequence of updates. Among dynamic graph problems, the question of maintaining exact or approximate shortest paths has received considerable attention in the past two decades. The main focus usually lies on maintaining a distance oracle that answers queries for the distance between a pair of nodes. For this problem, we call an algorithm *fully dynamic* if it supports both insertions and deletions of edges, and *partially dynamic* if it supports only one type of updates; in particular we call it *decremental* if it only supports edge deletions (which is the focus of this paper), and *incremental* if it only supports edge insertions.

The running times of partially dynamic algorithms are usually characterized by their bounds on the *total update time*, which is the accumulated time for processing all updates in a sequence of at most m deletions (where m is the maximum number of edges ever contained the graph). A typical design choice, which we also impose in this paper, is small (say polylogarithmic) query time. In particular, our algorithms will have constant query time. While fully dynamic algorithms are more general, the restriction to only one type of updates in partially dynamic algorithms often admits much faster update times. In particular, some partially dynamic algorithms have a total update time that almost matches the running time of the fastest static algorithm, i.e., computing *all* updates does not take significantly more time than processing the graph once.

Decremental shortest paths. For the decremental single-source shortest paths problem, conditional lower bounds [RZ11, HKNS15] suggest that *exact* decremental algorithms have an $\Omega(mn)$ bottleneck in their total update time (up to subpolynomial factors). On the other hand, this problem admits a $(1+\epsilon)$-approximation (also called *stretch*) with total update time $m^{1+o(1)}$ in weighted, undirected graphs [BR11, HKN18, BC16, Ber17, BGS21], which exceeds the running time of the state-of-the art static algorithm by only a subpolynomial factor. Hence for the single-source shortest paths problem on undirected graphs we can fully characterize for which multiplicative stretches the total update time of the fastest decremental algorithm matches (up to subpolynomial factors) the running time of the fastest static algorithm.

Obtaining a similar characterization for the decremental all-pairs shortest paths (APSP) problem is an intriguing open question. Conditional lower bounds [DHZ00, HKNS15] suggest an $\Omega(mn)$ bottleneck in the total update time of decremental APSP algorithms with (a) any finite stretch on *directed* graphs and (b) with any stretch guarantee with a multiplicative term of $\alpha \geq 1$ and an additive term of $\beta \geq 0$ with $2\alpha + \beta < 4$ on *undirected* graphs. This motivates the study of decremental (α, β)-approximate APSP algorithms such that $2\alpha + \beta \geq 4$, i.e., with multiplicative stretch $\alpha \geq 2$ or additive stretch $\beta \geq 2$.

Apart from two notable exceptions [HKN16, AC13], all known decremental APSP algorithms fall into one of two categories. They either (a) maintain exact distances or have a relatively small multiplicative stretch of $(1+\epsilon)$ or (b) they have a stretch of at least 3. The space in between is largely unexplored in the decremental setting. This stands in sharp

contrast to the static setting where for undirected graphs approximations guarantees different from multiplicative $(1+\epsilon)$ or "3 and above" have been the focus of a large body of works [ACIM99, DHZ00, CZ01, BK10, BK07, Kav12, BGS09, PR14, Som16, Knu17, AR20, AR21]. The aforementioned exceptions in dynamic algorithms [HKN16,AC13], concern unweighted undirected graphs and maintain $(1+\epsilon, 2)$-approximations that simultaneously have a multiplicative error of $1+\epsilon$ and an additive error of 2, which implies a purely multiplicative $(2+\epsilon)$-approximation. The algorithms run in $\tilde{O}(n^{5/2})$[1] and $n^{5/2+o(1)}$ total update time respectively. In the next section, we will detail how our algorithms improve over this total update time and generalize to weighted graphs.

Concerning the fully dynamic setting, Bernstein [Ber09] provided an algorithm for $(2+\epsilon)$-APSP which takes $m^{1+o(1)}$ time per update. Although we are not aware of any lower bounds, it seems to be hard to beat this: no improvements have been made since. The lack of progress in the fully-dynamic setting motivates the study in a partially dynamic setting, where we obtain improvements for the decremental case.

1.1 Our Results

In this paper, we provide novel decremental APSP algorithms with approximation guarantees that previously were mostly unexplored in the decremental setting. Our algorithms are randomized and we assume an oblivious adversary. For each pair of nodes, we can not only provide the distance estimate, but we can also report a shortest path π of this length in $\tilde{O}(|\pi|)$ time using standard techniques, see e.g. [FGNS23].

$(2+\epsilon)$-APSP for weighted graphs. Our first contribution is an algorithm for maintaining a $(2+\epsilon)$-approximation in weighted, undirected graphs.

Theorem 1.1. *Given a weighted graph G and a constant $0 < \epsilon < 1$, there is a decremental data structure that maintains a $(2+\epsilon)$-approximation of APSP. The algorithm has constant query time and the total update time is w.h.p.*

- $\tilde{O}(m^{1/2}n^{3/2}\log^2(nW))$ *if* $m = n^{1+\Omega(1)}$ *and* $m \leq n^{2-\rho}$ *for an arbitrary small constant* ρ*,*
- $\tilde{O}(m^{1/2}n^{3/2+\rho}\log^2(nW))$ *if* $m \geq n^{2-\rho}$ *for any constant* ρ*,*
- $\tilde{O}(m^{1/2}n^{3/2+o(1)}\log^2(nW))$ *otherwise,*

where W is the ratio between the maximum and minimum weight.

The fastest known algorithm with an approximation ratio at least as good as ours is the $(1+\epsilon)$-approximate decremental APSP algorithm by Bernstein [Ber16] with total update time $\tilde{O}(mn)$. With our $(2+\epsilon)$-approximation, we improve upon this total update time when $m = n^{1+\eta}$, for any $\eta > 0$. Furthermore, the fastest known algorithm with a larger approximation ratio than our algorithm is the $(3+\epsilon)$-approximate decremental distance oracle by Łącki and Nazari [ŁN22] with total update time $\tilde{O}(m\sqrt{n})$. Our result also has to be compared to the *fully dynamic* algorithm of Bernstein [Ber09] for maintaining a $(2+\epsilon)$-approximation that takes amortized time $m^{1+o(1)}$ per update. Note that in *unweighted* graphs, $(2+\epsilon)$-approximate decremental APSP algorithm can be maintained with total update time $\tilde{O}(n^{2.5})$

[1] In the introduction, we make two simplifying assumptions: (a) ϵ is a constant and (b) the ratio W between the maximum and minimum weight is polynomial in n. Unless otherwise noted, the cited algorithms have constant or polylogarithmic query time. Throughout we use $\tilde{O}(\cdot)$ notation to omit factors that are polylogarithmic in n.

(which is implied by the results of Abraham and Chechik [AC13] and Henzinger, Krinninger, and Nanongkai [HKN16]). Our approach improves upon this bound for most densities, and matches it for $m = n^2$.

$(2+\epsilon, 1)$-APSP for unweighted and $(2+\epsilon, W_{u,v})$-APSP for weighted graphs. Our second contribution is a faster algorithm with an additional additive error term of 1 for unweighted graphs, which in turn can be used to obtain our unweighted $(2+\epsilon)$-APSP result. The corresponding generalization to weighted graphs can be formulated as follows.

Theorem 1.2. *Given a weighted graph G and a constant $0 < \epsilon < 1$, there is a decremental data structure that maintains a $(2+\epsilon, W_{u,v})$-approximation for APSP, where $W_{u,v}$ is the maximum weight on a shortest path from u to v. The algorithm has constant query time and the total update time is w.h.p. $\tilde{O}(nm^{3/4}\log^2(nW))$ if $m = n^{1+\Omega(1)}$, and $\tilde{O}(n^{1+o(1)}m^{3/4}\log^2(nW))$ otherwise, where W is the ratio between the maximum and minimum weight.*

$(2+\epsilon)$-APSP for unweighted graphs. We obtain this result by a general reduction from mixed approximations to purely multiplicative approximations, which might be of independent interest beyond the dynamic setting. See Theorem 2.1 for the statement. We can then combine this with Theorem 1.2 to obtain a fast algorithm for non-dense unweighted graphs.

Theorem 1.3. *Given an undirected unweighted graph G, there is a decremental data structure that maintains a $(2+\epsilon)$-approximation for APSP with constant query time. W.h.p. the total update time is bounded by $\tilde{O}(m^{7/4})$ if $m = n^{1+\Omega(1)}$, and $\tilde{O}(m^{7/4+o(1)})$ otherwise.*

Note that for $m = o(n^{8/7})$ this gives the first subquadratic decremental $(2+\epsilon)$-approximate APSP algorithm. For $m \leq n^{6/5}$, this approach is faster than our weighted result, Theorem 1.1, which was already beating the unweighted state-of-the-art [AC13, HKN16].

$(1+\epsilon, 2(k-1))$-APSP for unweighted graphs. Our fourth contribution is an algorithm for unweighted, undirected graphs that maintains, for any $k \geq 2$, a $(1+\epsilon, 2(k-1))$-approximation, i.e., a distance estimate that has a multiplicative error of $1+\epsilon$ and an additive error of $2(k-1)$.

Theorem 1.4. *Given an undirected unweighted graph G, a constant $0 < \epsilon < 1$ and an integer $2 \leq k \leq \log n$, there is a decremental data structure that maintains $(1+\epsilon, 2(k-1))$-approximation for APSP with constant query time. The expected total update time is bounded by $\min\{O(n^{2-1/k+o(1)}m^{1/k}), \tilde{O}((n^{2-1/k}m^{1/k})O(1/\epsilon)^{k/\rho})\}$ where $m = n^{1+\rho}$.*

Note that for small values of k and if $m = n^{1+\rho}$ for a constant $\rho > 0$, we get total update time of $\tilde{O}(n^{2-1/k}m^{1/k})$, and otherwise we have an extra $n^{o(1)}$ factor. In addition, in the special case of $k = \log n$, we get a near-quadratic update time of $O(n^{2+o(1)})$. The state-of-the-art for a purely multiplicative $(1+\epsilon)$-approximation is the algorithm of Roditty and Zwick with total update time $\tilde{O}(mn)$.[2] It was shown independently by Abraham and Chechik [AC13] and by Henzinger, Krinninger, and Nanongkai [HKN16] how to improve upon this total update time bound at the cost of an additional small additive error term: a $(1+\epsilon, 2)$-approximation can be maintained with total update time $\tilde{O}(n^{2.5})$. This has been generalized by Henzinger, Krinninger, and Nanongkai [HKN14] to an additive error term of $2(1+\frac{2}{\epsilon})^{k-2}$ and total update time $\tilde{O}(n^{2+1/k}O(\frac{1}{\epsilon})^{k-1})$. We improve upon this tradeoff in two ways: (1) our additive term is independent of $1/\epsilon$ and *linear* in k and (2) our algorithm profits from graphs being sparse.

[2] Note that the algorithms of Roditty and Zwick [RZ12] for unweighted, undirected graphs precedes the more general algorithm of Bernstein [Ber16] for weighted, directed graphs.

Static follow-up work. Interestingly, our techniques inspired a new algorithm for a static 2-approximate distance oracle [DFK+24], giving the first such distance oracle algorithm with subquadratic construction time for sparse graphs. Moreover, for $m = n$ they match conditional lower bounds [PRT12, ABF23].

1.2 Related Work

Static algorithms. The baseline for the static APSP problem are the exact textbook algorithms with running times of $O(n^3)$ and $\tilde{O}(mn)$, respectively. There are several works obtaining improvements upon these running times by either shaving subpolynomial factors or by employing fast matrix multiplication, which sometimes comes at the cost of a $(1+\epsilon)$-approximation instead of an exact result. See [Zwi01] and [Wil18], and the references therein, for details on these approaches. For the regime of stretch 3 and more in undirected graphs, a multitude of algorithms has been developed with the distance oracle of Thorup and Zwick [TZ05] arguably being the most well-known constructions. In the following, we focus on summarizing the state of affairs for approximate APSP with stretch between $1+\epsilon$ and 3.

In *weighted*, undirected graphs, Cohen and Zwick [CZ01] obtained a 2-approximation with running time $\tilde{O}(m^{1/2}n^{3/2})$. This running time has been improved to $\tilde{O}(m\sqrt{n}+n^2)$ by Baswana and Kavitha [BK10] and, employing fast matrix multiplication, to $\tilde{O}(n^{2.25})$ by Kavitha [Kav12]. For $(2+\epsilon)$-APSP, Dory et al. [DFK+24] provide two algorithms, for dense graphs: $O(n^{2.214})$ and sparse graphs: $\tilde{O}(mn^{2/3})$. In addition, efficient approximation algorithms for stretches of $\frac{7}{3}$ [CZ01] and $\frac{5}{2}$ [Kav12] have been obtained. These results have recently been generalized by Akav and Roditty [AR21] who presented an algorithm with stretch $2+\frac{k-2}{k}$ and running time $\tilde{O}(m^{2/k}n^{2-3/k}+n^2)$ for any $k \geq 2$.

In *unweighted*, undirected graphs, a $(1,2)$-approximation algorithm with running time $\tilde{O}(n^{2.5})$ has been presented by Aingworth, Chekuri, Indyk and Motwani [ACIM99]. This has been improved by Dor, Halperin, and Zwick [DHZ00] that showed a $(1,2)$-approximation with running time $\tilde{O}(\min\{n^{3/2}m^{1/2}, n^{7/3}\})$. They also show a generalized version of the algorithm that gives stretch $(1,k)$ and running time $\tilde{O}(\min\{n^{2-2/(k+2)}m^{2/(k+2)}, n^{2+2/(3k-2)}\})$ for every even $k > 2$. Recently faster $(1,2)$-approximation algorithms based on fast matrix multiplication techniques were developed [DKR+22, Dür23], the fastest of them runs in $O(n^{2.260})$ time [Dür23]. This was extended to a $(1+\epsilon,2)$-approximation in $O(n^{2.152})$ [DFK+24]. In addition, recently Roditty [Rod23] extended the approach of [DHZ00] to obtain a combinatorial $(2,0)$-approximation for APSP in $\tilde{O}(n^{2.25})$ time in unweighted undirected graphs. Using fast matrix multiplication, this running time was improved to $O(n^{2.032})$ [DFK+24, SY24].

Berman and Kasiviswanathan [BK07] showed how to compute a $(2,1)$-approximation in time $\tilde{O}(n^2)$. Subsequent works [BGS09, BK10, PR14, Som16, Knu17] have improved the polylogarithmic factors in the running time and the space requirements for such nearly 2-approximations. Recently, slightly subquadratic algorithms have been given: an algorithm with stretch $(2(1+\epsilon),5)$ by Akav and Roditty [AR20], and an algorithm with stretch $(2,3)$ by Chechik and Zang [CZ22].

Decremental algorithms. The fastest algorithms for maintaining exact APSP under edge deletions have total update time $\tilde{O}(n^3)$ [DI06, BHS07, EFGW21]. There are several algorithms that are more efficient at the cost of returning only an approximate solution. In particular, a $(1+\epsilon)$-approximation can be maintained in total time $\tilde{O}(mn)$ [RZ12, Ber16, KŁ19]. If additionally, an additive error of 2 is tolerable, then a $(1+\epsilon,2)$-approximation can be maintained in total time $\tilde{O}(n^{2.5})$ in unweighted, undirected graphs [HKN16, AC13]. Note that such

a $(1+\epsilon, 2)$-approximation directly implies a $(2+\epsilon)$-approximation because the only paths of length 1 are edges between neighboring nodes. All of these decremental approximation algorithms are randomized and assume an oblivious adversary. Deterministic algorithms with stretch $1+\epsilon$ exist for unweighted, undirected graphs with running time $\tilde{O}(mn)$ [HKN16], for weighted, undirected graphs with running time $\tilde{O}(mn^{1+o(1)})$ [BGS21] and for weighted, directed graphs with running time $\tilde{O}(n^3)$ [KŁ20].

Decremental approximate APSP algorithms of larger stretch, namely at least 3, have first been studied by Baswana, Hariharan, and Sen [BHS03]. After a series of improvements [RZ12,BR11,ACT14,HKN18], the state-of-the-art algorithms of [Che18,ŁN22] maintain $(2k-1)(1+\epsilon)$-approximate all-pairs shortest paths for any integer $k \geq 2$ and $0 < \epsilon \leq 1$ in total update time $\tilde{O}((m + n^{1+o(1)})n^{1/k})$ with query time $O(\log\log(nW))$ and $O(k)$ respectively. All of these "larger stretch" algorithms are randomized and assume an oblivious adversary.

Recently, deterministic algorithms have been developed by Chuzhoy and Saranurak [CS21] and by Chuzhoy [Chu21]. One tradeoff in the algorithm of Chuzhoy [Chu21] for example provides total update time $\tilde{O}(m^{1+\mu})$ for any constant μ and polylogarithmic stretch. As observed by Mądry [M10], decremental approximate APSP algorithms that are deterministic – or more generally work against an adaptive adversary – can lead to fast static approximation algorithms for the maximum multicommodity flow problem via the Garg-Könemann-Fleischer framework [GK07,Fle00]. The above upper bounds for decremental APSP have recently been contrasted by conditional lower bounds [ABKZ22] stating that constant stretch cannot be achieved with subpolynomial update and query time under certain hardness assumptions on 3SUM or (static) APSP.

Fully dynamic algorithms. The reference point in fully dynamic APSP with subpolynomial query time is the *exact* algorithm of Demetrescu and Italiano [DI04] with update time $\tilde{O}(n^2)$ (with log-factor improvements by Thorup [Tho04]). For undirected graphs, several fully dynamic distance oracles have been developed. In particular, Bernstein [Ber09] developed a distance oracle of stretch $2+\epsilon$ (for any given constant $0 < \epsilon \leq 1$) and update time $O(m^{1+o(1)})$. In the regime of stretch at least 3, tradeoffs between stretch and update time have been developed by Abraham, Chechik, and Talwar [ACT14], and by Forster, Goranci, and Henzinger [FGH21]. Finally, most fully dynamic algorithm with update time sensitive to the edge density can be combined with a fully dynamic spanner algorithm leading to faster update time at the cost of a multiplicative increase in the stretch, see [BKS12] for the seminal work on fully dynamic spanners.

2 High-Level Overview

In this section we provide a high-level overview of our algorithms. First we describe our $(2+\epsilon)$-APSP algorithms for weighted graphs, by giving a simpler static version first. Then we describe our $(2+\epsilon, W_{u,v})$-APSP algorithm, where we introduce a notion of *bunch overlap threshold* to overcome the challenge of dynamically maintaining a well-known static adaptive sampling technique [TZ01] (see Section 2.2). We give a reduction from mixed approximations to purely multiplicative approximations, which together with the previous result gives our $(2+\epsilon)$-APSP algorithm for unweighted graphs. Finally, we describe our $(1+\epsilon, 2(k-1))$-APSP algorithm for any $k \geq 2$ in unweighted graphs.

2.1 $(2+\epsilon)$-APSP for Weighted Graphs

We first present a warm-up static algorithm, that we later turn into a dynamic algorithm.

2.1.1 Static 2-APSP

We start by reviewing the concepts of bunch and cluster as defined in the seminal distance oracle construction of [TZ05].

Let $0 < p < 1$ be a parameter and A a set of nodes sampled with probability p. For each node u, the pivot $p(u)$ is the closest node in A to u. The *bunch* $B(v)$ of a node v is the set $B(v) := \{u \in V : d(u,v) < d(v,p(v))\}$ and the *clusters* are $C(u) := \{v \in V : d(u,v) < d(v,p(v))\} = \{v \in V : u \in B(v)\}$. Hence bunches and clusters are reverse of each other: $u \in B(v)$ if any only if $v \in C(u)$.

These structures have been widely used in various settings, including the decremental model (e.g. [HKN16, RZ11, ŁN22, Che18]). We can use these existing decremental algorithms to maintain bunches and clusters, but will need to develop new decremental tools to get our desired tradeoffs that we will describe in Section 2.1.2.

Using the well-known distance oracles of [TZ05], we know that by storing the clusters and bunches and the corresponding distances inside them, we can query 3-approximate distances for any pair. At a high-level, for querying distance between a pair u, v we either have $u \in B(v)$ or $v \in B(u)$ and so we explicitly have stored their distance, or we can get a 3-approximate estimate by computing $\min\{d(u,p(u)) + d(p(u),v), d(v,p(v)) + d(p(v),u)\}$. In the following we explain that in some special cases we can use these estimates to a obtain 2-approximation for a pair u, v, and in other cases by storing more information depending on how $B(u)$ and $B(v)$ overlap we can also obtain a 2-approximation to $d(u,v)$.

Let us assume that for all nodes v we have computed $B(v)$ and have access to all the distances from v to each node $u \in B(v)$. Also assume that we have computed the distances from the set A to all nodes. We now describe how using this information we can get a 2-approximate estimate of *any* pair of nodes u, v in one case. In another case we will discuss what other estimates we need to precompute. Let π be the shortest path between u and v. We consider two cases depending on how the bunches $B(u)$ and $B(v)$ interact with π. A similar case analysis was used in [CHDKL21, DP22] in distributed approximate shortest paths algorithms.

1. There exists $w \in \pi$ such that $w \notin B(u) \cup B(v)$ (left case in Figure 1): Since w is on the shortest path, we either have $d(w,u) \leq d(u,v)/2$ or $d(v,w) \leq d(u,v)/2$. Suppose $d(w,u) \leq d(u,v)/2$. We observe that by definition $d(u,p(u)) \leq d(u,w)$, hence we obtain $d(u,p(u)) + d(p(u),v) \leq d(u,p(u)) + d(u,p(u)) + d(u,v) \leq 2d(u,v)$. The case that $d(v,w) \leq d(u,v)/2$ is analogous, and hence by computing $\min\{d(u,p(u)) + d(p(u),v), d(v,p(v)) + d(p(v),u)\}$ we get a 2-approximation.
2. There exists no $w \in \pi$ such that $w \notin B(u) \cup B(v)$. In other words, $B(u) \cap B(v) \cap \pi = \emptyset$ and there exists at least one edge $\{u', v'\}$ on π where $u' \in B(u)$ and $v' \in B(v)$ (right case in Figure 1[3]): in this case we can find the minimum over estimates obtained through all such pairs u', v', i.e. by computing $d(u,u') + w(u',v') + d(v',v)$.

If we had access to all the above distances for the bunches and the pivots, we could then use them to query 2-approximate distances between any pair $u, v \in V$. As we will see, in our algorithms we only have *approximate bunches and pivots* which lead to $(2+\epsilon)$-approximate queries.

[3] Although the picture implies $B(u)$ and $B(v)$ are disjoint, this also includes the case where they overlap, or even where $u \in B(v)$ or $v \in B(u)$.

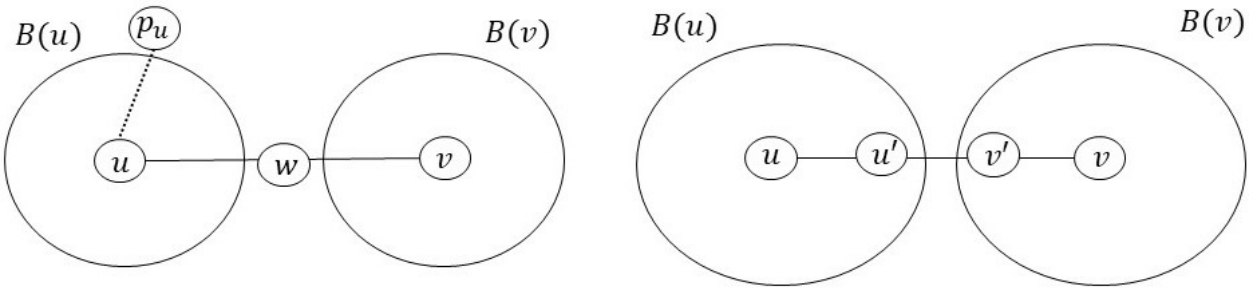


Figure 1: Possible scenarios for the overlap between the bunches $B(u)$, $B(v)$ and the shortest path π from u to v.

Running time. We can compute bunches and clusters in $\tilde{O}(\frac{m}{p})$ time [TZ05]. We need to compute shortest paths from A for the estimates in Case 1. This takes $O(|A|m) = \tilde{O}(pnm)$ time using Dijkstra's algorithm. Finally, we consider Case 2. A straight-forward approach is to consider each pair of vertices u, v, and all $u' \in B(u)$ and $v' \in B(v)$. This takes $O(\frac{n^2}{p^2})$ time.

We use a sophisticated intermediate step that computes the distance between u' and v if there is an edge $\{u', v'\} \in E$ such that $v' \in B(v)$. For each node $v \in V$, we consider all $v' \in B(v)$, for which we consider all neighbors $u' \in N(v')$ and compute the estimate $w(u', v') + d(v', v)$ (and replace current $u' - v$ minimum if it is smaller). Since $|B(v)| = \tilde{O}(\frac{1}{p})$, we can compute these estimates for $d(u', v)$ in $\tilde{O}(\frac{n^2}{p})$ time.

Next, for each pair $u, v \in V$, we take the minimum over the distances $d(u, u') + d(u', v)$ for all u' in the bunch $B(u)$ using the precomputed distances from the previous step in time $\tilde{O}(n^2/p)$, by iterating over all pairs $u, v \in V$, and all $u' \in B(u)$. So in total the adjacent case takes time $\tilde{O}(\frac{n^2}{p})$. Hence the total running time of the algorithm will be $\tilde{O}(pnm+m/p+n^2/p) = \tilde{O}(pnm + n^2/p)$. By setting $p = \sqrt{\frac{n}{m}}$ we obtain the total update time $\tilde{O}(m^{1/2}n^{3/2})$ as stated in Theorem 1.1, for details see Section 4.4.

There are several subtleties that make it more difficult to maintain these estimates in decremental settings since the bunches and which bunches are adjacent keep on changing over the updates. We next discuss how these can be handled. We obtain a total update time that matches the stated static running time (up to subpolynomial factors).

2.1.2 Dynamic Challenges

Maintaining bunches and pivots. First, we need to dynamically maintain bunches efficiently as nodes may join and leave a bunch throughout the updates using an adaptation of prior work. One option would be maintaining the clusters and bunches using the [RZ12] framework, however using this directly is slow for our purposes. Hence we maintain *approximate* clusters and bunches using hopsets of [ŁN22]. This algorithm also maintains approximate pivots, i.e. pivots that are within $(1+\epsilon)$-approximate distance of the true closest sampled node. These estimates let us handle the first case (up to $(1+\epsilon)$ approximation). One subtlety is that the type of approximate bunches used in [ŁN22] is slightly different with the type of approximate bunches we need for other parts of our algorithm. Specifically we need to bound the number of times nodes in a bunch can change. Roughly speaking, we can show this since the graph is decremental and the estimates obtained from the algorithm of [ŁN22] are monotone. Hence we perform *lazy bunch updates*, where we only let a bunch grow if the distance to the pivot grows by at least a factor $1 + \epsilon$. This means a bunch grows

at most $O(\log_{1+\epsilon}(nW))$ times. In addition, these approximate bunches need to be taken into account into our stretch analysis.

We note that $(1+\epsilon)$-approximate decremental SSSP takes $\tilde{O}(m^{1+o(1)})$ total update time. However, we use the multi-source shortest path result from [ŁN22], where the $n^{o(1)}$-term vanishes when applied with polynomially many sources, giving us a total update time of $\tilde{O}(|A|m)$.

Maintaining estimates for Case 2. When we are in Case 2, we need more tools to keep track of estimates going through these nodes since both the bunches and the distances involved are changing. In particular, we explain how by using heaps in different parts of our algorithm we get efficient update and query times. Moreover, note that we have an additional complication: there are two data structures in this step, where one impacts the other. We need to ensure that an update in the original graph does not lead to many changes in the first data structure, which all need to be processed for the second data structure.

To be precise, we need intermediate data structures $Q_{u',v}$ that store approximate distances for u' and v of the form $w(u',v') + \delta(v',v)$ for each $v' \in B(v)$ such that $\{u',v'\} \in E$, as explained for the static algorithm[4]. As opposed to the static algorithm, we do not only keep the minimum, but instead maintain a min-heap, from which the minimum is easily extracted.

However, this approach has a problem: $\delta(v',v)$ might change many times, and for each such change we need to update at most $|N(v')| \leq n$ heaps. To overcome this problem, we use another notion of *lazy distance update*: we only update a bunch if the estimate of a node changes by at least a factor $1+\epsilon$. Combining this with the fact that, due to the lazy ***bunch*** update, nodes only join a cluster at most $O(\log_{1+\epsilon}(nW))$ times, and thus there is only an $O(\log_{1+\epsilon}(nW)\log(nW))$ overhead in maintaining these min-heaps $Q_{u',v}$ dynamically due to bunch updates.

Similarly, $w(u',v')$ might change many times, which has an impact for every $v \in C(v')$. Since this can be many nodes, we need to use an additional trick so an adversary cannot increase our update time to mn. Again the solution is a lazy update scheme: instead of $w(u',v')$ we use $\tilde{w}(u',v') = (1+\epsilon)^{\lceil \log_{1+\epsilon} w(u',v') \rceil}$, which can only change $\log_{1+\epsilon} W$ times. This comes at the cost of a $(1+\epsilon)$ approximation factor.

We note that our bunches are approximate bunches in three different ways. We have one notion of approximation due to the fact that we are using hopsets (that implicitly maintain bunches on scaled graphs). We have a second notion of approximation due to our lazy bunch update, which only lets nodes join a bunch a bounded number of times. We have a third notion of approximation due to the lazy distance update, which only propagates distance changes $O(\log(nW))$ times. We need to carefully consider how these different notions of approximate bunch interact with each other and with the stretch of the algorithm.

Next, we want to use the min-heaps $Q_{u',v}$ to maintain the distance estimates for $u,v \in V$ (see again the right case in Figure 1). We construct min-heaps $Q'_{u,v}$, with for each pair u,v entries $\delta(u,u') + \delta(u',v)$ for $u' \in B(u)$. Here $\delta(u',v)$ is the minimum from the intermediate data structure $Q_{u',v}$.

An entry $\delta(u,u') + \delta(u',v)$ in $Q'_{u,v}$ has to be updated when either $\delta(u,u')$ or $\delta(u',v)$ change. We have to make sure we do not have to update these entries too often. For the first part of each entry, we bound updates to the distance estimates $\delta(u,u')$ by the lazy distance updates. One challenge here is that when the distance estimates in a bunch change we need to update all the impacted heaps i.e. if $\delta(u,u')$ inside a bunch $B(u)$ changes, we must update all the

[4]In this section, $\delta(\cdot,\cdot)$ denotes $(1+\epsilon)$-approximate distances in our dynamic data structures.

heaps $Q'_{u,v}$ such that there exists $\{u', v'\} \in E$ with $v' \in B(v)$. For this purpose one approach is the following: for each $u \in V, u' \in B(u)$ we keep an additional data structure $\text{Set}_{u,u'}$ that stores for which $v \in V$ we have $v' \in B(v)$ with $\{u', v'\} \in E$. This means that given a change in $\delta(u, u')$, we can update each of the heaps in logarithmic time.

For the second part of the entry, which is the distance estimate $\delta(u', v)$ from the first data structure, this is more complicated. Another subtlety for our decremental data structures is the fact that $\delta(u', v)$ may also decrease due to a new node being added to $B(v)$. However, our lazy bunch update ensures that $\delta(u', v)$ can only decrease $O(\log_{1+\epsilon}(nW))$ times. Further, we use lazy distance increases *in between* such decreases. Hence in total we propagate changes to $\delta(u', v)$ at most $O(\log_{1+\epsilon}(nW)\log(nW))$ times, and thus our decremental algorithm has $\tilde{O}(\log_{1+\epsilon}(nW)\log(nW))$ overhead compared to the static algorithm.

2.2 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs

For a moment, let us go back to the first static algorithm. It turns out, that if instead of computing an estimate for two 'adjacent' bunches, we keep an estimate for bunches that overlap in at least one node, we obtain a $(2, W_{u,v})$-approximation, where $W_{u,v}$ is the maximum weight on the shortest path from u to v.

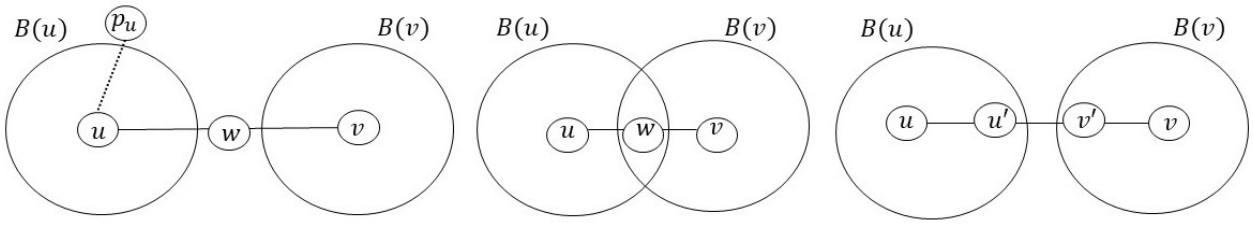


Figure 2: Possible scenarios for the overlap between the bunches $B(u)$ and $B(v)$ and the shortest path π from u to v.

The stretch analysis comes down to the following cases:

- There exists no $w \in \pi \cap B(u) \cap B(v)$ (the left and right case in Figure 2).
- There exists $w \in \pi$ such that $w \in B(u) \cap B(v)$ (the middle case in Figure 2).

The crucial observation here is that the distance estimate via the pivot (Case 1 in the previous section) already gives a $(2, W_{u,v})$-approximation for the adjacent case (the right case in Figure 2), see Section 5.1 for the details. So if we maintain an estimate for $d(u, v)$ if there exists $w \in \pi$ such that $w \in B(u) \cap B(v)$ (the 'overlap case'), then we obtain a $(2, W_{u,v})$-approximation in total.

Here we need to create a min-heap for the overlap case, later we show we can maintain in $\tilde{O}(nC_{\max}/p)$ time. First we focus on the challenge of bounding $C_{\max}$.

Efficiency challenge. While the bunches are small, there is no bound on the size of the clusters. To overcome this issue in the static case, Thorup and Zwick developed an alternative way to build the bunches and clusters, that guarantees that the clusters are also small [TZ01]. At a high-level, their approach is to adaptively change (grow) A by sampling big clusters into smaller clusters using a sampling rate proportional to the cluster size and adding these nodes to the set of pivots A. In total, in $\tilde{O}(m/p)$ time we obtain bunches and clusters, *both* of size $\tilde{O}(1/p)$. This idea was developed in the context of obtaining compact

routing schemes, and later found numerous applications in static algorithms for approximating shortest paths and distances (see e.g., [BK10, AR20, BRS+21, AR21]).

Using this trick, the static algorithm has running time $\tilde{O}(\frac{m}{p}+pnm+\frac{n}{p^2}) = \tilde{O}(nm^{3/4}+m^{4/3})$, for $p = m^{-1/3}$ or $\tilde{O}(mn^{1/2}+n^2)$ for $p = n^{-1/2}$.

However, the adaptive sampling procedure does not seem to be well-suited to a dynamic setting. The reason is that the sampling procedure is adaptive in such a way that when the bunches/clusters change, the set of sampled nodes can also change. Although (a dynamic adaptation of) the algorithm can guarantee that the set of sampled nodes is small at any particular moment, there is no guarantee on the *monotonicity* of the set A of sources that we need to maintain distances from. This would add a significant amount of computation necessary to propagate these changes to other parts of the algorithm. A possible solution is to enforce monotonicity by never letting nodes leave A. This would require us to bound the *total* number of nodes that would ever be sampled, which seems impossible with the current approach. To overcome this, we next suggest a new approach that gives us the monotonicity needed for our dynamic algorithm. For simplicity, we first present this in a static setting.

Introducing bunch overlap thresholds. Our proposed approach is to divide the nodes into two types depending on the number of bunches they appear in. In particular, we consider a threshold parameter τ and we define a node to be a *heavy node* if it is in more than τ bunches (equivalently its cluster contains more than τ nodes), and otherwise we call it a *light node*. In other words, for the set of light nodes we have $C_{\max} \leq \tau$. This threshold introduces a second type of pivot for each node u, which we denote by $q(u)$ that is defined to be the closest heavy node to u. In particular, instead of computing all distance estimates going through nodes $w \in B(u) \cap B(v)$, we only compute them for the case that w is light, and otherwise it is enough to compute the minimum estimates going through the heavy pivots, i.e. $\min\{d(u,q(u)) + d(q(u),v), d(v,q(v)) + d(q(v),u)\}$. We emphasize that these heavy pivots are *not a subset* of our original pivots $p(v), v \in V$, and they have a different behavior in bounding the running time than the *bunch pivots*. As we will see later, introducing these thresholds is crucial in efficiently obtaining the estimates required for Case 2.2 as we can handle the light and heavy case separately.

Stretch of the algorithm with bunch overlap thresholds. As discussed above, the stretch analysis was divided into two cases. In the new variant of the algorithm we add a third case, where we look at the distance estimates going through heavy pivots, i.e. $\min\{d(u,q(u)) + d(q(u),v), d(v,q(v)) + d(q(v),u)\}$. The main idea is that now we only need to consider the difficult case, Case 2.2, if the relevant node is light, which allows us to implement the algorithm efficiently.

If the relevant node $w \in B(u) \cap B(v)$ in Case 2 is heavy, we obtain a 2-approximation through the heavy pivot: we get that the $\min\{d(u,w), d(v,w)\} \leq d(u,v)/2$, since $d(u,v) = d(u,w) + d(w,v)$. W.l.o.g. say $d(u,w) \leq d(u,v)/2$. Then $d(u,q(u)) \leq d(u,w) \leq d(u,v)/2$, hence $d(u,q(u)) + d(q(u),v) \leq d(u,q(u)) + d(q(u),u) + d(u,v) \leq 2d(u,v)$.

Maintaining estimates through heavy nodes. For the stretch analysis above, we need to maintain a shortest path tree from each heavy node. We ensure a bunch grows at most $O(\log_{1+\epsilon}(nW))$ times. Hence by a total load argument we get that the *total* number of nodes in all bunches over all updates is at most $O(\frac{n}{p}\log_{1+\epsilon}(nW))$. This implies there cannot be too many heavy nodes in total, hence we can enforce monotonicity by keeping in V_{heavy} all

nodes that were once heavy (for details see Lemma 4.1). As a consequence, we can maintain multi-source approximate distances from all heavy nodes efficiently: we can compute shortest paths from this set to all other nodes in $O(|V_{\text{heavy}}|m) = O(\frac{nm}{p\tau})$ time.

Running time of the algorithm with bunch overlap thresholds. Summarizing, we compute bunches and clusters in $O(m/p)$ time, compute distances from A to V in $O(|A|m) = \tilde{O}(pnm)$ time, and we can compute shortest paths from ththe heavy nodes in $O(|V_{\text{heavy}}|m) = O(\frac{nm}{p\tau})$ time.

Finally, we consider Case 2.2. For each node $u \in V$, we consider every light node $w \in B(u)$, and then all nodes $v \in C(W)$. Since w is light, we have $|C(w)| \leq \tau$, so this takes time $\tilde{O}(n \cdot \frac{1}{p} \cdot \tau)$.

Hence, in the total update time, we obtain $\tilde{O}((pnm + \frac{nm}{p\tau} + n\tau/p)\log^2(nW))$. Setting $p = m^{-1/4}$ and $\tau = m^{1/2}$ gives a total update time of $\tilde{O}(nm^{3/4}\log(nW))$. We provide the details in Section 5.

Again for a decremental algorithm, we need to combine this new idea with the subtleties we had in the previous section: the bunches, their overlaps, and the heavy nodes keep on changing over the updates. We obtain a total update time that matches the stated static running time (up to subpolynomial factors) for the algorithm using bunch overlap thresholds.

2.2.1 $(2+\epsilon)$-APSP for Unweighted Graphs

We observe that in unweighted graphs mixed approximation can always be turned into a multiplicative approximation – at the cost of a blow-up in the number of vertices. More precisely, we prove the following reduction.

Theorem 2.1. *Let $\mathcal{A}$ be an algorithm that provides a $(a+\epsilon, k)$-approximation for APSP in $\tau_{\mathcal{A}}(n', m')$ time, for any unweighted n'-node m'-edge graph G', and any constants $a, k \in \mathbb{N}_{\geq 1}$ and $\epsilon \in [0, 1)$. Given an unweighted graph $G = (V, E)$ on n nodes and m vertices, we can compute $(a + (k+2)\epsilon, 0)$-APSP in $\tau_{\mathcal{A}}(n + km, (k+1)m)$ time.*

If $\mathcal{A}$ is a dynamic algorithm, then each update takes $(k+1) \cdot$ [update time of $\mathcal{A}$] time. The query time remains the same (up to a constant factor).

The result is obtained by splitting every edge into k edges, by introducing new nodes on the edge. Now a path corresponding to a $+k$ approximation cannot take a non-trivial detour.

We combine this result with Theorem 1.2 to obtain a $(2+\epsilon)$-approximation for unweighted graphs in $\tilde{O}(m^{7/4})$ total update time. For details, see Section 5.2.

2.3 $(1+\epsilon, 2(k-1))$-APSP for Unweighted Graphs

In this section, we describe our near-additive APSP algorithm. Our work is inspired by a classic result of Dor, Halperin, and Zwick [DHZ00], that presented a static algorithm that computes purely additive $+2(k-1)$ approximation for APSP in $\tilde{O}(n^{2-1/k}m^{1/k})$ time in unweighted graphs. Our goal is to obtain similar results dynamically. More concretely, we obtain decremental $(1+\epsilon, 2(k-1))$-approximate APSP in $O(n^{2-1/k+o(1)}m^{1/k})$ total update time in unweighted graphs.

2.3.1 $(1+\epsilon, 2)$-APSP

To explain the high-level idea of the algorithm, we first focus on the special case that $k = 2$, and that we only want to approximate the distances between pairs of nodes at distance at

most d from each other, for a parameter d. In this case, we obtain a $+2$-additive approximation in $\tilde{O}(n^{3/2}m^{1/2}d)$ time. This case already allows to present many of the high-level ideas of the algorithm. Later we explain how to extend the results to the more general case. We start by describing the static algorithm from [DHZ00], and then explain the dynamic version. The static version of the data structure is as follows:

- Let $s_1 = (\frac{m}{n})^{1/2}$. Let V_1 be the set of *dense nodes*: $V_1 := \{v \in V : \deg(v) \geq s_1\}$. Also let E_2 be the set of *sparse edges*, i.e. edges with at least one endpoint with degree less than s_1.
- **Node set D_1.** Construct a hitting set D_1 of nodes in V_1. This means that every node $v \in V_1$ has a neighbor in D_1. The size of D_1 is $O(n \log n/s_1)$.
- **Edge set E^*.** Let E^* be a set of size $O(n)$ such that for each $v \in V_1$, there exists $u \in D_1 \cap N(v)$ such that $\{u, v\} \in E^*$.
- **Computing distances from D_1.** Store distances $D_1 \times V$ by running a BFS from each $u \in D_1$ on the input graph G.
- **Computing distances from $V \setminus D_1$.** For each $u \in V \setminus D_1$ store a shortest path tree, denoted by T_u rooted at u by running Dijkstra on $(V, E_2 \cup E^* \cup E_u^{D_1})$, where $E_u^{D_1}$ is the set of weighted edges corresponding to distances in $(\{u\} \times D_1)$ computed in the previous step.

Dynamic data structure. The static algorithm computes distances in two steps. First, it computes distances from D_1 by computing BFS trees in the graph G. This step can be maintained dynamically by using *Even-Shiloach trees (ES-trees)* [ES81], a data structure that maintains distances in a decremental graph. Maintaining the distances up to distance $O(d)$ takes $O(|D_1|md)$ time. The more challenging part is computing distances from $V \setminus D_1$. Here the static algorithm computes distances in the graphs $H_u = (V, E_2 \cup E^* \cup E_u^{D_1})$. Note that the graphs H_u change dynamically in several ways. First, in the static algorithm, the set E_2 is the set of light edges. To keep the correctness of the dynamic algorithm, every time that a node v no longer has a neighbor in D_1, we should add all its adjacent edges to E_2. Second, when an edge in E^* is deleted from G, we should replace it by an alternative edge if such an edge exists. Finally, the weights of the edges in $E_u^{D_1}$ can change over time, when the estimates change. This means that other than deletions of edges, we can also add new edges to the graphs H_u or change the weights of their edges. Because of these edge insertions we can no longer use the standard ES-tree data structure to maintain the distances, because that only works in a decremental setting.

To overcome this, we use *monotone ES-trees*, a generalization of ES-trees proposed by [HKN16, HKN14]. In this data structure, to keep the algorithm efficient, when new edges are inserted, the distance estimates do not change. In particular, when edges are inserted, some distances may decrease, however, in such cases the data structure keeps an old larger estimate of the distance. The main challenge in using this data structure is to show that the stretch analysis still holds. In particular, in our case, the stretch analysis of the static algorithm was based on the fact the distances from the nodes $V \setminus D_1$ are computed in the graphs H_u, where in the dynamic setting, we do not have this guarantee anymore.

Stretch analysis. The high-level idea of the stretch analysis uses the special structure of the graphs H_u. To prove that the distance estimate between u and v is at most $d(u, v) + 2$, we distinguish between two cases. First, we maintain the property that as long as v has a

neighbor $x \in D_1$, then an edge of the form $\{v, x\}$ is in E^* for $x \in D_1 \cap N(v)$, and since we maintain correctly the distances from D_1, we can prove that we get an additive stretch of at most 2 in this case.

In the second case, all the edges adjacent to v are in $E_2 \subseteq H_u$, and here we can use an inductive argument on the length of the path to prove that we get an additive stretch of at most 2. Note that the estimate that we get can be larger compared to the distance between u and v in the graph H_u, but we can still show that the additive approximation is at most 2 as needed. For a detailed stretch analysis, see Section 6.2.2.

Update time. Our update time depends on the size of the graphs H_u. While in the static setting it is easy to bound the number of edges in E_2 and E^*, in the decremental setting, as new edges are added to these sets, we need a more careful analysis. For example, in the static setting $|E^*| = O(n)$, as any node only adds one adjacent edge to the set E^*, where in the decremental setting, we may need to add many different edges to this set because the previous ones got deleted. However, we can still prove that even if a node needs to add to E^* edges to all its adjacent neighbors in D_1 during the algorithm, the size of H_u is small enough as needed. A detailed analysis of the update time appears in Section 6.2.3, where we show that the total update time is $\tilde{O}(m^{1/2}n^{3/2}d)$ in expectation, this matches the static complexity up to the factor d.

Handling large distances. The algorithm described above is efficient if d is small, to obtain an efficient algorithm for the general case, we combine our approach with a decremental near-additive APSP algorithm. More concretely, we use an algorithm from [HKN14] that allows to compute $(1+\epsilon, \beta)$-approximation for APSP in $O(n^{2+o(1)})$ total update time, for $\beta = n^{o(1)}$. Now we set $d = \Theta(\beta/\epsilon)$, and distinguish between two cases. For pairs of nodes at distance at most d from each other, we get a $+2$-additive approximation as discussed above, in $O(m^{1/2}n^{3/2+o(1)})$ time. For pairs of nodes at distance larger than d, the near-additive approximation is already a $(1 + O(\epsilon))$-approximation by the choice of the parameter d. For such pairs, the additive β term becomes negligible, as their distance is $\Omega(\beta/\epsilon)$. Overall, we get a $(1+\epsilon, 2)$-approximation for APSP in $O(m^{1/2}n^{3/2+o(1)})$ time. In fact, if the graph is dense enough ($m = n^{1+\Omega(1)}$), the $n^{o(1)}$ term is replaced by a poly-logarithmic term. For more details see Section 6.3.4.

2.3.2 General k

We can extend the algorithm to obtain a $(1+\epsilon, 2(k-1))$-approximate APSP in $O(n^{2-1/k+o(1)}m^{1/k})$ total update time, by adapting the general algorithm of [DHZ00] to the dynamic setting. At a high-level, instead of having one hitting set D_1, we have a series of hitting sets $D_1, D_2, ..., D_k$, such that the set D_i hits nodes of degree s_i. We compute distances from D_i in appropriate graphs H_u^i, that are sparser when the set D_i is larger. Balancing the parameters of the algorithm leads to the desired total update time. For full details and proofs see Section 6.3.

2.3.3 Limitations of previous approaches

We next explain why approaches used in previous decremental algorithms cannot be generalized to obtain our near-additive results. First, the unweighted $(1+\epsilon, 2)$-approximate APSP algorithm that takes $\tilde{O}(n^{5/2}/\epsilon)$ time [HKN16], is inspired by static algorithms for $+2$-additive emulators, sparse graphs that preserve the distances up to a $+2$-additive stretch. The main idea is to maintain a sparse emulator of size $\tilde{O}(n^{3/2})$, and exploit its sparsity to obtain a fast algorithm. This construction however is specific for the $+2$ case, and cannot be generalized to a general k. Note that the update time of the algorithm crucially depends on the size of the

emulator, and to obtain a better algorithm for a general k, we need to be able to construct a sparser emulator for a general k. However, a beautiful lower bound result by Abboud and Bodwin [AB17] shows that for any constant k, a purely additive $+k$ emulator should have $\Omega(n^{4/3-\epsilon})$ edges. This implies that dynamic algorithms that are based on purely additive emulators seem to require $\Omega(n^{7/3-\epsilon})$ time, and we cannot get running time arbitrarily close to $O(n^2)$. An alternative approach is to build a *near-additive* emulator. This is done in [HKN14], where the authors show a $(1+\epsilon, 2(1+2/\epsilon)^{k-2})$-approximation algorithm for decremental APSP with expected total update time of $\tilde{O}(n^{2+1/k}(37/\epsilon)^{k-1})$. Here the running time indeed gets closer to $O(n^2)$, but this comes at a price of a much worse additive term of $2(1+2/\epsilon)^{k-2}$. While in the static setting there are also other bounds obtained for near-additive emulators, such as $O(1+\epsilon, O(k/\epsilon)^{k-1})$ emulators of size $O(kn^{1+\frac{1}{2^{k+1}-1}})$ [EP04, TZ06, ABP18, EN20], in all of the constructions the additive term depends on ϵ, and the dependence on ϵ is known to be nearly tight for small k by lower bounds from [ABP18]. Hence this approach cannot lead to small constant additive terms that do not depend on ϵ.

Outline. The detailed description of our $(2+\epsilon)$-APSP algorithms for weighted graphs can be found in Section 4. In Section 5, we provide the algorithm for $(2+\epsilon, W_{u,v})$-APSP. In Section 5.2, we provide our reduction for purely multiplicative APSP and the result for $(2+\epsilon)$-APSP for unweighted graphs. And finally in Section 6, we describe our $(1+\epsilon, 2(k-1))$-APSP algorithm for any $k \geq 2$ in unweighted graphs.

3 Preliminaries

3.1 Notation and Terminology

Throughout this paper, neighbors of a node v are denoted by $N(v)$. We denote by $d_G(u,v)$ the distance between u and v in the graph G. If G is clear from the context, we use the notation $d(u,v)$. We denote W for the ratio between the maximum and minimum weight of the graph.

Dynamic graph algorithms. We are interested in designing dynamic graph algorithms for shortest path problems, which allow both for update and query operations. In particular, we look at *decremental* algorithms, which allow for weight increases and edge deletions. This is opposed by *fully dynamic* algorithms, which also allow for weight decreases and edge insertions. To be concrete, we allow for the following three operations:

- `Delete`(u,v): delete the edge $\{u,v\}$ from the graph;
- `Update`(u,v,Δ): increase the weight of an edge $\{u,v\}$ to Δ, i.e., $w(u,v) \leftarrow \Delta$;
- `Distance`(u,v): return a distance approximation $\hat{d}_G(u,v)$ between the nodes u and v in the current graph G.

We assume that we have an *oblivious adversary*, i.e., that the sequence of updates is fixed from the start. This is as opposed to the *adaptive adversary*, which can use the random choices made by the algorithm to determine the next update.

Edge weights can be positive integers bounded by W, for some parameter $W \in \mathbb{N}$. We say that `Distance`(u,v) returns (α,β)-*approximate distances*, denoted by $\hat{d}_G$, if $d_G(u,v) \leq \hat{d}_G(u,v) \leq \alpha d_G(u,v) + \beta$ for all $u,v \in V$. Moreover we denote $(\alpha,0)$-approximate by α-approximate.

In all our algorithms, we maintain a distance oracle with constant *query time*, i.e., the time needed to answer a distance query `Distance(u,v)`. Further, we give the *total update time*, which is the time needed to process a sequence of up to m edge deletions or weight increases, where m is the number of edges before the first deletion.

For any object, value, or data structure x that changes over time, we use a superscript $x^{(t)}$ to specify its state at time t, i.e., $x^{(t)}$ is x after processing t updates.

3.2 Decremental Tools

We start by describing several tools from previous work that we need to use in our new decremental algorithm.

Cluster and bunch maintenance. Recall Thorup and Zwick [TZ05] clusters as follows: Given a parameter $p \in [0,1]$, called the *cluster sampling rate*, we let A be a set of sampled nodes, where each node is sampled uniformly at random with probability p. Then we define the *pivot* of a node v to be the closest node in A, i.e., $p(v) = \arg\min_{s \in A} d(s,v)$.

The following lemma is implicitly proven in [ŁN22], but for completeness we sketch a proof in Appendix B.

Lemma 3.1. *Given a decremental graph $G = (V, E, w)$, and parameters $0 < \epsilon < 1$, $0 < \rho < 1/2$ and the sampling probability $0 < p < 1$ we can maintain for each $v \in V$ an approximate pivot $\tilde{p}(v)$, such that $d_G(v, p(v)) \leq \delta_{\mathtt{B}}(v, \tilde{p}(v)) \leq (1+\epsilon) d_G(v, p(v))$. Moreover, we maintain approximate bunches $\tilde{B}(v)$ that contains $\hat{B}(v) := \{w \in V : (1+\epsilon) d_G(v,w) < d_G(v, p(v))\}$ and we have monotone (non-decreasing over updates) distance estimates $\delta_{\mathtt{B}}$ such that for any $u \in \tilde{B}(v)$ we have $d_G(u,v) \leq \delta_{\mathtt{B}}(u,v) \leq (1+\epsilon) d_G(u,v)$, and with high probability, we have $|\tilde{B}(v)| = O(\frac{\log n \log nW}{p})$. The algorithm has total update time $\tilde{O}(\frac{mn^\rho}{p}) \cdot O(\frac{1}{\rho\epsilon})^{1+2/\rho}$.*

In our algorithms for balancing out different terms in the running time we set the parameter ρ depending on the graph density so that $m = n^{1+\rho}$. Hence for graphs with $m = n^{1+\Omega(1)}$ edges, we can set ρ to be a constant and for sparser graphs we can set $\rho = \frac{\sqrt{\log n}}{\log \log n}$. For simplicity of our statements in the rest of this section we assume $0 < \epsilon < 1$ is a constant, and thus we hide $(1/\epsilon)^c$ factors in $\tilde{O}$ when c is constant, or in $n^{o(1)}$ when $c = \omega(1)$.

Multi-source shortest paths. We will use the following lemma (from [ŁN22]) repeatedly to maintain $(1+\epsilon)$ approximate shortest paths from a given set of sources.

Lemma 3.2 ([ŁN22]). *There is a data structure which given a weighted undirected graph $G = (V, E)$ explicitly maintains $(1+\epsilon)$-approximate distances from a set of sources S in G under edge deletions, where $0 < \epsilon < 1/2$ is a constant. After each update, the algorithm returns all nodes for which the distance estimate has changed. During the execution of the algorithm, nodes can be added to S. We write s for the final size of S. Assuming that $|E| = n^{1+\Omega(1)}$ and $s = n^{\Omega(1)}$, the total update time is w.h.p. $\tilde{O}(sm)$. If $|E| = n^{1+o(1)}$ or $s = n^{o(1)}$, then the total update time is $\tilde{O}(smn^{o(1)})$.*

4 $(2+\epsilon)$-Approximate APSP for Weighted Graphs

Given an undirected, weighted graph $G = (V, E, w)$, our goal is to maintain a $(2+\epsilon)$-approximate all-pair distances in G, denoted by $\hat{d}$. We provide an algorithm $\mathtt{APSP}_2$ with parameters $0 < p, \epsilon \leq 1$, and $0 < \rho \leq 1$ that gives a $(2+\epsilon)$-approximation of APSP for weighted graphs, with ratio between the maximum and minimum weight W. Overall, the total update time is

$\tilde{O}((pnm + \frac{n^2}{p}) \log^2(nW))$, for an exact statement see Lemma 4.7. The algorithm has constant query time. We provide a high-level algorithm in Section 4.1. Then we give further details and parametrized running time analysis of each step in Section 4.2 and the stretch analysis in Section 4.3. In Section 4.4, we combine these and set the parameter p appropriately to obtain our final result.

4.1 Decremental Data Structures

We will make use of the $(1+\epsilon_2)$-approximate bunches and clusters that are introduced in Section 3.2. However, for efficiency purposes, we do not use these bunches themselves, but instead look at bunches with some additional approximation. The goal is to ensure that the bunches do not increase too often.

Let A be a set of sampled nodes with a probability p. Recall that, for each node v, the *pivot* $p(v)$ is the closest node in A to v.

Lemma 3.1 gives us an approximate pivot $\tilde{p}(v)$, together with a distance estimate $\delta_{\mathbf{B}}$, such that $d_G(v, p(v)) \leq \delta_{\mathbf{B}}(v, \tilde{p}(v)) \leq (1+\epsilon_2) d_G(v, p(v))$. Further, we have $(1+\epsilon_2)$-approximate bunches: $\tilde{B}(v) := \{w \in V : (1+\epsilon_2) d(v,w) < \delta_{\mathbf{B}}(v, \tilde{p}(v))\}$, and distance estimate $\delta_{\mathbf{B}}$ such that $d(u,v) \leq \delta_{\mathbf{B}}(u,v) \leq (1+\epsilon_2) d(u,v)$ for all $u \in \tilde{B}(v)$.

We now define our final approximate bunch, which we denote by $B(v)$[5].

Definition 1. We need the following notions related to bunches and clusters:

- For all $v \in V$, we keep track of a $(1+\epsilon_3)$-approximate *bunch radius* $r(v)$, which is a $(1+\epsilon_3)$-approximation of the approximate distance to the approximate pivot, i.e., of $\delta_{\mathbf{B}}(v, \tilde{p}(v))$. We denote by $r^{(t)}$ the bunch radius at time t. Initially, we set $r^{(0)}(v) := \delta_{\mathbf{B}}(v, \tilde{p}(v))$. We update $r^{(t)}(v)$ whenever $\delta_{\mathbf{B}}(v, \tilde{p}(v)) > (1+\epsilon_3) r^{(t-1)}(v)$, if so then we set $r^{(t)}(v) := \delta_{\mathbf{B}}(v, \tilde{p}(v))$.
- Initially we set $B^{(0)}(v) := \tilde{B}^{(0)}(v)$ and whenever $r^{(t)}(v) \neq r^{(t-1)}(v)$ we set $B^{(t)}(v) := \tilde{B}^{(t)}(v)$.
- The *cluster* of a node $u \in V$ is the set $C(u) := \{v \in V : u \in B(v)\}$.

Note that these bunches $B(v)$ increase at most $\log_{1+\epsilon_3}(nW)$ times by construction. Next, let us investigate the sizes of these sets.

Lemma 4.1. *At any given time*

(i) With high probability, A has size $\tilde{O}(pn)$;

(ii) With high probability, each bunch $B(v)$ has size $\tilde{O}(\frac{1}{p})$;

Proof. (i) This follows immediately from the definition of A.

(ii) This holds trivially as $B(v) \subseteq \tilde{B}(v)$, and $|\tilde{B}(v)| = \tilde{O}(\frac{1}{p})$ at any moment in time due to Lemma 3.1. □

Now, we give a description of the algorithm, which is a dynamic version of the static algorithm given in the high-level overview, Section 2.1. Next, we give a proof of correctness

[5] In the high-level overview $B(v)$ has been used for the exact bunches. Since exact bunches do not occur in our actual algorithm, $B(v)$ will denote these approximate bunches from here on.

and show how to implement each of the steps. We provide a separate initialization procedure, as for some steps it will turn out that the initialization step and the update steps are quite different.

Throughout the algorithm, we set $\tilde{\delta} := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta) \rceil}$ whenever we want a distance estimate δ to update at most $\log_{1+\epsilon_4}(nW)$ times. Note that we can always maintain such a $\tilde{\delta}$ with only a constant multiplicative factor in the total update time. Moreover, it is clear that $\delta \leq \tilde{\delta} \leq (1+\epsilon_4)\delta$. Below we define the functions `Initialization()`, and functions used after each edge update $\{u,v\} \in E$: `Delete`(u,v), `Update`$(u,v,w(u,v))$. Finally we have the `Query()` function that returns a final estimate over different data structures maintained.

Algorithm $\texttt{APSP}_2(V,E,w,p,\epsilon)$

`Initialization()`:

Set $\epsilon_1 = \epsilon_2 = \epsilon_3 = \epsilon_4 = \epsilon/3$.

1. **Sampling.** Sample each node with probability p to construct A.
2. **Distances from** A**.** Compute $(1+\epsilon_1)$-approximate distances $\delta_A(s,v)$ for $s \in A$ and $v \in V$.
3. **Bunches and clusters.** Initialize the algorithm for maintaining the approximate bunches $\tilde{B}$ and pivots $\tilde{p}$ according to Lemma 3.1, denoting with $\delta_{\mathbb{B}}(v,w)$ the $(1+\epsilon_2)$-approximate distances for $w \in \tilde{B}(v)$.
   Initialize $(1+\epsilon_3)$-approximate bunch radii $r(v)$ with respect to these approximate pivots $\tilde{p}(v)$. We initialize another approximation to the bunches, denoted by $B(v)$, which will be defined with respect to the bunch radius $r(v)$. Initially, we simply set $B(v) = \tilde{B}(v)$. We let $\delta_B(u,v) := \delta_{\mathbb{B}}(u,v)$, and set $\tilde{\delta}_B(u,v) := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta_B(u,v)) \rceil}$.
4. **Neighboring bunch data structure.** For $x \in V$ and $v \in V$ such that there exists $y \in N(x) \cap B(v)$ initialize a Min-Heap $Q^{\text{nbr}-\text{bunch}}_{x,v}$, with an entry for *each* $y \in N(x) \cap B(v)$. The key for the Min-Heap is the distance estimate $\tilde{w}(x,y) + \tilde{\delta}_B(y,v)$. Moreover, to make sure the minimal entry, denoted by $\delta^{\text{nbr}-\text{bunch}}(x,v)$, does not change too often, we maintain an additional approximation: $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v) := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta^{\text{nbr}-\text{bunch}}(x,v)) \rceil}$.
5. **Adjacent data structure.** For $u,v \in V$ initialize a Min-Heap $Q^{\text{adjacent}}_{u,v}$, with an entry for each $\{u',v'\} \in E$ such that $u' \in B(u)$ and $v' \in B(v)$. The key for the Min-Heap is the distance estimate $\tilde{\delta}_B(u,u') + \tilde{\delta}^{\text{nbr}-\text{bunch}}(u',v)$.

`Delete`(u,v):

`Update`(u,v,∞)

`Update`$(u,v,w(u,v))$:

(1) **Distances from** A**.** For all $s \in A$, run `Update`$(u,v,w(u,v))$ on the SSSP approximation denoted by $\delta_A(s,\cdot)$.

(2) **Update bunches and heaps.** Maintain the bunches and clusters, and update the heaps accordingly. See below for the details.

`Query`(u,v):

Output $\hat{d}(u,v)$ to be the minimum of

(b) $\min\{\delta_{\mathbb{B}}(u,\tilde{p}(u)) + \delta_A(v,\tilde{p}(u)), \delta_A(u,\tilde{p}(v)) + \delta_{\mathbb{B}}(v,\tilde{p}(v))\}$;

(c) The minimum entry of $Q^{\text{adjacent}}_{u,v}$, which gives $\min\{\tilde{\delta}_B(u,u') + \tilde{\delta}^{\text{nbr}-\text{bunch}}(u',v) : u' \in B(u)\}$.

Update bunches and heaps (Update Step (2)). Now let us look more closely on how we maintain the bunches and clusters, and how these effect the heaps. We maintain the approximate bunches $\tilde{B}(v)$ for all $v \in V$ by Lemma 3.1. From here maintain the additional approximate bunches $B(v)$ for all $v \in V$ as given in Definition 1, i.e., If there is a change to $\delta_{\mathbb{B}}(v', \tilde{p}(v'))$, do the following two things:

- We check if ${\delta_{\mathbb{B}}}^{(t)}(v', \tilde{p}^{(t)}(v')) > (1+\epsilon_3) r^{(t-1)}(v')$, and if so we set $r^{(t)}(v') = {\delta_{\mathbb{B}}}^{(t)}(v', p^{(t)}(v'))$ and we set $B^{(t)}(v) := \tilde{B}^{(t)}(v)$. If not, we set $r^{(t)}(v') = r^{(t-1)}(v')$ and $B^{(t)}(v) = B^{(t-1)}(v)$.

Let $u' \in V$ be a node for which there is a change to the bunch involving a node $w \in V$. Such a change can be: (i) w can leave the bunch of u' (ii) the distance estimate of w to u' can change while w stays within the bunch, or (iii) w can join the bunch of u'. We formalize this as follows, using Δ to denote which case we are in.

(i) $w \in B^{(t-1)}(u')$ and $w \notin B^{(t)}(u')$, we set $\Delta = \infty$ to indicate that w is no longer in $B(u')$.

(ii) $w \in B^{(t-1)}(u')$ and $w \in B^{(t)}(u')$ with an increased distance estimate $\tilde{\delta}^{(t)}_B(u', w)$, we set $\Delta = \tilde{\delta}^{(t)}_B(u', w)$ to be the new distance.

(iii) $w \notin B^{(t-1)}(u')$ and $w \in B^{(t)}(u')$, we set $\Delta = \tilde{\delta}^{(t)}_B(u', w)$ to be the distance.

Next, we do the following.

- `UpdateNbrBunchDSBunchChange`(w, u', ∞) to update the corresponding entries $\tilde{w}(v', w) + \tilde{\delta}_B(w, u')$ in the data structures $Q^{\text{nbr}-\text{bunch}}_{v',u'}$ for every $v' \in \text{Set}^{\text{nbr}-\text{bunch}}_{w,u'}$. If this changes the minimal entry $\delta^{\text{nbr}-\text{bunch}}(x,y)$ in $Q^{\text{nbr}-\text{bunch}}_{x,y}$ for some $x, y \in V$, we check whether we need to update $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,y)$, if so we run `UpdateAdjacentDSMinHeapChange`$(x, y, \tilde{\delta}^{\text{nbr}-\text{bunch}}(x,y))$.
- `UpdateAdjacentDS`(w, u', Δ) to update the corresponding entries $\tilde{\delta}_B(u', w) + \tilde{\delta}^{\text{nbr}-\text{bunch}}(w, v')$ in the data structures $Q^{\text{adjacent}}_{u',v'}$ for every $v' \in \text{Set}^{\text{adj}-\text{bunch}}_{u',w}$.

4.2 Further Details and Time Analysis

In this section, we add further details of the procedures described and analyze the total update time.

Sampling. Note that A remains unchanged throughout the algorithm, hence it is only sampled once at the initialization. This takes $O(n)$ time.

Distances from A. We use the decremental shortest path algorithm from Lemma 3.2 together with Lemma 4.1, which bounds the size of A, to obtain the following lemma.

Lemma 4.2. *With high probability, we can maintain $(1+\epsilon_2)$-approximate SSSP from each node in A, providing us with $\delta_A(s,v)$ for $s \in A$ and $v \in V$. We can do this in time $\tilde{O}(pnm)$ if $m = n^{1+\Omega(1)}$ and $pn = n^{\Omega(1)}$, and in time $\tilde{O}(pn^{1+o(1)}m)$ otherwise.*

Bunches and clusters. We have approximate bunches $\tilde{B}(v)$ and pivots $\tilde{p}(v)$ by Lemma 3.1, now we maintain the bunch radii $r(v)$ and approximate bunches $B(v)$ and clusters $C(v)$ according to Definition 1. As before, we simply set $\tilde{\delta}_B(u,v) := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta_B(u,v))\rceil}$. It is immediate that these extra steps do not incur more than a constant factor in the running time with respect to Lemma 3.1, so we obtain the following result.

Lemma 4.3. *We can maintain $(1+\epsilon_3)$-approximate bunch radii $r(v)$ with respect to the approximate pivots $\tilde{p}$. With high probability, we can maintain approximate bunches $B(v)$ and clusters $C(v)$, and distance estimates $\delta_B(v,u)$ for $u \in B(v)$, for all nodes $v \in V$. Moreover, we can maintain $\tilde{\delta}_B(u,v)$, a $(1+\epsilon_4)$-approximation of $\delta_B(u,v)$ that updates at most $O(\log_{1+\epsilon_4}(nW))$ times. This takes total update time $\tilde{O}(\frac{mn^\rho}{p})$, for some arbitrarily small constant ρ, if $m = n^{1+\Omega(1)}$, and $\tilde{O}(\frac{mn^{o(1)}}{p})$ otherwise.*

Neighboring bunch data structure. To maintain $Q^{\text{adjacent}}_{u,v}$, we maintain an auxiliary data structure, which is another Min-Heap $Q^{\text{nbr}-\text{bunch}}_{x,v}$. This data structure consists of the following ingredients. We say $B(v)$ is a *neighboring (approximate) bunch* of x if there exists an edge from x to $B(v)$. For every node x, and every neighboring bunch $B(v)$ our goal is to store all neighbors y such that $y \in B(v) \cap N(x)$ together with $\tilde{w}(x,y) + \tilde{\delta}_B(y,v)$, which allows us to find the neighbor y minimizing this sum for each neighboring bunch[6]. Later, we will combine this with the distance estimate $\tilde{\delta}_B(u,x)$ for $u \in B(x)$ to get a distance estimate from u to v, which will be stored in $Q^{\text{adjacent}}_{u,v}$. To make sure we do not have to update the entries in $Q^{\text{adjacent}}_{u,v}$, we do not maintain the minimum of $Q^{\text{nbr}-\text{bunch}}_{x,v}$ exactly, but instead we maintain an additional approximation, denoted $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v)$ that only changes at most a polylogarithmic number of times.

Lemma 4.4. *With high probability, we can maintain the data structures $Q^{\text{nbr}-\text{bunch}}_{x,v}$ for all $x,v \in V$ in total update time $\tilde{O}(\frac{n^2}{p} \cdot (\log_{1+\epsilon_4}(nW)) + \log_{1+\epsilon_3}(nW)))$. Moreover, in the same total update time, we can maintain an approximation $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v)$ of the minimum entry of $Q^{\text{nbr}-\text{bunch}}_{x,v}$ that only changes at most $\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW)$ times for each pair (x,v).*

Proof. This data structure is initialized as follows:

- Iterate over every vertex $v \in V$, and over all $y \in B(v)$. Then iterate over all neighbors $x \in N(y)$ of y and add an entry to the data structure $Q^{\text{nbr}-\text{bunch}}_{x,v}$ with key $\tilde{w}(x,y) + \tilde{\delta}_B(y,v)$.

  This takes $\tilde{O}(\frac{n^2}{p})$ initialization time, since there are n options for v, it has bunch size $\tilde{O}(\frac{1}{p})$, so $\tilde{O}(\frac{1}{p})$ options for y, which has at most n neighbors.

We need to facilitate two types of updates for an entry of $Q^{\text{nbr}-\text{bunch}}_{x,v}$ corresponding to an edge $\{x,y\}$: increases in the weight $\tilde{w}(x,y)$ and increases in the distance estimates $\tilde{\delta}_B(y,v)$. We define the functions `UpdateNbrBunchDSEdgeChange` and `UpdateNbrBunchDSBunchChange` respectively.

Next, we consider how to update this data structure. Suppose a value $\tilde{\delta}_B(y,v)$ changed for some $y \in B(v)$ due to an update to the bunches (Lemma 4.3), then we would need to change the corresponding entries in $Q^{\text{nbr}-\text{bunch}}_{x,v}$. Lemma 4.3 gives us y, v, and the new distance $\tilde{\delta}_B(y,v)$, which we need to update in all data structures $Q^{\text{nbr}-\text{bunch}}_{x,v}$ for which $\{x,y\} \in E$.

[6]Recall that we denote $\tilde{w}$ for $(1+\epsilon_4)^{\lceil \log_{1+\epsilon_4} w\rceil}$.

To do this efficiently, we maintain a set $\text{Set}_{x,y}^{\text{nbr}-\text{bunch}-\text{edge}}$ for each edge $\{x, y\} \in E$ that stores for which $v \in V$ there is an entry in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and a pointer to where in that data structure it appears. We equip this set with the structure of a self-balancing binary search tree, so we have polylogarithmic insertion and removal times. We update this every time the appearance or location in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ changes. Now if $\tilde{w}(x, y)$ changes due to an update, we can efficiently change the corresponding places in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ for all relevant $v \in V$.

Moreover, for the changes in the distances from the bunches, we maintain a set $\text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$ for each $v \in V, y \in B(v)$ that stores for which $x \in V$ there is an entry in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and a pointer to where in that data structures it appears. We equip this set with the structure of a self-balancing binary search tree, so we have polylogarithmic insertion and removal times. We update this every time the appearance or location in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ changes. Now if $\tilde{\delta}_B(y, v)$ changes due to an update, we can efficiently change the corresponding places in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ for all relevant $x \in V$.

We define the function `UpdateNbrBunchDSEdgeChange`(x, y, Δ), where $\{x, y\} \in E$, and Δ is the new value $\tilde{w}^{(t)}(x, y)$, or ∞ if the edge $\{x, y\}$ is deleted. Depending on Δ, we do one of the following:

(i) If $\Delta < \infty$, then we update the corresponding estimates:

- For each entry $v \in \text{Set}_{x,y}^{\text{nbr}-\text{bunch}-\text{edge}}$, update the corresponding entry $\tilde{w}(x, y) + \tilde{\delta}_B(y, v)$ in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and update the entry in $\text{Set}_{x,y}^{\text{nbr}-\text{bunch}-\text{edge}}$ accordingly.

(ii) If $\Delta = \infty$, then we remove the corresponding estimates:

- For each entry $v \in \text{Set}_{x,v}^{\text{nbr}-\text{bunch}-\text{edge}}$, remove the corresponding entry in $Q_{x,v}^{\text{nbr}-\text{bunch}}$.

Concerning the total update time of `UpdateNbrBunchDSEdgeChange`, we recall that $\tilde{w}(x, y) = (1 + \epsilon_4)^{\lceil \log_{1+\epsilon_4}(w(x,y)) \rceil}$. This means that each weight $\tilde{w}(x, y)$ gets updated at most $\log_{1+\epsilon_4}(W)$ times, giving total update time $\tilde{O}(\frac{n^2}{p} \log_{1+\epsilon_4}(W)$.

We define the function `UpdateNbrBunchDSBunchChange`(y, v, Δ), where y is an element of $B^{(t-1)}(v)$ and/or $B^{(t)}(v)$, and Δ is the new value $\tilde{\delta}_B^{(t)}(y, v)$, or ∞ if y is no longer in $B(v)$. `UpdateNbrBunchDSBunchChange` does one of three things:

(i) If $\Delta < \infty$, $y \notin B^{(t-1)}(v)$, and $y \in B^{(t)}(v)$, then we add the corresponding new estimates:

- We initialize $\text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$ to be empty.
- For all neighbors $x \in N(y)$ of y, we add $\tilde{w}(x, y) + \tilde{\delta}_B(y, v)$ to $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and store x together with its location in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ in $\text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$.

(ii) If $\Delta < \infty$, $y \in B^{(t-1)}(v)$, and $y \in B^{(t)}(v)$, then we update the corresponding estimates:

- For each entry $x \in \text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$, update the corresponding entry $\tilde{w}(x, y) + \tilde{\delta}_B(y, v)$ in $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and update the entry in $\text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$ accordingly.

(iii) If $\Delta = \infty$, $y \in B^{(t-1)}(v)$, and $y \notin B^{(t)}(v)$, then we remove the corresponding estimates:

- For each entry $x \in \text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$, remove the corresponding entries from $Q_{x,v}^{\text{nbr}-\text{bunch}}$ and from $\text{Set}_{y,v}^{\text{nbr}-\text{bunch}}$.

Moreover, if one of these three actions change the minimum of $Q^{\text{nbr}-\text{bunch}}_{x,v}$, this has an impact on the data structures $Q^{\text{adjacent}}_{\cdot,v}$. We run `UpdateAdjacentDSMinHeapChange`$(x, v, \Delta)$, which is defined and analyzed in Lemma 4.5, to propagate this change. Here Δ is the minimum of $Q^{\text{nbr}-\text{bunch}}_{x,v}$. The total update time of `UpdateAdjacentDSMinHeapChange` is included in Lemma 4.5, and not in this lemma.

As $\tilde{\delta}_B(y, v)$ can change at most $O(\log_{1+\epsilon_4}(nW))+\log_{1+\epsilon_3}(nW))$ times, and for each time we need at most $\tilde{O}(\deg(y)) = O(n)$ time to update the data structure, we get at most $\tilde{O}(\deg(y)(\log_{1+\epsilon_4}(nW))+\log_{1+\epsilon_3}(nW)))$ total update time for the pair (y, v). Now note that there are at most $\tilde{O}(\frac{n}{p})$ such pairs, since there are n options for v, with $\tilde{O}(\frac{1}{p})$ options for $y \in B(v)$. So we get total update time $\tilde{O}(\frac{n^2}{p} \cdot (\log_{1+\epsilon_4}(nW)) + \log_{1+\epsilon_3}(nW)))$.

Further, we maintain $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$, a $(1+\epsilon_4)$-approximation of the minimal entry $\delta^{\text{nbr}-\text{bunch}}(x, v)$ of $Q^{\text{nbr}-\text{bunch}}_{x,v}$, defined by $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v) := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta^{\text{nbr}-\text{bunch}}(x,v))\rceil}$. Note that $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$ is *not* monotone over time, so it is not trivial how often the estimate changes. However, note that $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$ increases at most $\log_{1+\epsilon_4}(nW)$ times in a row. It can only decrease $\log_{1+\epsilon_3}(nW)$ times, since the values $\tilde{w}(x, y) + \tilde{\delta}_B(y, v)$ are non-decreasing, except if new vertices $y \in B^{(t)}(v)$ are considered. This is only the case when the bunch $B^{(t)}(v)$ grows, which happens at most $\log_{1+\epsilon_3}(nW)$ times by Lemma 4.3. □

Adjacent data structure. Using $Q^{\text{nbr}-\text{bunch}}_{x,v}$, we construct the required Min-Heap $Q^{\text{adjacent}}_{u,v}$ for each pair $u, v \in V$. For every pair of nodes $u, v \in V$ our goal is to store all edges $\{x, y\}$ such that $x \in B(u)$ and $y \in B(v)$, together with the distance $\tilde{\delta}_B(u, x) + \tilde{w}(x, y) + \tilde{\delta}_B(y, v)$.

Lemma 4.5. *With high probability, we can maintain the data structures $Q^{\text{adjacent}}_{u,v}$ for all $u, v \in V$ in total update time $\tilde{O}(\frac{n^2}{p} \log_{1+\epsilon_4}(nW) \log_{1+\epsilon_3}(nW))$.*

Proof. This data structure is initialized as follows:

- Iterate over all nodes u, all $x \in B(u)$, and all neighboring bunches $B(v)$ of x and add an entry for x to $Q^{\text{adjacent}}_{u,v}$ with key $\tilde{\delta}_B(u, x) + \tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$.

  As there are at most n bunches in total, this takes time $\tilde{O}(n \cdot \frac{1}{p} \cdot n) = \tilde{O}(\frac{n^2}{p})$.

Again, we maintain additional data structures to facilitate updates: for each $x, v \in V$, we maintain $\text{Set}^{\text{adjacent}}_{x,v}$ which shows for which $u \in V$ there is an entry in $Q^{\text{adjacent}}_{u,v}$ and a pointer to where in that data structure it appears. We update this every time the appearance or location in $Q^{\text{adjacent}}_{u,v}$ changes. Now if $\tilde{\delta}_B(u, w)$ changes due to an update, we can efficiently change the corresponding places in $Q^{\text{adjacent}}_{u,v}$.

We define the update function `UpdateAdjacantDSBunchChange` analogously to the function `UpdateNbrBunchDSBunchChange`. `UpdateAdjacantDSBunchChange`(u, x, Δ) does one of the following three things:

(i) If $\Delta < \infty$, $x \notin B^{(t-1)}(u)$, and $x \in B^{(t)}(u)$, then we add the corresponding new estimates:

- We initialize $\text{Set}^{\text{adj}-\text{bunch}}_{u,x}$ to be empty.
- For all nodes with non-empty $\tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$, we add $\tilde{\delta}_B(u, x) + \tilde{\delta}^{\text{nbr}-\text{bunch}}(x, v)$ to $Q^{\text{adjacent}}_{u,v}$ and store v together with its location in $Q^{\text{adjacent}}_{u,v}$ in $\text{Set}^{\text{adj}-\text{bunch}}_{u,x}$.

(ii) If $\Delta < \infty$, $x \in B^{(t-1)}(u)$, and $x \in B^{(t)}(u)$, then we update the corresponding estimates:

- For each entry $v \in \text{Set}_{u,x}^{\text{adj}-\text{bunch}}$, update the corresponding entry $\tilde{\delta}_B(u,x)+\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v)$ in $Q_{u,v}^{\text{adjacent}}$ and update the entry in $\text{Set}_{u,x}^{\text{adj}-\text{bunch}}$ accordingly.

(iii) If $\Delta = \infty$, $x \in B^{(t-1)}(u)$, and $x \notin B^{(t)}(u)$, then we remove the corresponding estimates:

- For each entry $v \in \text{Set}_{u,x}^{\text{adj}-\text{bunch}}$, remove the corresponding entries from $Q_{u,v}^{\text{adjacent}}$ and from $\text{Set}_{u,x}^{\text{adj}-\text{bunch}}$.

As updates in bunches/distance estimates in bunches happen at most $O(\log_{1+\epsilon_4}(nW)+\log_{1+\epsilon_3}(nW))$ times, see Lemma 4.1, we get total update time $\tilde{O}(\frac{n^2}{p} \cdot (\log_{1+\epsilon_4}(nW) + \log_{1+\epsilon_3}(nW)))$.

We also define the function `UpdateAdjacentDSMinHeapChange` to process changes in $\tilde{\delta}^{\text{nbr}-\text{bunch}}(\cdot,\cdot)$. `UpdateAdjacentDSMinHeapChange`$(x,v,\Delta)$ does one of the following three things:

(i) If $\Delta < \infty$ and the distance estimate from x to v is new, then we add the corresponding new estimates:

- We initialize $\text{Set}_{x,v}^{\text{adj}-\text{heap}}$ to be empty.
- For all $u \in C(x)$, we add $\tilde{\delta}_B(u,x) + \tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v)$ to $Q_{u,v}^{\text{adjacent}}$ and store u together with its location in $Q_{x,v}^{\text{adjacent}}$ in $\text{Set}_{x,v}^{\text{adj}-\text{heap}}$.

(ii) If $\Delta < \infty$ and there already was a distance estimate from x to v, then we update the corresponding estimates:

- For each entry $x \in \text{Set}_{x,v}^{\text{adj}-\text{heap}}$, update the corresponding entry $\tilde{\delta}_B(u,x)+\tilde{\delta}^{\text{nbr}-\text{bunch}}(x,v)$ in $Q_{u,v}^{\text{adjacent}}$ and update the entry in $\text{Set}_{x,v}^{\text{adj}-\text{heap}}$ accordingly.

(iii) If $\Delta = \infty$ and there is no longer a distance estimate from x to v, then we remove the corresponding estimates:

- For each entry $x \in \text{Set}_{x,v}^{\text{adj}-\text{heap}}$, remove the corresponding entries from $Q_{u,v}^{\text{adjacent}}$ and from $\text{Set}_{x,v}^{\text{adj}-\text{heap}}$.

Since each $\tilde{\delta}^{\text{nbr}-\text{bunch}}(\cdot,\cdot)$ updates at most $\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW)$ times by Lemma 4.4, we get total update time $\tilde{O}(\frac{n^2}{p}\log_{1+\epsilon_4}(nW))$ for this operation.

Together, `UpdateNbrBunchDSBunchChange` and `UpdateAdjacantDSBunchChange` form the function `UpdateAdjacentDS`. To be precise, we define `UpdateAdjacentDS` as follows:
`UpdateAdjacentDS`(u,v,Δ):

1. `UpdateNbrBunchDSBunchChange`(u,v,Δ)
2. `UpdateAdjacantDSBunchChange`(u,v,Δ)

This means we obtain total update time

$$\tilde{O}(\tfrac{n^2}{p}(\log_{1+\epsilon_4}(nW)+\log_{1+\epsilon_3}(nW)+\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW))) = \tilde{O}(\tfrac{n^2}{p}\log_{1+\epsilon_4}(nW)\log_{1+\epsilon_3}(nW)).$$

□

Query time. Finally, we observe that the query is relatively straightforward and can be done in constant time.

Lemma 4.6. $\texttt{APSP}_2$ *has constant query time.*

Proof. A query `Query(u,v)` consists of the minimum over four values. The first two can be computed in constant time, as the components are all maintained explicitly. The last two are maintained explicitly. This gives us constant query time in total. □

We summarize this section in the following lemma.

Lemma 4.7. *With high probability, the algorithm* $\texttt{APSP}_2$ *has total update time* $\tilde{O}((pnm + \frac{n^2}{p})\log^2(nW))$ *if* $m = n^{1+\Omega(1)}$, $p = n^{\Omega(1)-1}$, *and* $m \leq n^{2-\rho}$ *for an arbitrarily small constant* ρ. *If* $m \geq n^{2-\rho}$, *we have total update time* $\tilde{O}((pnm + \frac{mn^\rho+n^2}{p})\log^2(nW))$. *Otherwise we have total running time* $\tilde{O}((pnm + \frac{n^2}{p})n^{o(1)}\log^2(nW))$.

Proof. We list the total update times as follows:

- **Sampling.** $O(n)$
- **Distances from** A**.**
  - $\tilde{O}(pnm)$ if $m = n^{1+\Omega(1)}$ and $pn = n^{\Omega(1)}$
  - $\tilde{O}(pn^{1+o(1)}m)$ otherwise.
- **Bunches and clusters.**
  - $\tilde{O}(\frac{mn^\rho}{p})$, for some arbitrarily small constant ρ, if $m = n^{1+\Omega(1)}$,
  - $\tilde{O}(\frac{mn^{o(1)}}{p})$ otherwise.
- **Neighboring bunch data structure.** $\tilde{O}(\frac{n^2}{p}(\log_{1+\epsilon_3}(nW) + \log_{1+\epsilon_4}(nW)))$
- **Adjacent data structure.** $\tilde{O}(\frac{n^2}{p}\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW))$.

This means for the case that $m = n^{1+\Omega(1)}$ and $p = n^{\Omega(1)-1}$, we have total update time

$$\tilde{O}((pnm + \tfrac{mn^\rho}{p} + \tfrac{n^2}{p})\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW)) = \tilde{O}((pnm + \tfrac{mn^\rho}{p} + \tfrac{n^2}{p})\log^2(nW)),$$

using that we hide constant factors of $1/\epsilon$ in the $\tilde{O}$-notation. If we furthermore assume that $n^\rho = O(n/p)$, i.e., $p = O(n^{1-\rho})$ for some arbitrarily small ρ, then we obtain the simplified version

$$\tilde{O}((pnm + \tfrac{n^2}{p})\log^2(nW)).$$

For other choices of $m, n,$ and p, we obtain

$$\tilde{O}((pnm + \tfrac{n^2}{p})n^{o(1)}\log^2(nW)).$$

□

4.3 Stretch Analysis

We can now follow the same structure as outlined in the high-level overview to prove the stretch in different case for each pair $u, v \in V$. In our analysis we also discuss how maintaining approximate bunches and distances impacts the final stretch.

Lemma 4.8. *For all $u, v \in V$, the distance estimate $\hat{d}(u, v)$ returned by a query to* $\texttt{APSP}_2$ *is a $(2+\epsilon)$-approximation of the distance $d(u, v)$.*

Proof. Let $u, v \in V$. We will show that at any time t, if `Query`(u, v) returns $\hat{d}^{(t)}(u, v)$, then we have $d^{(t)}(u, v) \leq \hat{d}^{(t)}(u, v) \leq (2+\epsilon)d^{(t)}(u, v)$. Since we look at all data structures at the same time t, we omit the superscripts $\cdot^{(t)}$ in the remainder of this proof.

First, let us note that $\hat{d}(u, v)$ consists of different distance estimates that all correspond to actual paths in G, hence we clearly have $d(u, v) \leq \hat{d}(u, v)$. Heaps do not contain 'old' entries, entries that correspond to paths that are no longer in G at the time of query, since the update functions `UpdateNbrBunchDSBunchChange`, `UpdateNbrBunchDSEdgeChange`, `UpdateAdjacentDSMinHeapChange`, and `UpdateAdjacantDSBunchChange` take such entries out as soon as they are no longer relevant.

Next, we do a case distinction as follows. Let π denote the shortest path from u to v.

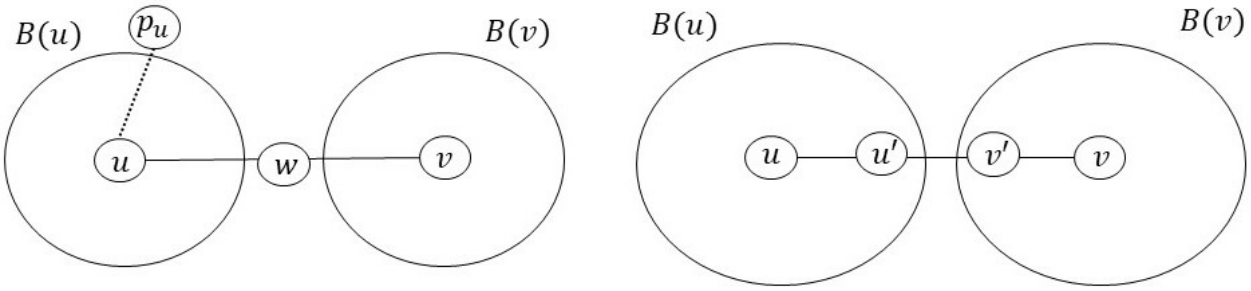


Figure 3: Possible scenarios for the overlap between the bunches $B(u)$ and $B(v)$ and the shortest path π from u to v.

Case 1: There exists $w \in \pi$ such that $w \notin (B(u) \cup B(v))$ (the left case in Figure 3).
Since $w \notin B(u)$, we have $d(u, w) \geq r(u) \geq \frac{\delta_{\mathbb{B}}(u, \tilde{p}(u))}{1+\epsilon_3}$, so $\delta_{\mathbb{B}}(u, \tilde{p}(u)) \leq (1+\epsilon_3)d(u, w)$. Similarly $\delta_{\mathbb{B}}(v, \tilde{p}(v)) \leq (1+\epsilon_3)d(w, v)$. Since $d(u, w) + d(w, v) = d(u, v)$, we have either $d(u, w) \leq \frac{d(u,v)}{2}$ or $d(v, w) \leq \frac{d(u,v)}{2}$. Without loss of generality, assume $d(u, w) \leq \frac{d(u,v)}{2}$, then by Query Step (b) in the algorithm we have:

$$\begin{aligned}
\hat{d}(u, v) &\leq \delta_{\mathbb{B}}(u, \tilde{p}(u)) + \delta_A(v, \tilde{p}(u)) \\
&\leq \delta_{\mathbb{B}}(u, \tilde{p}(u)) + (1+\epsilon_1)d(v, \tilde{p}(u)) \\
&\leq \delta_{\mathbb{B}}(u, \tilde{p}(u)) + (1+\epsilon_1)(d(u, \tilde{p}(u)) + d(u, v)) \\
&\leq \delta_{\mathbb{B}}(u, \tilde{p}(u)) + (1+\epsilon_1)(\delta_{\mathbb{B}}(u, \tilde{p}(u)) + d(u, v)) \\
&\leq (2+\epsilon_1)(1+\epsilon_3)d(u, w) + (1+\epsilon_1)d(u, v) \\
&\leq (2 + \tfrac{3}{2}\epsilon_1 + \tfrac{1}{2}\epsilon_1\epsilon_3 + \epsilon_3)d(u, v) \\
&\leq (2+\epsilon)d(u, v),
\end{aligned}$$

where in the last step we use $\epsilon_1 = \epsilon_3 = \epsilon/3$.

Case 2: There is no $w \in \pi$ such that $w \notin (B(u) \cup B(v))$ (the right case in Figure 3).
This means there has to be a direct edge from the cluster of u to the cluster of v: the shortest path π consists of a path from u to u', for some $u' \in B(u)$, an edge $\{u', v'\} \in E$, for some $v' \in B(v)$, and a path from v' to v (see the right case in Figure 3). Note that in particular this includes the cases where $u \in B(v)$ or $v \in B(u)$, or more general when $B(u) \cap B(v) \neq \emptyset$, since we do not require $u \neq u'$, $v \neq v'$, $u' \notin B(v)$, or $v' \notin B(u)$.
By Query Step (c) we have

$$\begin{aligned}
\hat{d}(u,v) &\leq \tilde{\delta}_B(u,u') + \tilde{\delta}^{\text{nbr}-\text{bunch}}(u',v) \\
&\leq (1+\epsilon_4)\delta_B(u,u') + (1+\epsilon_4)\delta^{\text{nbr}-\text{bunch}}(u',v) \\
&\leq (1+\epsilon_4)(1+\epsilon_2)d(u,u') + (1+\epsilon_4)(\tilde{w}(u',v') + \tilde{\delta}_B(v',v)) \\
&\leq (1+\epsilon_4)(1+\epsilon_2)d(u,u') + (1+\epsilon_4)^2(w(u',v') + \delta_B(v',v)) \\
&\leq (1+\epsilon_4)(1+\epsilon_2)d(u,u') + (1+\epsilon_4)^2(w(u',v') + (1+\epsilon_2)d(v',v)) \\
&\leq (1+\epsilon_4)^2(1+\epsilon_2)d(u,v) \\
&\leq (2+\epsilon)d(u,v),
\end{aligned}$$

where in the last step we use $\epsilon_2 = \epsilon_4 = \epsilon/3$, and $\epsilon < 1$. □

4.4 Putting Everything Together

In this section we combine our lemmas in the following theorem. To obtain this result, we pick p to balance the different terms in the total update time.

Theorem 1.1. *Given a weighted graph G and a constant $0 < \epsilon < 1$, there is a decremental data structure that maintains a $(2+\epsilon)$-approximation of APSP. The algorithm has constant query time and the total update time is w.h.p.*

- $\tilde{O}(m^{1/2}n^{3/2}\log^2(nW))$ *if* $m = n^{1+\Omega(1)}$ *and* $m \leq n^{2-\rho}$ *for an arbitrary small constant* ρ*,*
- $\tilde{O}(m^{1/2}n^{3/2+\rho}\log^2(nW))$ *if* $m \geq n^{2-\rho}$ *for any constant* ρ*,*
- $\tilde{O}(m^{1/2}n^{3/2+o(1)}\log^2(nW))$ *otherwise,*

where W is the ratio between the maximum and minimum weight.

Proof. This follows from Lemma 4.7 and 4.8, by picking p to minimize $pnm + \frac{n^2}{p}$. This gives $p = \sqrt{\frac{n}{m}}$, implying $pnm = \frac{n^2}{p} = m^{1/2}n^{3/2}$. Note that whenever $m = n^{1+\Omega(1)}$ and $m \leq n^{2-\rho}$, this choice of p indeed satisfies the other requirements of Lemma 4.7: $p = n^{\Omega(1)-1}$. When $m = n^{1+o(1)}$, obtain an extra factor $n^{o(1)}$ in the total update time. And when $m \geq n^{2-\rho}$, we obtain the extra factor n^{ρ}. □

5 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs and Unweighted 2-APSP

In this section, we show that an adapted version of our algorithm returns $(2+\epsilon, W_{u,v})$-approximate distances with improved update time, where $W_{u,v}$ is the maximum weight on a shortest path from u to v. This result follows by the fact that in this case we do not need to handle the adjacent bunch case, but instead can consider an *overlap* bunch case (see the middle case in Figure 4), which leads to an improved running time.

To make this efficient, there are some additional complications. We introduce a parameter τ, the *bunch overlap threshold*, which bounds the cluster size. For nodes $w \in B(u) \cap B(v)$ with cluster smaller than τ, we can compute the overlap case. If all nodes in the overlap $B(u) \cap B(v)$ have larger clusters, we obtain an approximate by computing shortest paths from the heavy nodes. More details can be found in Section 5.1.

Moreover, in Section 5.2, we show a general reduction from mixed APSP to purely multiplicative APSP for unweighted graphs. We use this reduction to obtain our unweighted $(2+\epsilon, 0)$-APSP result.

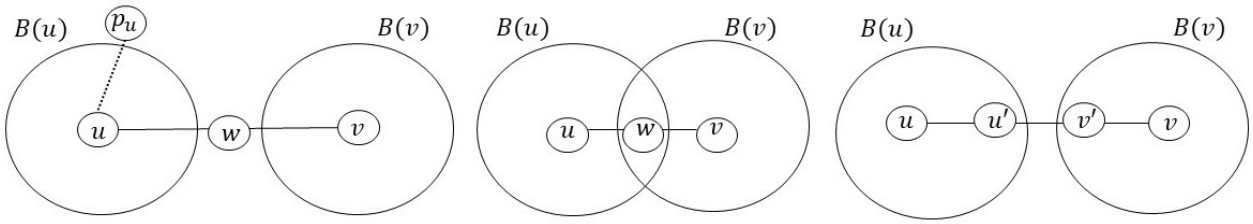


Figure 4: Possible scenarios for the overlap between the bunches $B(u)$ and $B(v)$ and the shortest path π from u to v.

5.1 $(2+\epsilon, W_{u,v})$-APSP for Weighted Graphs

We use the same structure as in Section 4: we have bunches, and clusters exactly as before. We introduce the parameter $1 \leq \tau \leq n$ to be set later. Now we define nodes to be heavy or light based on their cluster size.

- We call a node u *heavy* if it is or ever was contained in at least τ approximate bunches $B(v)$, i.e., u is heavy at time t if there is a $t' \leq t$ such that $|C^{(t)}(u)| \geq \tau$. If a node is not heavy, we call it *light*.
- Initially, we denoted V_{heavy} for the set of heavy nodes, and $V_{\text{light}} = V \setminus V_{\text{heavy}}$ for the light nodes. By definition of heavy nodes, a light node can become heavy, but a heavy node can never become light.
- Suppose we have a distance estimate $\delta_H(s,v)$ for all $s \in V_{\text{heavy}}, v \in V$. For each node $u \in V$, the *heavy pivot* $q(u)$ is the closest node in V_{heavy} to u according to the distance estimate δ_H

We can show that the size of V_{heavy} is bounded.

Lemma 5.1. *At any given time, with high probability, V_{heavy} has size $\tilde{O}(\frac{n}{p\tau}\log_{1+\epsilon_3}(nW))$.*

Proof. Let $B^{(t)}(v)$ and $C^{(t)}(w)$ denote the bunch of v and the cluster of w respectively at time t. Since bunch radius $r(v)$ increases by at least a factor $1+\epsilon_3$ if the bunch changes, we have at most $\log_{1+\epsilon_3}(nW)$ different bunch radii. Until the bunch radius increases, the bunch can only lose vertices, which does not contribute to the total size. Since each bunch is of size $\tilde{O}(\frac{1}{p})$ with high probability, we obtain that the total size of a bunch over the course of the

algorithm is bounded by $\left|\bigcup_t B^{(t)}(v)\right| = \tilde{O}\left(\frac{\log_{1+\epsilon_3}(nW)}{p}\right)$. Hence we have

$$\begin{aligned}
|V_{\text{heavy}}|\tau &\leq \sum_{w \in V_{\text{heavy}}} \tau \\
&\leq \sum_{w \in V_{\text{heavy}}} \left|\bigcup_t C^{(t)}(w)\right| \\
&\leq \sum_{w \in V} \left|\bigcup_t C^{(t)}(w)\right| \\
&= \sum_{v \in V} \left|\bigcup_t B^{(t)}(v)\right| \\
&= \tilde{O}(\tfrac{n}{p} \log_{1+\epsilon_3}(nW)).
\end{aligned}$$

Thus $|V_{\text{heavy}}| = \tilde{O}(\frac{n}{p\tau} \log_{1+\epsilon_3}(nW))$. □

Now we are ready to present our new algorithm. **Algorithm** $\texttt{APSP}_{2,W}(V, E, w, p, \tau, \epsilon)$

`Initialization():`

Set $\epsilon_1 = \epsilon_2 = \epsilon_3 = \epsilon_4 = \epsilon/3$.

1. **Sampling.** Sample each node with probability p to construct A.
2. **Distances from** A**.** Compute $(1+\epsilon_1)$-approximate distances $\delta_A(s, v)$ for $s \in A$ and $v \in V$.
3. **Bunches and clusters.** Initialize the algorithm for maintaining the approximate bunches $\tilde{B}$ and pivots $\tilde{p}$ according to Lemma 3.1, denoting with $\delta_{\mathbb{B}}(v, w)$ the $(1+\epsilon_2)$-approximate distances for $w \in \tilde{B}(v)$.
   Initialize $(1+\epsilon_3)$-approximate bunch radii $r(v)$ with respect to these approximate pivots $\tilde{p}(v)$. We initialize another approximation to the bunches, denoted by $B(v)$, which will be defined with respect to the bunch radius $r(v)$. Initially, we simply set $B(v) = \tilde{B}(v)$. We let $\delta_B(u, v) := \delta_{\mathbb{B}}(u, v)$, and set $\tilde{\delta}_B(u, v) := (1+\epsilon_4)^{\lceil \log_{1+\epsilon_4}(\delta_B(u,v)) \rceil}$. Initialize $V_{\text{heavy}} := \{w \in V : |C(w)| > \tau\}$.
4. **Distances from** V_{heavy}**.** Compute $(1+\epsilon_1)$-approximate distances $\delta_H(s, v)$ for $s \in V_{\text{heavy}}$ and $v \in V$. Further, for all $v \in V$, we initialize a Min-Heap $Q_v^{\text{heavy-pivot}}$ that stores $\delta_H(s, v)$ for $s \in V_{\text{heavy}}$. The heavy pivot $q(v)$ is the minimum entry of $Q_v^{\text{heavy-pivot}}$.
5. **Overlap data structure.** For $u, v \in V$, initialize a Min-Heap $Q_{u,v}^{\text{overlap}}$, with an entry for each $w \in B(u) \cap B(v)$ that is not contained in V_{heavy}, with key $\tilde{\delta}_B(u, w) + \tilde{\delta}_B(v, w)$.

`Delete(`u, v`):`

`Update(`u, v, ∞`)`

`Update(`$u, v, w(u, v)$`):`

(1) **Distances from** A**.** For all $s \in A$, run `Update(`$u, v, w(u, v)$`)` on the SSSP approximation denoted by $\delta_A(s, \cdot)$.

(2) **Distances from V_{heavy}.** For all $s \in V_{\text{heavy}}$, run `Update`$(u, v, w(u, v))$ on the SSSP approximation denoted by $\delta_H(s, \cdot)$, and update $Q_{\cdot}^{\text{heavy-pivot}}$ accordingly.

(3) **Update bunches and heaps.** Maintain the bunches and clusters, and update the heaps accordingly. See below for the details.

`Query`(u, v):
Output $\hat{d}(u, v)$ to be the minimum of

(b) $\min\{\delta_{\mathbb{B}}(u, \tilde{p}(u)) + \delta_A(v, \tilde{p}(u)), \delta_A(u, \tilde{p}(v)) + \delta_{\mathbb{B}}(v, \tilde{p}(v))\}$;

(c) $\min\{\delta_H(u, q(u)) + \delta_H(v, q(u)), \delta_H(u, q(v)) + \delta_H(v, q(v))\}$;

(d) The minimum entry of $Q_{u,v}^{\text{overlap}}$, which gives $\min\{\tilde{\delta}_B(u, w) + \tilde{\delta}_B(v, w) : w \in B(u) \cap B(v)$ and $w \in V_{\text{light}}\}$.

Updating bunches and heaps. For completeness, we recall how we update the bunches and clusters. We now maintain different heaps. Again, we maintain the approximate bunches $\tilde{B}(v)$ for all $v \in V$ by Lemma 3.1. From here maintain the additional approximate bunches $B(v)$ for all $v \in V$ as given in Definition 1, i.e., If there is a change to $\delta_{\mathbb{B}}(v', \tilde{p}(v'))$, do the following two things:

- We check if ${\delta_{\mathbb{B}}}^{(t)}(v', \tilde{p}^{(t)}(v')) > (1 + \epsilon_3) r^{(t-1)}(v')$, and if so we set $r^{(t)}(v') = {\delta_{\mathbb{B}}}^{(t)}(v', p^{(t)}(v'))$ and we set $B^{(t)}(v) := \tilde{B}^{(t)}(v)$. If not, we set $r^{(t)}(v') = r^{(t-1)}(v')$ and $B^{(t)}(v) = B^{(t-1)}(v)$.

Let $u' \in V$ be a node for which there is a change to the bunch involving a node $w \in V$. Such a change can be: (i) w can leave the bunch of u' (ii) the distance estimate of w to u' can change while w stays within the bunch, or (iii) w can join the bunch of u'. We formalize this as follows, using Δ to denote which case we are in.

(i) $w \in B^{(t-1)}(u')$ and $w \notin B^{(t)}(u')$, we set $\Delta = \infty$ to indicate that w is no longer in $B(u')$.

(ii) $w \in B^{(t-1)}(u')$ and $w \in B^{(t)}(u')$ with an increased distance estimate $\tilde{\delta}_B^{(t)}(u', w)$, we set $\Delta = \tilde{\delta}_B^{(t)}(u', w)$ to be the new distance.

(iii) $w \notin B^{(t-1)}(u')$ and $w \in B^{(t)}(u')$, we set $\Delta = \tilde{\delta}_B^{(t)}(u', w)$ to be the distance.

Next, we do the following. If we are in Case (iii), we check the cluster size of w: if $|C^{(t)}(w)| \geq \tau$, then we add w to V_{heavy} and we initialize a SSSP approximation, denoted by $\delta_H(w, \delta)$. Moreover, for each u' in $C^{(t-1)}(w)$ we do the following:

- `UpdateOverlapDS`(w, u', ∞) to remove the corresponding entries $\tilde{\delta}_B(u', w) + \tilde{\delta}_B(v', w)$ from the data structures $Q_{u',v'}^{\text{overlap}}$ for all $v' \in \text{Set}_{w,u'}^{\text{overlap}}$. Here $\text{Set}_{w,u'}^{\text{overlap}}$ stores exactly those v' such that there is an entry for w in $Q_{u',v'}^{\text{overlap}}$. The details on $\text{Set}^{\text{overlap}}$ can be found together with the definition and total update time of `UpdateOverlapDS` are given in Lemma 5.3.

If we are in Case (iii) but $|C^{(t)}(w)| < \tau$, or we are in Case (i) or (ii), then we do the following

- `UpdateOverlapDS`(w, u', Δ) to update the corresponding entries $\tilde{\delta}_B(u', w) + \tilde{\delta}_B(v', w)$ in the data structures $Q_{u',v'}^{\text{overlap}}$ for all $v' \in \text{Set}_{w,u'}^{\text{overlap}}$.

Distances from V_{heavy}. First, we note that a node is heavy if it is contained in at least τ bunches, or equivalently, if $|C(w)| > \tau$. Hence by Lemma 4.3, we can easily keep track when nodes become heavy and should join V_{heavy}.

Next, we use the shortest path algorithm from Lemma 3.2, which we combine with Lemma 4.1, which states that V_{heavy} might be growing over time, but its size is always bounded by $\tilde{O}(\frac{n}{p\tau}\log_{1+\epsilon_3}(nW))$. Moreover, we maintain the heavy pivots $q(v)$ by maintaining a Min-Heap $Q_v^{\text{heavy pivot}}$ for each node v, with as entries all nodes $u \in V_{\text{heavy}}$ with key $\delta_H(u,v)$. Throughout, we will use that we can extract the minimum entry of a Min-Heap in constant time, that we can insert entries in $\log(n)$ time, and that we can edit or delete entries in $\log(n)$ time, given that we know either the key or have a pointer to their location in the Min-Heap.

This clearly does not incur more than a constant factor in the running time, so together we obtain the following lemma.

Lemma 5.2. *With high probability, we can maintain $(1+\epsilon_2)$-SSSP from each node in V_{heavy}, providing us with $\delta_H(s,v)$ for $s \in V_{\text{heavy}}$ and $v \in V$. Further, for all $v \in V$, we can maintain heavy pivots $q(v)$ if $d(v,q(v)) \le d$. We can do this in total update time $\tilde{O}(\frac{mn}{p\tau}\log_{1+\epsilon_3}(nW))$ if $m = n^{1+\Omega(1)}$ and $\frac{n}{p\tau} = n^{\Omega(1)}$, and $\tilde{O}(\frac{mn^{1+o(1)}}{p\tau}\log_{1+\epsilon_3}(nW))$ otherwise.*

Overlap data structure. Now we provide the details of how we maintain the data structure $Q_{u,v}^{\text{overlap}}$ for all $u,v \in V$, which has entries for all node $w \in B(u) \cap B(v) \cap V_{\text{light}}$.

Lemma 5.3. *With high probability, we can maintain the data structures $Q_{u,v}^{\text{overlap}}$ for all $u,v \in V$ in total update time $\tilde{O}(\frac{n\tau}{p}\cdot\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW))$.*

Proof. Conceptually, we initialize a Min-Heap $Q_{u,v}^{\text{overlap}}$ for each pair $(u,v) \in V \times V$. In practice, we only create the Min-Heap if the first entry is about to be added. This avoids a $\Theta(n^2)$ initialization time. Now fix $v \in V$, and consider the moment we add a node w to $B(v)$. For this data structure, we only need to consider $w \in V_{\text{light}}$, hence w is part of at most τ bunches. When we add w to to $B(v)$, we know it is part of at most τ other bunches $B(u)$. For each such tuple (u,w,v), we add an entry to the Min-Heap $Q_{u,v}^{\text{overlap}}$ for (u,v), which stores w with as key the distance estimate $\delta_w(u,v) := \tilde{\delta}_B(u,w) + \tilde{\delta}_B(w,v)$. For the total time, notice that for each $v \in V$, we have at most $\tilde{O}(\frac{1}{p})$ different $w \in B(v)$ for which we store at most τ distances, hence we have total initialization time $\tilde{O}(n \cdot \frac{1}{p} \cdot \tau)$.

Next, we consider how to update this data structure. Suppose a value $\tilde{\delta}_B(u,w)$ changed for some $w \in B(u)$ due to an update to the bunches (Lemma 4.3), then we would need to change the corresponding entries in $Q_{u,v}^{\text{overlap}}$. Lemma 4.3 gives us u, w, and the new distance $\tilde{\delta}_B(u,w)$, which we need to update in all data structures $Q_{u,v}^{\text{overlap}}$ for which $w \in B(u) \cap B(v)$. Clearly checking all possible v and looking through the entire $Q_{u,v}^{\text{overlap}}$ is too slow. Instead, we keep an additional data structure for each node $w \in B(u)$: a set $\text{Set}_{w,u}^{\text{overlap}}$ that shows for which $v \in V$ it appears in $Q_{u,v}^{\text{overlap}}$ and a pointer to where in $Q_{u,v}^{\text{overlap}}$ it appears. We equip this set with the structure of a self-balancing binary search tree, so we have polylogarithmic insertion and removal times. We update this every time the appearance or location in $Q_{u,v}^{\text{overlap}}$ changes. Now if $\tilde{\delta}_B(u,w)$ changes due to an update, we can efficiently change the corresponding places in $Q_{u,v}^{\text{overlap}}$ as follows.

We define the function `UpdateOverlapDS`(u, w, Δ), where w is an element of $B^{(t-1)}(u)$ and/or $B^{(t)}$, and Δ is the new value $\tilde{\delta}_B^{(t)}(u, w)$, or ∞ if w is no longer in $B^{(t)}(u)$. Note that we will only call this function for light w.[7] `UpdateOverlapDS` does one of three things:

(i) If $\Delta < \infty$, $w \notin B^{(t-1)}(u)$ and $w \in B^{(t)}(u)$. We add the new estimates:

- We initialize $\text{Set}_{w,u}^{\text{overlap}}$ to be empty.
- For each $v \in C(w)$ we add $\delta_w(u, v) := \tilde{\delta}_B(u, w) + \tilde{\delta}_B(w, v)$ to $Q_{u,v}^{\text{overlap}}$ and store v together with its location in $Q_{u,v}^{\text{overlap}}$ as entries in $\text{Set}_{w,u}^{\text{overlap}}$ and $\text{Set}_{w,v}^{\text{overlap}}$.

(ii) If $\Delta < \infty$, $w \in B^{(t-1)}(u)$ and $w \in B^{(t)}(u)$. We update the estimates:

- For each entry of v in $\text{Set}_{w,u}^{\text{overlap}}$, update $\delta_w(u, v) = \tilde{\delta}_B(u, w) + \tilde{\delta}_B(w, v)$ in $Q_{u,v}^{\text{overlap}}$ and update the entries in $\text{Set}_{w,u}^{\text{overlap}}$ and $\text{Set}_{w,v}^{\text{overlap}}$ accordingly.

(iii) If $\Delta = \infty$, then $w \in B^{(t-1)}(u)$ and $w \notin B^{(t)}(u)$. We remove the corresponding estimates:

- For each entry of v in $\text{Set}_{w,u}^{\text{overlap}}$, remove $\delta_w(u, v)$ from $Q_{u,v}^{\text{overlap}}$ and remove the corresponding entries from $\text{Set}_{w,u}^{\text{overlap}}$ and $\text{Set}_{w,v}^{\text{overlap}}$.

For the total update time, notice that in total at most $\tilde{O}(\frac{n}{p} \cdot \log_{1+\epsilon_3}(nW))$ nodes get added to a bunch, see Lemma 4.1. Since we only call this function when the added node w is light at that time, we have $|C(w)| \leq \tau$ and thus step (i) takes $O(\frac{n\tau}{p} \cdot \log_{1+\epsilon_3}(nW))$ total update time. For step (ii), note that the distance can be updated at most $O(\log_{1+\epsilon_4}(nW))$ times and therefore this step has total update time $O(\frac{n\tau}{p} \cdot \log_{1+\epsilon_3}(nW) \log_{1+\epsilon_4}(nW))$. Further nodes get deleted from bunches at most as often as they get added, so the total update time of step (iii) is dominated by step (i). We obtain total update time $\tilde{O}(\frac{n\tau}{p} \cdot \log_{1+\epsilon_3}(nW) \log_{1+\epsilon_4}(nW))$. □

Update time. We summarize the running times in the following lemma.

Lemma 5.4. *With high probability, the algorithm* $\texttt{APSP}_{2,W}$ *has total update time* $\tilde{O}((pnm + \frac{mn}{p\tau} + \frac{n\tau}{p}) \log^2(nW))$ *if* $m = n^{1+\Omega(1)}$, $p = n^{\Omega(1)-1}$, $\frac{n}{p\tau} = n^{\Omega(1)}$, *and* $\tau = O(n^{1-\rho})$. *Otherwise we have total running time* $\tilde{O}((pnm + \frac{mn}{p\tau} + \frac{n\tau}{p}) n^{o(1)} \log^2(nW))$.

Proof. We list the total update times as follows:

- **Sampling.** $O(n)$
- **Distances from** A**.**
  - $\tilde{O}(pnm)$ if $m = n^{1+\Omega(1)}$ and $pn = n^{\Omega(1)}$
  - $\tilde{O}(pn^{1+o(1)}m)$ otherwise.
- **Bunches and clusters.**
  - $\tilde{O}(\frac{mn^{\rho}}{p})$, for some arbitrarily small constant ρ, if $m = n^{1+\Omega(1)}$,
  - $\tilde{O}(\frac{mn^{o(1)}}{p})$ otherwise.
- **Distances from** V_{heavy}**.**

[7] If w becomes heavy we remove all the corresponding estimates.

– $\tilde{O}(\frac{mn}{p\tau}\log_{1+\epsilon_3}(nW))$ if $m = n^{1+\Omega(1)}$ and $\frac{n}{p\tau} = n^{\Omega(1)}$

– $\tilde{O}(\frac{mn^{1+o(1)}}{p\tau}\log_{1+\epsilon_3}(nW))$ otherwise.

- **Overlap data structure.** $\tilde{O}(\frac{n\tau}{p}\cdot\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW))$.

This means for the case that $m = n^{1+\Omega(1)}$, $p = n^{\Omega(1)-1}$, and $\frac{n}{p\tau} = n^{\Omega(1)}$, we have total update time

$$\tilde{O}((pnm+\tfrac{mn^\rho}{p}+\tfrac{mn^\rho}{p}+\tfrac{mn}{p\tau}+\tfrac{n\tau}{p})\log_{1+\epsilon_3}(nW)\log_{1+\epsilon_4}(nW)) = \tilde{O}((pnm+\tfrac{mn^\rho}{p}+\tfrac{mn}{p\tau}+\tfrac{n\tau}{p})\log^2(nW)),$$

using that we hide constant factors of $1/\epsilon$ in the $\tilde{O}$-notation. For other choices of m, n, p and τ, we obtain

$$\tilde{O}((pnm + \tfrac{mn^\rho}{p} + \tfrac{mn}{p\tau} + \tfrac{n\tau}{p})n^{o(1)}\log^2(nW)).$$

If we furthermore assume that $n^\rho = O(n/\tau)$, i.e., $\tau = O(n^{1-\rho})$ for some arbitrarily small ρ, then we obtain the simplified version

$$\tilde{O}((pnm + \tfrac{mn}{p\tau} + \tfrac{n\tau}{p}\log^2(nW)).$$

For other choices of $m, n, p,$ and τ, we obtain

$$\tilde{O}((pnm + \tfrac{mn}{p\tau} + \tfrac{n\tau}{p})n^{o(1)}\log^2(nW)).$$

□

Theorem 1.2. *Given a weighted graph G and a constant $0 < \epsilon < 1$, there is a decremental data structure that maintains a $(2+\epsilon, W_{u,v})$-approximation for APSP, where $W_{u,v}$ is the maximum weight on a shortest path from u to v. The algorithm has constant query time and the total update time is w.h.p. $\tilde{O}(nm^{3/4}\log^2(nW))$ if $m = n^{1+\Omega(1)}$, and $\tilde{O}(n^{1+o(1)}m^{3/4}\log^2(nW))$ otherwise, where W is the ratio between the maximum and minimum weight.*

Proof. This result follows from a exchanging the adjacent data structure for the overlap data structure as described in this section.

By Lemma 5.4 we have terms a running time scaling with $pnm + mn/(p\tau) + n\tau/p$. To balance out these terms, we set $mn/(p\tau) = n\tau/p$, hence $\tau = \sqrt{m}$. We set $pnm = mn/(p\tau)$, hence $p = m^{-1/4}$. Note that for $m = n^{1+o(1)}$, this choice for ρ and τ also satisfies the additional constraints of Lemma 5.4, so in this case we obtain total update time $\tilde{O}((pnm + mn/(p\tau) + n\tau/p)\log^2(nW)) = \tilde{O}(nm^{3/4}\log^2(nW))$. If $m = n^{1+o(1)}$ we obtain $\tilde{O}(n^{1+o(1)}m^{3/4}\log^2(nW))$.

Stretch analysis. It remains to show that $\hat{d}(u,v)$ is a $(2+\epsilon, W_{u,v})$-approximation of the distance $d(u,v)$. We do a similar case distinction as in Lemma 4.8. Case 1 is exactly the same, since we maintain the relevant estimate through the pivots. For the remainder, we know there is no $w \in \pi$ such that $w \in (B(u) \cup B(v))$, hence the two bunches are adjacent. We know create cases depending on whether the bunches overlap, and if so, if the node in the overlap is heavy or light.

Case 2: $\pi \cap B(u) \cap B(v) = \emptyset$ (the right case in Figure 4).
Hence π consists of a path from u to $u' \in B(u)$, the edge $\{u', v'\}$ with $v' \in B(v)$, and a path from v' to v. Since $v' \notin B(u)$, we have $d(u,v') \geq r(u) \geq \frac{\delta_{\mathbb{B}}(u,\tilde{p}(u))}{1+\epsilon_3}$, so $\delta_{\mathbb{B}}(u,\tilde{p}(u)) \leq (1+\epsilon_3)d(u,v')$. Similarly, we have $\delta_{\mathbb{B}}(v,\tilde{p}(v)) \leq (1+\epsilon_3)d(u',v)$. Combining this, we obtain

$\delta_{\mathbb{B}}(u,\tilde{p}(u)) + \delta_{\mathbb{B}}(v,\tilde{p}(v)) \leq (1+\epsilon_3)(d(u',v)+d(u,v')) = (1+\epsilon_3)(d(u,v)+w(u',v'))$. Without loss of generality, assume that $\delta_{\mathbb{B}}(u,\tilde{p}(u)) \leq \frac{1+\epsilon_3}{2}(d(u,v)+w(u',v'))$. By Query Step (b) we have:

$$\begin{aligned}
\hat{d}(u,v) &\leq \delta_{\mathbb{B}}(u,\tilde{p}(u)) + \delta_A(v,\tilde{p}(u)) \\
&\leq \delta_{\mathbb{B}}(u,\tilde{p}(u)) + (1+\epsilon_1)d(v,\tilde{p}(u)) \\
&\leq \delta_{\mathbb{B}}(u,\tilde{p}(u)) + (1+\epsilon_1)(d(u,\tilde{p}(u)) + d(u,v)) \\
&\leq \delta_{\mathbb{B}}(u,\tilde{p}(u)) + (1+\epsilon_1)(\delta_{\mathbb{B}}(u,\tilde{p}(u)) + d(u,v)) \\
&\leq (2+\epsilon_1)\tfrac{1+\epsilon_3}{2}(d(u,v)+w(u',v')) + (1+\epsilon_1)d(u,v) \\
&\leq (2+\tfrac{3}{2}\epsilon_1 + \tfrac{1}{2}\epsilon_1\epsilon_3 + \epsilon_3)d(u,v) + (1+\tfrac{1}{2}\epsilon_1 + \tfrac{1}{2}\epsilon_1\epsilon_3 + \epsilon_3)w(u',v') \\
&\leq (2+\epsilon/2)d(u,v) + (1+\epsilon/2)W_{u,v} \\
&\leq (2+\epsilon)d(u,v) + W_{u,v}.
\end{aligned}$$

Case 3: There exists $w \in \pi$ such that $w \in (B(u) \cap B(v))$ and $w \in V_{\text{light}}$ (the middle case in Figure 4).
By Query Step (d), we have

$$\begin{aligned}
\hat{d}(u,v) &\leq \tilde{\delta}_B(u,w) + \tilde{\delta}_B(v,w) \\
&\leq (1+\epsilon_4)(\delta_B(u,w) + \delta_B(v,w)) \\
&\leq (1+\epsilon_4)(1+\epsilon_2)(d(u,w)+d(v,w)) \\
&= (1+\epsilon_2+\epsilon_2\epsilon_4+\epsilon_4)d(u,v) \\
&\leq (1+\epsilon)d(u,v).
\end{aligned}$$

Case 4: There exists $w \in \pi$ such that $w \in (B(u) \cap B(v))$ and $w \in V_{\text{heavy}}$ (See Figure 5).
If $w = q(u)$ then by Query Step (c), we have $\hat{d}(u,v) \leq \delta_H(u,q(u)) + \delta_H(v,q(u)) \leq (1+\epsilon_1)(d(u,q(u)) + d(v,q(u)) = (1+\epsilon_1)d(u,v) \leq (1+\epsilon)d(u,v)$. If $w = q(v)$, the argument is analogous.

If $q(u) \neq w \neq q(v)$, then by definition of $q(u)$ and $q(v)$ we obtain $\delta_H(u,q(u)) \leq \delta_H(u,w)$ and $\delta_H(v,q(v)) \leq \delta_H(v,w)$. So we have $\delta_H(u,q(u)) + \delta_H(v,q(v)) \leq (1+\epsilon_1)d(u,v)$, hence either $\delta_H(u,q(u)) \leq (1+\epsilon)\frac{d(u,v)}{2}$ or $\delta_H(v,q(v)) \leq (1+\epsilon_1)\frac{d(u,v)}{2}$. Without loss of generality, assume $\delta_H(u,q(u)) \leq (1+\epsilon_1)\frac{d(u,v)}{2}$. Combining this with the triangle inequality, we obtain

$$\begin{aligned}
\hat{d}(u,v) &\leq \delta_H(u,q(u)) + \delta_H(q(u),v) \\
&\leq \delta_H(u,q(u)) + (1+\epsilon_1)d(q(u),v) \\
&\leq \delta_H(u,q(u)) + (1+\epsilon_1)(d(u,q(u)) + d(u,v)) \\
&\leq (2+\epsilon_1)\delta_H(u,q(u)) + (1+\epsilon_1)d(u,v) \\
&\leq (2+\tfrac{5}{2}\epsilon_1 + \tfrac{1}{2}\epsilon_1^2)d(u,v) \\
&\leq (2+\epsilon)d(u,v).
\end{aligned}$$

Query time. A query `Query(u,v)` consists of the minimum over three values. The first two can be computed in constant time, as the components are all maintained explicitly. The last one is maintained explicitly. This gives us constant query time in total. □

In particular, this means we get $(2+\epsilon, 1)$-approximate APSP for unweighted graphs.

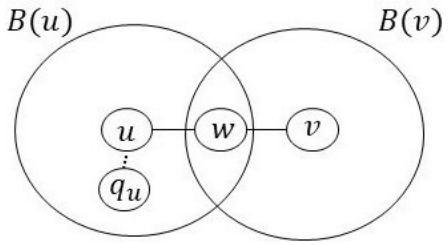


Figure 5: Possible scenario for the overlap between the bunches $B(u)$ and $B(v)$ and the shortest path π from u to v where the estimate can be obtained using heavy pivots q_u and q_v.

5.2 Reduction and Unweighted $(2+\epsilon)$-APSP

Intuitively, we split every edge into $k+1$ edges to form G'. Then we compute $(2,k)$-APSP on G', divide the result by $k+1$, and round down to the nearest integer. Because the additive error k is not enough to reach a new node in G, we can simply ignore this part of the path. In the following theorem we make this formal.

Theorem 2.1. *Let $\mathcal{A}$ be an algorithm that provides a $(a+\epsilon,k)$-approximation for APSP in $\tau_{\mathcal{A}}(n',m')$ time, for any unweighted n'-node m'-edge graph G', and any constants $a,k \in \mathbb{N}_{\geq 1}$ and $\epsilon \in [0,1)$. Given an unweighted graph $G=(V,E)$ on n nodes and m vertices, we can compute $(a+(k+2)\epsilon,0)$-APSP in $\tau_{\mathcal{A}}(n+km,(k+1)m)$ time.*

If $\mathcal{A}$ is a dynamic algorithm, then each update takes $(k+1)\cdot$ [update time of $\mathcal{A}$] time. The query time remains the same (up to a constant factor).

Proof. We construct the auxiliary graph $G'=(V',E')$ as follows.

$$V' := V \cup \{\{u,v\}_i : \{u,v\} \in E, i \in \{1,2,\ldots,k\}\};$$
$$E' := \bigcup_{\{u,v\}\in E} \{\{u,\{u,v\}_1\},\{\{u,v\}_1,\{u,v\}_2\},\ldots,\{\{u,v\}_{k-1},\{u,v\}_k\},\{\{u,v\}_k,v\}\}.$$

Clearly this graph has $n+km$ vertices and $(k+1)m$ edges. We compute (a,k)-APSP on G' in $\tau_{\mathcal{A}}(n+km,(k+1)m)$ time using algorithm $\mathcal{A}$, and denote the output by $\delta_{G'}$. We set $\delta(u,v) := \lfloor \frac{\delta_{G'}(u,v)}{k+1} \rfloor$ for all $u,v \in V$.

Claim. δ gives $(a+(k+2)\epsilon,0)$-approximate APSP on G.

Let $u,v \in V$. We remark that $d_{G'}(u,v) = (k+1)d_G(u,v)$ for all $u,v \in V$. Which we will use twice.

First, we show that $d_G(u,v) \leq \delta(u,v)$. This follows from $\delta(u,v) = \lfloor \frac{\delta_{G'}(u,v)}{k+1} \rfloor \geq \lfloor \frac{d_{G'}(u,v)}{k+1} \rfloor = \lfloor \frac{(k+1)d_G(u,v)}{k+1} \rfloor = d_G(u,v)$.

Second, we show that $\delta(u,v) \leq (a+(k+2)\epsilon)\cdot d_G(u,v)$. This follows from

$$\delta(u,v) = \lfloor \tfrac{\delta_{G'}(u,v)}{k+1} \rfloor \leq \lfloor \tfrac{(a+\epsilon)\cdot d_{G'}(u,v)+k}{k+1} \rfloor = \lfloor \tfrac{(a+\epsilon)\cdot(k+1)d_G(u,v)+k}{k+1} \rfloor = \lfloor (a+\epsilon)\cdot d_G(u,v) + \tfrac{k}{k+1} \rfloor.$$

Now if $\epsilon = 0$, then $a\cdot d_G(u,v)$ is an integer, hence $\lfloor a\cdot d_G(u,v) + \frac{k}{k+1} \rfloor = a\cdot d_G(u,v)$. If $\epsilon > 0$, then $\epsilon d_G(u,v) + \frac{k}{k+1}$ can be bigger than 1 – if it is not, we are done by the same argument. Now we note that

$$(k+1)\epsilon \cdot d_G(u,v) \geq (k+1)\left(1 - \tfrac{k}{k+1}\right) = 1.$$

Hence we obtain

$$\begin{aligned}\lfloor (a+\epsilon)\cdot d_G(u,v) + \tfrac{k}{k+1}\rfloor &\leq (a+\epsilon)\cdot d_G(u,v) + 1\\ &\leq (a+\epsilon)\cdot d_G(u,v) + (k+1)\epsilon \cdot d_G(u,v)\\ &= (a+(k+2)\cdot\epsilon) d_G(u,v).\end{aligned}$$

For the claim about dynamic algorithms, note that any update to an edge in G, corresponds to updating $k+1$ edges in G'. The query consists of computing $\delta(u,v) = \lfloor \frac{\delta_{G'}(u,v)}{k+1}\rfloor$, which requires a query to the distance in G' plus constant time operations. □

In particular, this can give fast algorithms for sparse graphs. For the decremental case, we obtain the following result.

Theorem 1.3. *Given an undirected unweighted graph G, there is a decremental data structure that maintains a $(2+\epsilon)$-approximation for APSP with constant query time. W.h.p. the total update time is bounded by $\tilde{O}(m^{7/4})$ if $m = n^{1+\Omega(1)}$, and $\tilde{O}(m^{7/4+o(1)})$ otherwise.*

Proof. We apply the theorem above with the $(2+\epsilon, 1)$-APSP of this paper, Theorem 1.2. This takes $\tilde{O}(nm^{3/4})$ time if $m = n^{1+\Omega(1)}$, and $\tilde{O}(n^{1+o(1)}m^{3/4})$ time otherwise. □

6 Approximate APSP with Additive Factors

In this section we give our second result, which is maintaining APSP on unweighted graphs with a mixed additive and multiplicative stretch. We start by reviewing useful tools in Section 6.1. We then provide a decremental $(1+\epsilon, 2)$-APSP algorithm as a warm-up in Section 6.2. Finally in Section 6.3 we generalize our techniques to maintain a $(1+\epsilon, 2(k-1))$-APSP algorithm efficiently.

6.1 Decremental Tools

We start with describing several useful tools we use in our algorithms.

Even-Shiloach tree (ES-tree). An ES-tree is a central data structure for maintaining distances in a decremental setting. It maintains the exact distances from a source s under edge deletions, giving the following.

Lemma 6.1 ([ES81, HK95, Kin99]). *There is a data structure called ES-tree that, given a weighted directed graph G undergoing deletions and edge weight increases, a root node s, and a depth parameter d, maintains, for every node v a value $\ell_s(v)$ such that $\ell_s(v) = d_G(s,v)$ if $d_G(s,v) \leq d$ and $\ell_s(v) = \infty$ if $d_G(s,v) > d$. It has constant query time and a total update time of $O(md+n)$, where m is the number of edges in G.*

Monotone Even-Shiloach tree (Monotone ES-tree). A monotone ES-tree is a generalization of ES-tree studied in [HKN16,HKN18] that handles edge insertions. This is a data structure that maintains distances from a source s in a graph undergoing deletions, insertions and edge weight increases. Note that when edges are inserted to the graph, it can potentially decrease some distances in the graph. However, the data structure ignores such decreases, and maintains distance estimates $\ell_s(v)$ from s to any other node, that can only *increase* during the algorithm. In particular, these distance estimates are not necessarily the correct distances in the graph, and when using this data structure the goal is to prove that these estimates still provide a good approximation for the current distances. We call $\ell_s(v)$ the level of v in the monotone ES-tree rooted at s.

We next describe the main properties of monotone ES-trees that we use in our algorithms. A pseudo-code appears in Appendix A. The following lemma follows from [HKN16] and from the pseudo-code.

Lemma 6.2. *The following holds for a monotone ES-tree of a graph H rooted at s, that is maintained up to distance d:*

(1) At the beginning of the algorithm, $\ell_s(v) = d_H(s,v)$ for all nodes v where $d_H(s,v) \leq d$.

(2) It always holds that $d_H(s,v) \leq \ell_s(v)$.

(3) The level $\ell_s(v)$ of a node v never decreases.

(4) When the level $\ell_s(v)$ increases to a value that is at most d, we have $\ell_s(v) = \min_{x \in N(v)} \{\ell_s(x) + w_H(x,v)\}$.

Lemma 6.3 ([HKN16]). *For every $d \geq 1$, the total update time of a monotone ES-tree up to maximum level d on a graph H undergoing edge deletions, edge insertions, and edge weight increases is $O(\mathcal{E}(H) \cdot d + \mathcal{W}(H) + n)$, where $\mathcal{E}(H)$ is the total number of edges ever contained in H and $\mathcal{W}(H)$ is the total number of updates to H.*

Handling large distances. In [HKN14], the authors show an algorithm that obtains near-additive approximations for decremental APSP, giving the following.

Theorem 6.4 (Theorem 1.3 in [HKN14]). *For any constant $0 < \epsilon < 1$ and integer $2 \leq k \leq \log n$, there is a $(1+\epsilon, 2(1+2/\epsilon)^{k-2})$-approximation algorithm for decremental APSP with constant worst-case query time and expected total update time of $\tilde{O}(n^{2+1/k}(37/\epsilon)^{k-1})$.*

This will help us in handling *large distances*, because of the following reason. Note that if we have a $(1+\epsilon, \beta)$-approximation algorithm for APSP, then for any pair of nodes at distance $\Omega(\beta/\epsilon)$ from each other, the near-additive approximation is actually a $(1+O(\epsilon))$-approximation because the additive β term becomes negligible. Hence, we can focus our attention on estimating distances between pairs of nodes at distance $O(\beta/\epsilon)$ from each other.

6.2 Warm-up: $(1+\epsilon, 2)$-APSP

Our goal in this section is to use a simpler algorithm based on [DHZ00] that let us maintain $(1+\epsilon, 2)$-APSP in total update time of $O(m^{1/2}n^{3/2+o(1)})$ in unweighted undirected graphs with $m = \Omega(n)$ edges. Such an algorithm would outperform the approach in the Section 4 when $m = \Omega(n^{5/3+o(1)})$, and it also gives a better *near-additive* approximation. We focus on obtaining an algorithm that can maintain $+2$-additive approximate distances for pairs within distance d in total update time $\tilde{O}(m^{1/2}n^{3/2}d)$. This approach can be combined with Theorem 6.4 to get our desired bounds.

6.2.1 Dynamic Data Structure

In Section 2.3, we described the static algorithm of [DHZ00]. We next describe the dynamic version of the algorithm. As mentioned above, we first focus on maintaining a data structure for pairs of nodes at distance at most d from each other. The algorithm is a variant of the static algorithm. At a high-level, we use ES-trees to maintain distances from nodes in D_1 in the graph G, and we use monotone ES-trees to maintain distances from nodes $u \in V \setminus D_1$ in the graph $H_u := (V, E_2 \cup E^* \cup E_u^{D_1})$. The algorithm is summarized as follows.

1. **Node set D_1.** We define a random set D_1 by sampling each node to D_1 with probability $\frac{c \ln n}{s_1}$ for $s_1 = (\frac{m}{n})^{1/2}$ and a constant $c \geq 2$ of our choice.
2. **Edge set E^*.** For each node v that has a neighbor in D_1, we add one edge $\{v, u\}$ to E^*, where u is a neighbor of v in D_1.
3. **Edge set E_2.** For each node v that does not have a neighbor in D_1, we add all the adjacent edges to v to the set E_2.
4. **Maintaining ES-trees from nodes in D_1.** For every $u \in D_1$, we maintain an ES-tree rooted at u up to distance $d+2$ in the graph G.
5. **Maintaining monotone ES-trees from nodes in $V \setminus D_1$.** For each node $u \in V \setminus D_1$, we maintain a monotone ES-tree rooted at u up to distance $d+2$ in the graph $H_u = (V, E_2 \cup E^* \cup E_u^{D_1})$, where $E_u^{D_1}$ are all the edges of the form $\{u, v\}$ where $v \in D_1$. The weight of an edge $\{u, v\} \in E_u^{D_1}$ is $\ell_v(u)$, the level of u in the ES-tree rooted at v computed in the previous step.

Maintaining the monotone ES-trees. Next, we give more details about maintaining the monotone trees from nodes $u \in V \setminus D_1$.

1. Initially, we construct a shortest path tree from $u \in V \setminus D_1$ in the original graph H_u, up to distance $d+2$.

2. Each time the distance estimate between u and a node $x \in D_1$ increases, we update the weight of the edge $\{u, x\} \in E_u^{D_1}$. Note that we can maintain these distances exactly (up to distance $d+2$) as we compute them in the ES-tree rooted at $x \in D_1$.

3. Each time that an edge $\{v, x\} \in E^*$ between a node $v \in V \setminus D_1$ and $x \in D_1$ is deleted from the graph G, we distinguish between two cases. If v still has a neighbor $w \in D_1$, we first add a new edge $\{v, w\}$ to the set E^* and then delete the edge $\{v, x\}$.Otherwise, we add to E_2 all the edges incident to v in the current graph G, and then delete the edge $\{v, x\}$.

4. If an edge $e \in E_2$ is deleted, we delete it from H_u.

There are several challenges in maintaining the monotone ES-trees. First, during the algorithm there are insertions to the graph H_u, as new edges are added to E_2 and E^*, and our goal is to show that our distance estimates still give a good approximation to the actual distances. In the static version of the algorithm, the algorithm computes the exact distances in the graphs H_u and this provides a good approximation for the distances. However, in our case, the monotone ES-trees are not guaranteed to keep the actual distances in H_u because they ignore distance decreases that come from edge insertions, and we should prove that this does not affect the stretch of the algorithm.

In addition, in the static version of the algorithm it is easy to argue that the graphs H_u are sparse. For example, E^* is clearly a set of at most $O(n)$ edges. In our case, as H_u changes dynamically, we need to prove that the total number of edges added to it is small. In particular, as each time that an edge in $\{v, x\} \in E^*$ gets deleted, we replace it by another edge, it could be the case that overall in the algorithm we add many different edges to E^* which can affect the total update time of the algorithm. In addition, each time that a node v no longer has a neighbor in D_1, we add all its adjacent edges to E_2, and we should argue that the number of edges added to E_2 is small. We will discuss these issues in more detail later, and show that we can implement the monotone ES-trees efficiently. Next, we provide the stretch analysis of the algorithm.

6.2.2 Stretch Analysis

For any pair of nodes u, v, we denote by $\ell_u(v)$ the level of v in the ES-tree or monotone ES-tree rooted at u. We start with the following simple observation.

Claim 6.5. *For any $u \in V, x \in D_1$ such that $d_G(u, x) \leq d+2$, we have that $\ell_x(u) = \ell_u(x) = d_G(u, x)$.*

Proof. As $x \in D_1$, we maintain the exact distances from x up to distance $d+2$ in the ES-tree rooted at x. Hence, we have $\ell_x(u) = d_G(u, x)$. If $u \in D_1$, the same argument holds also for $\ell_u(x)$. If $u \in V \setminus D_1$, in Step 2 of the algorithm for maintaining the ES-tree we make sure that we also maintain the correct distance between u and x in the monotone ES-tree rooted at u, hence $\ell_u(x) = d_G(u, x)$. Note that as the edge $\{u, x\}$ always appears in H_u, and by part (4) of Lemma 6.2 we always maintain the exact distance estimate in this case. □

Claim 6.6. *For any $u, v \in V$ such that $d_G(u,v) \leq d$, we have that $d_G(u,v) \leq \ell_u(v) \leq d_G(u,v)+2$.*

Proof. If $v \in D_1$, this already follows from Claim 6.5. Hence we focus on the case that $v \in V \setminus D_1$. The left side of the inequality follows as $d_G(u,v) \leq d_{H_u}(u,v) \leq \ell_u(v)$, using part (2) of Lemma 6.2, and since the distances in H_u can only increase compared to distances in G. This holds as the edges in H_u are either edges in G or have a weight that is at least the weight of the shortest path between the corresponding endpoints in G. We next show that $\ell_u(v) \leq d_G(u,v)+2$. We prove the claim by double-induction. First, on the time in the algorithm, and second on the distance $d_G(u,v)$.

Base Case. At the beginning of the algorithm, this follows from the correctness of the static algorithm, we give the details for completeness. Note that at the beginning of the algorithm, from part (1) of Lemma 6.2, we have that $\ell_u(v) = d_{H_u}(u,v)$ as long as $d_{H_u}(u,v) \leq d+2$, and we show that this is indeed the case when $d_G(u,v) \leq d$.

Case 1: v has a neighbor in D_1. In this case there is an edge $\{v,x\} \in E^*$ such that $x \in D_1$. Note that as x is a neighbor of v it holds that $d_G(u,x) \leq d+1$. As $x \in D_1$ and $d_G(u,x) \leq d+1$, the graph H_u has an edge $\{u,x\}$ with weight $d_G(u,x)$, and it also has the edge $\{v,x\} \in E^*$. Hence, at the beginning of the algorithm, we have

$$\ell_u(v) = d_{H_u}(u,v) \leq d_G(u,x) + d_G(v,x) = d_G(u,x) + 1.$$

Now, by the triangle inequality

$$d_G(u,x) \leq d_G(u,v) + d_G(v,x) = d_G(u,v) + 1.$$

Hence, overall, we get that $\ell_u(v) \leq d_G(u,v)+2$. Note that we get the $d_{H_u}(u,v) \leq d+2$, which implies that the distance is indeed maintained correctly in the monotone ES-tree.

Case 2: v does not have a neighbor in D_1. In this case, all the edges adjacent to v are in $E_2 \subseteq H_u$. Let $\{w,v\}$ be the edge adjacent to v in the shortest path between u and v in G. Then, $d_G(u,w) = d_G(u,v)-1$. From the induction hypothesis, we have that $\ell_u(w) \leq d_G(u,w)+2$. Since the edge $\{w,v\}$ exists in H_u, we get that

$$\ell_u(v) = d_{H_u}(u,v) \leq d_{H_u}(u,w) + d_{H_u}(w,v) = \ell_u(w) + 1 \leq d_G(u,w) + 3 = d_G(u,v) + 2.$$

As $d_{H_u}(u,v) \leq d+2$, we indeed maintain the value correctly in the monotone ES-tree.

Induction Step. Assume now that the claim holds after $t-1$ edge deletions, and we will prove it holds after t edge deletions. We prove it by induction on the length of the path between u and v in the current graph G. In the base case that $u = v$ it trivially holds. First, if the value $\ell_u(v)$ did not change after the t-th edge deletion, then the claim follows from the induction hypothesis (as the distance $d_G(u,v)$ can never decrease). Hence, we focus on the case that $\ell_u(v)$ changed. By parts (3) and (4) of Lemma 6.2, when the level $\ell_u(v)$ is changed it is necessarily increased, and the new value is equal to $\min_{x \in N(v)} \{\ell_u(x) + w_{H_u}(x,v)\}$.

Case 1: When the value $\ell_u(v)$ increases, v has a neighbor in D_1. By Step 3 of the algorithm for maintaining the ES-tree, it is guaranteed that there is an edge $\{v,x\} \in E^* \subseteq H_u$. Also, $\ell_u(x) = d_G(u,x)$ by Claim 6.5 and since $d_G(u,x) \leq d+1$, as x is a neighbor of v. Hence,

when we update the value $\ell_u(v)$, the new value is at most $\ell_u(x)+w_{H_u}(x,v) = d_G(u,x)+1$. The exact same proof that we used in the base case, shows that $d_G(u,x) \leq d_G(u,v)+1$. Hence, $\ell_u(v) \leq d_G(u,v)+2$, as needed. Again, as this value is bounded by $d+2$ it is maintained correctly.

Case 2: When the value $\ell_u(v)$ increases, v does not have a neighbor in D_1. It follows from Step 3 of the algorithm for maintaining the ES-tree that all the edges adjacent to v are in H_u. Let x be the neighbor of v in the shortest $u-v$ path in G. By the induction hypothesis, $\ell_u(x) \leq d_G(u,x)+2$. Hence,

$$\ell_u(v) \leq \ell_u(x) + w_{H_u}(x,v) \leq d_G(u,x)+3 = d_G(u,v)+2,$$

this value is again at most $d+2$, hence it is maintained correctly, completing the proof. □

6.2.3 Time Complexity

To analyze the time complexity of the algorithm, we first bound the total number of edges added to E_2 and E^* in the algorithm.

Claim 6.7. *The total number of edges added to E_2 during the algorithm is bounded by $O(ns_1)$ with high probability.*

Proof. By definition, the edges in E_2 are the edges adjacent to all nodes that do not have a neighbor in D_1. We show that as long as the degree of a node v is s_1 then with high probability it has a neighbor in D_1. This follows as D_1 is obtained by sampling each node with probability $\frac{c\ln n}{s_1}$. Hence, as long as the degree of v is at least s_1, the probability that none of its neighbors is in D_1 is at most $(1-\frac{c\ln n}{s_1})^{s_1} \leq e^{-c\ln n} = \frac{1}{n^c}$. It follows that with high probability each node adds less than s_1 edges to E_2, proving the claim. □

Claim 6.8. *The number of edges added to E^* during the algorithm is bounded by $O((mn)^{1/2}\log n)$ in expectation.*

Proof. Recall that each node is added to the set D_1 with probability $\frac{c\ln n}{s_1}$. Hence, in expectation each node v has $O(\deg(v)\frac{\log n}{s_1})$ neighbors in the set D_1, where $\deg(v)$ is the degree of v at the beginning of the algorithm. These edges are candidates to be added to the set E^*, and in the worst case all of them are added to E^* during the algorithm. Summing over all nodes, the expected number of edges added to E^* is bounded by

$$\sum_{v\in V} O\left(\frac{\deg(v)\log n}{s_1}\right) = O\left(\frac{m\log n}{(\frac{m}{n})^{1/2}}\right) = O((mn)^{1/2}\log n).$$

□

Lemma 6.9. *The total update time of the algorithm is $\tilde{O}(m^{1/2}n^{3/2}d)$ in expectation.*

Proof. Computing the set D_1 takes $O(n)$ time by sampling each node to the set independently with probability $\frac{c\ln n}{s_1}$.

Maintaining the sets E^* and E_2 is done as follows. At the beginning of the algorithm, each node v scans its neighbors until it finds the first neighbor u in the set D_1 if exists, and it adds the edge $\{v,u\}$ to the set E^*. If there is no such u, then v adds all its adjacent edges to the set E_2. During the algorithm, when the edge $\{v,u\}$ is deleted, v continues scanning

its neighbors until it finds the next neighbor w in the set D_1 if exists. If w exists it adds the edge $\{v, w\}$ to E^*, and otherwise it adds all its adjacent edges to E_2. Note that in the whole algorithm, v only needs to scan its neighbors once to find all the edges it adds to E^*. Hence, overall maintaining the sets E^* and E_2 takes $O(m)$ time. We can also keep a list of the new edges added to E^* and E_2 after a deletion, to help maintaining the monotone ES-trees.

We maintain the ES-trees from nodes $u \in D_1$ as follows. Every time that an edge is deleted we delete it from all the trees. In addition, every time that a level $\ell_u(v)$ increases for $u \in D_1, v \in V \setminus D_1$, we update the weight of the edge $\{v, u\}$ in the graph H_v, and update it in the monotone ES-tree rooted at v. Maintaining the ES-trees from nodes in D_1 takes $O(|D_1|(md + n))$ time by Lemma 6.1. By the definition of D_1, this equals

$$O\left(\frac{n \log n}{s_1}(md) + n^2\right) = O(m^{1/2} n^{3/2} (\log n) d)$$

in expectation, when $m = \Omega(n)$.

We maintain the monotone ES-trees rooted at nodes $u \in V \setminus D_1$ as follows. After an edge e is deleted, we update the weights of the edges $\{u, v\} \in E_u^{D_1}$ when their weights are increased in the ES-trees rooted at nodes $v \in D_1$, as described above. In addition, we add to H_u all the edges that were added to E_2 and E^* after the deletion of the edge e (we kept them in a list as described above), and finally we delete the edge e from the monotone ES-tree. By Lemma 6.3, the total update time for maintaining the monotone ES-tree rooted at the node u is $O(\mathcal{E}(H_u) \cdot d + \mathcal{W}(H_u) + n)$, where $\mathcal{E}(H_u)$ is the total number of edges ever contained in H_u and $\mathcal{W}(H_u)$ is the total number of updates to H_u. There are $O(n)$ edges in $E_u^{D_1}$ and each of them is updated $O(d)$ times, as the distance estimates only increase and can have $O(d)$ values. This adds $O(nd)$ term to the complexity of one tree, and $O(n^2 d)$ summing up over all trees. The edges in E_2 and E^* are only added and removed once in the algorithm. The total number of edges in E_2 is $O(ns_1)$ with high probability from Claim 6.7. Summing up over all trees these edges add $O(n^2 s_1 d) = O(n^{3/2} m^{1/2} d)$ term to the complexity. The total number of edges in E^* is bounded by $O((mn)^{1/2} \log n)$ in expectation from Claim 6.8. Summing up over all trees they add $O(n^{3/2} (\log n) m^{1/2} d)$ term to the complexity, completing the proof. □

Claim 6.10. *The query time of the algorithm is $O(1)$.*

Proof. To answer a distance query for a pair of nodes u, v, we just return the value $\ell_u(v)$. This can be done in $O(1)$ time if we keep for each node u an array of the values $\ell_u(v)$ which is updated each time a distance estimate change. □

In conclusion, we get the following.

Theorem 6.11. *Given an undirected unweighted graph G, there is a decremental data structure that maintains $+2$-additive approximation for APSP for pairs of nodes at distance at most d from each other. The expected total update time is bounded by $\tilde{O}(m^{1/2} n^{3/2} d)$, and the query time is constant.*

To get rid of the d term in the complexity, we can combine our approach with Theorem 6.4. This would lead to a $(1 + \epsilon, 2)$-approximation and replace the d term in the complexity by $n^{o(1)}$ term. We will provide full details of the approach when we describe our general algorithm in the next section (see in particular Section 6.3.4). The high-level idea is to use our algorithm with a parameter $d = n^{o(1)}$, and combine it with Theorem 6.4 for computing $(1 + \epsilon, \beta)$-approximation for large distances. For an appropriate choice of the

parameters, we will get a $(1+\epsilon)$-approximation for the distances between pairs of nodes at distance greater than d from each other, and a $+2$ approximation for pairs of nodes at distance at most d from each other.

6.3 $(1+\epsilon, 2k-2)$-APSP

With this warm-up we can move on to the more general algorithm based on [DHZ00] that leads to a larger stretch but a smaller update time. Our goal is to obtain a $(1+\epsilon, 2k-2)$-approximation in $\tilde{O}(n^{2-1/k}m^{1/k})$ time. The algorithm works in unweighted undirected graphs. As before, we start by describing a data structure that maintains the approximate distances for pairs of nodes at distance at most d from each other, and then in Section 6.3.4 we combine it with Theorem 6.4 to handle the general case.

6.3.1 Dynamic Data Structure

The algorithm is summarized as follows.

1. **Node sets D_i.** We define a series of sets $D_1, D_2, ..., D_k$ such that for $i = 1, ..., k-1$, the set D_i is a random set obtained by sampling each node with probability $p_i = \frac{c \ln n}{s_i}$, for $s_i = (\frac{m}{n})^{1-i/k}(\ln n)^{i/k}$, and a constant $c \geq 2$ for our choice. If a node is sampled to several different sets we keep it only in the set with minimum index i among them. Finally, we define $D_k = V \setminus \bigcup_{i=1}^{k-1} D_i$. Note that by definition, each node is in exactly one set D_i.

2. **Indices i_v for all $v \in V$.** For a node $v \in V$, we denote by i_v the minimum index i such that v has a neighbor in D_i.

3. **Edge set E^*.** For each node v, we add one edge $\{v, x\}$ to E^*, where x is a neighbor of v in D_{i_v}.

4. **Edge sets E_i.** The edges E_i are all the edges $\{u, v\}$ where $\max\{i_u, i_v\} \geq i$. In other words, it is all the edges adjacent to nodes that do not have a neighbor in $D_{i'}$ for $i' < i$.

5. **Maintaining monotone ES-trees.** The algorithm maintains monotone ES-trees as follows.

   - For every $u \in D_1$, we maintain an ES-tree rooted at u up to distance $d + 3k$ in the graph G.

   For $i = 2$ to k:

   - For every $u \in D_i$, we maintain a monotone ES-tree rooted at u up to distance $d+3k$ in the graph $H_u^i = (V, E_i \cup E^* \cup E_u^i)$, where E_u^i are all the edges $\{u, v\}$, where $v \in D_{i'}$ for $i' < i$. The weight of the edge $\{u, v\}$ is equal to the value $\ell_v(u)$ in the monotone ES-tree rooted at v.

Note that we maintain the monotone ES-trees up to distance $d + 3k$. As we show in the stretch analysis, this is needed in order to guarantee that we maintain correct estimates for any pair of nodes at distance at most d from each other. Intuitively, as the distance estimate we maintain is at most $d(u, v) + 2k$, we would like to maintain correct estimates for pairs of nodes at distance $d+2k$ from each other. Another k term comes from the inductive argument of the stretch analysis. For more details, see Section 6.3.2.

In Section 6.3.3, we explain in detail how to maintain the data structures efficiently. When we maintain the graphs H_u^i we make sure that the following holds (for a proof, see Section 6.3.3).

Claim 6.12. *Let $u \in D_i, v \in V$. Throughout the updates, it always holds that v either has an edge $\{v, w\} \in H_u^i$, where $w \in D_{i'}$ for $i' < i$, or all the edges adjacent to v are in H_u^i.*

We are in the first case of the claim when $i_v < i$, and we are in the second case when $i_v \geq i$.

6.3.2 Stretch Analysis

We need the following claims. First, from part (1) of Lemma 6.2 we have the following.

Claim 6.13. *At the beginning of the algorithm, for any $u \in D_i$, and any v at distance at most $d + 3k$ from u in H_u^i, we have $\ell_u(v) = d_{H_u^i}(u, v)$.*

We next show the following.

Claim 6.14. *Let $u \in D_i, w \in D_j$ such that $j < i$, then $\ell_u(w) \leq \ell_w(u)$.*

Proof. The claim follows as we always have an edge $\{u, w\}$ with weight $\ell_w(u)$ in the graph H_u^i, and the value $\ell_w(u)$ is monotonically increasing during the algorithm. Hence, and from part (4) of Lemma 6.2, every time that the estimate $\ell_u(w)$ changes it always changes to a value that is at most $\ell_w(u)$. □

We next prove the stretch guarantee of the algorithm.

Lemma 6.15. *For any node $u \in D_i$, and any node $v \in V$ where $d_G(u, v) \leq d+(k-i)$, throughout the updates we have $d_G(u, v) \leq \ell_u(v) \leq d_G(u, v) + 2(i - 1)$.*

Proof. First, the left inequality holds since $d_G(u, v) \leq d_{H_u^i}(u, v) \leq \ell_u(v)$, where the first inequality follows from the definition of H_u^i, and the second follows from part (2) of Lemma 6.2. We prove the right inequality by triple-induction, on the value i, on the time in the algorithm, and on the distance $d_G(u, v)$. For $i = 1$, the claim follows as we maintain ES-trees up to distance $d + 3k$ from the nodes of D_1 in the graph G. We next assume that it holds for any $i' < i$, and prove that it holds for i. The proof is by induction on the time in the algorithm and on the distance $d_G(u, v)$.

Base Case. We first show that the claim holds at the beginning of the algorithm. From Claim 6.13, at the beginning of the algorithm, for any u, v where $d_{H_u^i}(u, v) \leq d + 3k$, we have that $\ell_u(v) = d_{H_u^i}(u, v)$. Hence, our goal is to show that $d_{H_u^i}(u, v) \leq d_G(u, v) + 2(i - 1)$. Note that if this holds, then since $d_G(u, v) \leq d+(k-i)$, we get that $d_{H_u^i}(u, v) \leq d+3k$, hence the distance is maintained in the monotone ES-tree as needed. The proof is by induction on $d_G(u, v)$.

Case 1: $i_v < i$. In this case, there is an edge $\{v, w\} \in E^* \subseteq H_u^i$ such that $w \in D_{i'}$ for $i' < i$. Note that since $d_G(u, v) \leq d + (k - i)$, and w is a neighbor of v, then we have $d_G(u, w) \leq d+(k-i)+1 = d+(k-(i-1)) \leq d+(k-i')$. Hence, we can use the induction hypothesis on w and u. From the induction hypothesis and Claim 6.13, we have $\ell_w(u) = d_{H_u^i}(w, u) \leq d_G(w, u) + 2(i - 2)$ (note that this value is smaller than $d + 3k$, so we can indeed use Claim 6.13). Hence, we get that

$$d_{H_u^i}(u, v) \leq d_{H_u^i}(u, w) + d_{H_u^i}(w, v) \leq d_G(w, u) + 2(i - 2) + 1.$$

We have, $d_G(w, u) \leq d_G(w, v) + d_G(v, u) = d_G(u, v) + 1$. Hence, we get, $d_{H_u^i}(u, v) \leq d_G(u, v) + 2(i - 1)$, as needed.

Case 2: $i_v \geq i$. In this case, by definition, all the edges adjacent to v are in $E_i \subseteq H_u^i$. Let w be the neighbor of v on the shortest $u - v$ path. From the induction hypothesis, $d_{H_u^i}(u, w) \leq d_G(u, w) + 2(i-1)$. So we have

$$d_{H_u^i}(u, v) \leq d_{H_u^i}(u, w) + d_{H_u^i}(w, v) \leq d_G(u, w) + 1 + 2(i-1) = d_G(u, v) + 2(i-1).$$

Induction Step. We assume that the claim holds after $t-1$ edge deletions, and show that it holds after t edge deletions. The proof is by induction on the distance $d_G(u, v)$. If $u = v$, the claim trivially holds. If the estimate $\ell_u(v)$ did not change after the t'th edge deletion, then the claim holds from the induction hypothesis. Note that an edge deletion can potentially lead to several changes of the value $\ell_u(v)$. It is enough to prove that the stretch guarantee holds in the last time the value $\ell_u(v)$ increases, as the value stays the same afterwards. We divide into cases based on Claim 6.12.

Case 1: When the value $\ell_u(v)$ increases in the last time, there is an edge $\{v, w\} \in H_u^i$ such that $w \in D_{i'}$ for $i' < i$. From part (4) of Lemma 6.2, when $\ell_u(v)$ changes, it gets the value $\min_{x \in N(v)} \{\ell_u(x) + w_{H_u^i}(x, v)\}$, assuming this value is at most $d + 3k$, and we will show that this is indeed the case. Hence,

$$\ell_u(v) \leq \ell_u(w) + w_{H_u^i}(w, v) \leq \ell_w(u) + 1 \leq d_G(u, w) + 2(i-2) + 1.$$

Here we use Claim 6.14 and the fact that $w \in D_{i'}$ for $i' < i$ (note that since $d_G(u, v) \leq d + (k-i)$, and w is a neighbor of v, it follows that $d_G(u, w) \leq d + (k-i')$, so we can indeed use the induction hypothesis). It also holds that $d_G(u, w) \leq d_G(u, v) + d_G(v, w) = d_G(u, v) + 1$. Hence, overall, we have $\ell_u(v) \leq d_G(u, v) + 2(i-2) + 2 = d_G(u, v) + 2(i-1)$, as needed. Note that this value is indeed smaller than $d + 3k$, hence we maintain the correct estimate in the monotone ES-tree.

Case 2: When the value $\ell_u(v)$ increases in the last time, all the adjacent edges to v are in the graph H_u^i. Let w be the neighbor of v on the shortest $u - v$ path. When the distance estimate of v changes we have

$$\ell_u(v) \leq \ell_u(w) + d_{H_u^i}(w, v) \leq d_G(u, w) + 2(i-1) + 1 = d_G(u, v) + 2(i-1),$$

where we use the induction hypothesis on w and part (4) of Lemma 6.2. Note that it follows that the values $\ell_u(v), \ell_u(w)$ are smaller than $d + 3k$, hence they are maintained correctly in the monotone ES-tree. □

6.3.3 Time Complexity

We next analyze the time complexity of the algorithm. We start by showing that the first steps of the algorithm can be maintained efficiently.

Claim 6.16. *The first 4 steps of the algorithms can be maintained in $O((m+n)k)$ time.*

Proof. **Step 1.** Computing the sets D_i takes $O(kn)$ time, by just sampling each node to the set D_i with probability p_i. For each node v, we can stop the sampling after the first time v is sampled to one of the sets. This is only done once at the beginning of the algorithm. For each node v, we keep the index i such that $v \in D_i$. For each i, we create a list of all the nodes in the set D_i. Creating all these lists takes $O(n)$ time by scanning the nodes.

Steps 2 and 3. The indices i_v and the set E^* are maintained dynamically in the algorithm, as follows. At the beginning, each node v scans its neighbors to find the minimum index i such that v has a neighbor in D_i, it also saves the name of an edge $\{v, u\}$ to the first neighbor of v in D_{i_v}, this edge is added to set E^*. In the algorithm, if the edge $\{v, u\}$ is deleted, then v goes over its next neighbors, and finds the next neighbor $w \in D_{i_v}$ if exists. If such a neighbor exists, then $\{v, w\}$ is added to E^*. Otherwise, the value i_v increases, and v scans its neighbors again to determine the new index i_v and adds the first edge $\{v, w\}$ from v to a neighbor in D_{i_v} to E^*. Note that overall in the algorithm each node needs to scan its neighbors $O(k)$ times, because as long as the value i_v stays the same, we just need to go over the neighbors once during the whole algorithm to determine all the edges added to E^*. Overall these steps take $O(mk)$ time.

Step 4. Maintaining the sets E_i is done as follows. Initially, we add to E_i all the edges adjacent to nodes v where $i_v \geq i$. During the algorithm, every time that the value i_v of a node v changes, say from j to j', then for all $j < i \leq j'$, we add the edges adjacent to v to the set E_i. We also keep a list of all the newly added edges in E_i after a deletion of an edge, so that we can update them in all the relevant monotone ES-trees. The time complexity for maintaining the set E_i is proportional to the number of edges added to E_i during the algorithm, which can trivially be bounded by $O(m)$, hence this step takes at most $O(mk)$ time. □

Maintaining the monotone ES-trees. The main challenge in the algorithm is maintaining the monotone ES-trees in Step 5 of the algorithm. Before analyzing the time complexity of the algorithm, we start by describing the algorithm in more detail. Note that each time an edge is deleted it can lead to many changes, including edge insertions and weight changes in all the trees we maintain. We next explain in detail in which order we make these changes in order to maintain the correctness of the algorithm. We use the algorithms for ES trees and monotone ES trees described in Section 6.1.

ES-trees rooted at nodes of D_1. First, for any node $u \in D_1$, we start by creating an ES-tree rooted at u in the graph G, maintained up to distance $d+3k$. After creating this tree, for any $v \in D_i$ for $i > 1$, we add an edge $\{u, v\}$ with weight $\ell_u(v)$ to the set of edges E^i_v. During the algorithm, every time an edge is deleted, we delete it from the ES-tree and update it accordingly. For any v where $\ell_u(v)$ has increased after the deletion and where $v \in D_i$ for $i > 1$, we update the corresponding edge in E^i_v, and save it in a list of edges in E^i_v that were updated after the deletion. This completes the description for $u \in D_1$.

Monotone ES-trees rooted at nodes of D_i. For $u \in D_i$ for larger values of i, we maintain the monotone ES-trees according to the order of the index i, from 2 to k. We work as follows. Initially, we create a monotone ES-tree rooted at u up to distance $d+3k$ in the graph $H^i_u = (V, E_i \cup E^* \cup E^i_u)$, with the edges E_i, E^* at the beginning of the algorithm, and with the edges E^i_u that were obtained when creating the monotone ES-trees in previous levels. Note that all these edges of E^i_u are of the form $\{u, v\}$ where $v \in D_{i'}$ for $i' < i$, so we indeed already created the monotone ES-tree rooted at the relevant nodes v. After creating the tree, for any node $v \in D_{i'}$ for $i' > i$, we add a new edge to the set $E^{i'}_v$ with weight $\ell_u(v)$. During the algorithm, when an edge e is deleted, we update the monotone ES-trees in several steps as follows, again we work according to the order of the index i.

- *First step: adding the new edges of E^* and E_i to H^i_u.* The deletion of the edge e can lead to adding new edges to E^* or E_i in steps 3 and 4 of the algorithm. As explained in these

steps, we maintain the newly added edges, and can add them to a list of newly added edges (to E^* and to E_i separately). When we update the monotone ES-tree rooted at $u \in D_i$, we start by adding all these newly added edges that were added to E^* and E_i to H_u^i.

- *Second step: updating the edges in E_u^i.* Next, there are possibly edges in $\{u, v\} \in E_u^i$ that their weight is increased after the deletion. This is already observed when we update the monotone ES-tree rooted at v, and if it happens, we add the edge $\{u, v\}$ with new weight $\ell_v(u)$ to a list of edges that were updated in E_u^i. When we update the monotone ES-tree rooted at u, we update the weights of all these edges (we already have their list after we finish maintaining the ES-trees in previous levels).
- *Third step: deleting the edge e.* Lastly, we delete the edge e that was deleted in this step from the monotone ES-tree.

Note that the weight increases and deletion can increase the levels $\ell_u(v)$ of some nodes in the tree. For any v in D_j for $j > i$ such that $\ell_u(v)$ increases, when the value increases, we add an edge $\{v, u\}$ with the new weight $\ell_u(v)$ to the list of newly updated edges in E_v^j. This completes the description of the algorithm. From the above description, we can now prove Claim 6.12.

Proof of Claim 6.12. Let $u \in D_i, v \in V$. At the beginning of the algorithm, if $i_v < i$, then there exists an edge $\{v, w\} \in E^* \subseteq H_u^i$ where $w \in D_{i'}$ for $i' < i$. Otherwise, $i_v \geq i$, and all the edges adjacent to v are in $E_i \subseteq H_u^i$. During the algorithm, every time an edge e of the form $\{v, w\} \in E^*$ is deleted, then if $i_v < i$ after the deletion, we add a new edge $\{v, w'\}$ to E^* for $w' \in D_{i'}$ for $i' < i$, and if $i_v \geq i$, we add all the edges adjacent to v to $E_i \subseteq H_u^i$. In the algorithm, we first add these new edges to H_u^i, and only then delete the edge e, which proves the claim. □

To bound the time complexity, we need the following useful claims that bound the total number of edges added to the sets E_i and E^* during the algorithm.

Claim 6.17. *For $i \geq 2$, the total number of edges added to E_i during the algorithm is bounded by $O(ns_{i-1})$ with high probability.*

Proof. The edges E_i are all the edges adjacent to nodes v where $i_v \geq i$. We show that at the first time in the algorithm that $i_v \geq i$, the degree of v is at most s_{i-1} with high probability. This implies that each node v adds at most s_{i-1} edges to the set E_i, proving the claim. To bound the degree of v, note that by definition the set D_{i-1} is obtained by sampling each node with probability $\frac{c \ln n}{s_{i-1}}$, all these nodes are added to the set D_{i-1}, unless they are already part of a set $D_{i'}$ for $i' < i-1$. As long as the degree of v is at least s_{i-1}, the probability that none of the neighbors of v is sampled to D_{i-1} is at most $(1 - \frac{c \ln n}{s_{i-1}})^{s_{i-1}} \leq e^{-c \ln n} = \frac{1}{n^c}$. Hence, with high probability v has a neighbor sampled to D_{i-1}, implying that $i_v \leq i-1$ as long as $\deg(v) \geq s_{i-1}$. From union bound, this holds for all nodes with high probability. This proves that the total number of edges added to E_i in the algorithm is bounded by $O(ns_{i-1})$. Note that the proof assumes that we have an oblivious adversary and that the order in which edges are deleted does not depend on the random choices of the algorithm. □

Claim 6.18. *In expectation, the total number of edges added to the set E^* in the algorithm is bounded by $O(kn^{1-1/k}m^{1/k}(\log n)^{1-1/k})$.*

Proof. We analyze the number of edges v adds to the set E^* when $i_v = i$ for all i. First, when $i_v = 1$, then in the worst case v adds to E^* edges to all its neighbors in D_1. By the definition of D_1, each node is added to D_1 with probability $\frac{c \ln n}{s_1}$. Hence, in expectation v has $O(\frac{\deg(v) \log n}{s_1})$ neighbors in D_1, where $\deg(v)$ is the degree of v at the beginning of the algorithm. When we sum up over all v, this adds at most $O(\frac{m \log n}{s_1}) = O(n^{1-1/k} m^{1/k} (\log n)^{1-1/k})$ edges in expectation, where the last equality follows from the definition of s_1.

When $i_v = i$ for $i > 1$, as we proved in Claim 6.17, with high probability the degree of v is at most s_{i-1}. Note that each one of the neighbors of v is added to D_i with probability $\frac{c \ln n}{s_i}$. Hence, in expectation, v has $O(s_{i-1} \cdot \frac{\ln n}{s_i})$ neighbors in D_i. In the worst case, it adds edges to all of them during the algorithm. In total, the expected number of edges added by all nodes to E^* in the case that $i_v \geq 2$ is bounded by $\sum_{i=2}^{k} O(\frac{n \log n s_{i-1}}{s_i})$. By the definition of s_i, we have that $\frac{s_{i-1}}{s_i} = (\frac{m}{n})^{1/k} (\ln n)^{-1/k}$. Hence, in total the expected number of edges added is bounded by $O(k n^{1-1/k} m^{1/k} (\log n)^{1-1/k})$. □

Next, we prove the complexity of the algorithm.

Lemma 6.19. *Maintaining the ES-trees takes $O(k n^{2-1/k} m^{1/k} (\log n)^{1-1/k} d)$ time in expectation.*

Proof. Maintaining the ES-trees rooted at nodes in D_1 takes $O(|D_1|(md+n))$ time by Lemma 6.1. Since $|D_1| = O(\frac{n \log n}{s_1})$ in expectation, and by the choice of s_1, this adds $O(n^{2-1/k} m^{1/k} (\log n)^{1-1/k} d)$ term to the complexity, when $m = \Omega(n)$.

From Lemma 6.3, maintaining a monotone ES-tree rooted at $u \in D_i$ takes $O(\mathcal{E}(H_u^i) \cdot d + \mathcal{W}(H_u^i) + n)$ time, where $\mathcal{E}(H_u^i)$ is the total number of edges ever contained in H_u^i and $\mathcal{W}(H_u^i)$ is the total number of updates to H_u^i. To complete the proof, we should analyze these values. Recall that $H_u^i = (V, E_i \cup E^* \cup E_u^i)$. Note that E_u^i has at most $n-1$ edges, that are updated $O(d)$ times during the algorithm, as we maintain distances up to distance $d+3k$, hence these edges add $O(nd)$ term to the complexity of maintaining one tree, and $O(n^2 d)$ for all n trees. The edges in E^* and E_i are only added and removed once from a tree during the algorithm. From Claim 6.17, the total number of edges added to E_i during the algorithm is bounded by $O(n s_{i-1})$ with high probability. Since these edges are part of all the ES-trees rooted at nodes in D_i, maintaining these edges adds $O(|D_i| n s_{i-1} d)$ term to the complexity. This equals

$$O\left(\frac{n \log n}{s_i} \cdot n s_{i-1} \cdot d\right) = O(n^{2-1/k} m^{1/k} (\log n)^{1-1/k} d),$$

following the same calculations done in Claim 6.18 (but with an extra nd term). Summing over all k adds another multiplicative k term to the complexity. From Claim 6.18, the total number of edges added to E^* is bounded by $O(k n^{1-1/k} m^{1/k} (\log n)^{1-1/k})$ in expectation. Summing over all n trees, it adds another $O(k n^{2-1/k} m^{1/k} (\log n)^{1-1/k} d)$ term to the complexity, completing the proof. □

Claim 6.20. *The query complexity of the algorithm is constant.*

Proof. To answer a distance query for a pair of nodes u, v, we work as follows. If $u \in D_i, v \in D_j$, and $i < j$, our distance estimate is $\ell_u(v)$, otherwise our distance estimate is $\ell_v(u)$. Note that it is easy to maintain the values $\ell_u(v)$ in a way that the query time is constant. For example, we can maintain for each node u an array with the values $\ell_u(v)$, and update the value in constant time each time it was changed in the algorithm. □

6.3.4 Putting Everything Together

In the last sections we showed an algorithm that for any pair of nodes at distance at most d maintains a $+2(k-1)$-additive approximation in $\tilde{O}(n^{2-1/k}m^{1/k}d)$ time. To get our final result, we combine it with Theorem 6.4 to handle large distances, proving the following.

Theorem 1.4. *Given an undirected unweighted graph G, a constant $0 < \epsilon < 1$ and an integer $2 \leq k \leq \log n$, there is a decremental data structure that maintains $(1+\epsilon, 2(k-1))$-approximation for APSP with constant query time. The expected total update time is bounded by* $\min\{O(n^{2-1/k+o(1)}m^{1/k}), \tilde{O}((n^{2-1/k}m^{1/k})O(1/\epsilon)^{k/\rho})\}$ *where $m = n^{1+\rho}$.*

Note that for small values of k and if $m = n^{1+\rho}$ for a constant $\rho > 0$, we get a complexity of $\tilde{O}(n^{2-1/k}m^{1/k})$, where in the general case we have an extra $n^{o(1)}$ term.

Proof. We maintain simultaneously our dynamic data structure to handle short distances of at most d, and the data structure from Theorem 6.4 to handle large distances of more than d, for a parameter d which we specify next. We run the algorithm from Theorem 6.4 with parameters $\epsilon' = \epsilon/2$, and k' to be specified later. Recall that this algorithm gives a $(1+\epsilon', \beta)$-approximation for $\beta = 2(1+2/\epsilon')^{k'-2}$. We define $d = \beta/\epsilon'$.

To answer a distance query for a pair of nodes u, v, we return the minimum between the values obtained in both algorithms. As in both algorithms the query time is constant, we get a constant query time.

For the stretch analysis, if $d(u,v) \leq d$, then by Lemma 6.15, we get an additive stretch of at most $+2(k-1)$. If $d(u,v) \geq d = \beta/\epsilon'$, then the distance estimate we get from the $(1+\epsilon', \beta)$-approximation algorithm is at most

$$(1+\epsilon')d(u,v) + \beta \leq (1+\epsilon')d(u,v) + \epsilon' d(u,v) = (1+2\epsilon')d(u,v) = (1+\epsilon)d(u,v).$$

Hence, overall we get a $(1+\epsilon, 2(k-1))$-approximation, as needed.

We now analyze the time complexity. From Lemma 6.19, the time complexity of our algorithm is $O(kn^{2-1/k}m^{1/k}(\log n)^{1-1/k}d)$, and from Theorem 6.4 the time complexity of the near-additive approximation algorithm is $\tilde{O}(n^{2+1/k'}(37/\epsilon)^{k'-1})$. We distinguish between two choices for the value k'. First we choose $k' = k/\rho$, where $m = n^{1+\rho}$. In this case, the complexity of both expressions is bounded by $\tilde{O}((n^{2-1/k}m^{1/k})O(1/\epsilon)^{k/\rho})$, as $d = \beta/\epsilon', \beta = 2(1+2/\epsilon')^{k'-2}$, and $O(n^{2-1/k}m^{1/k}) = O(n^{2-1/k}n^{(1+\rho)/k}) = O(n^{2+\rho/k}) = O(n^{2+1/k'})$.

Second, we choose $k' = \sqrt{\log n}$. In this case, $\tilde{O}(n^{2+1/k'}(37/\epsilon)^{k'-1}) = O(n^{2+o(1)})$. Similarly, $d = \beta/\epsilon' = O(1/\epsilon')^{k-1} = O(1/\epsilon)^{O(\sqrt{\log n})} = n^{o(1)}$. Hence, in total we get a complexity of $O(n^{2-1/k+o(1)}m^{1/k})$, when $m = \Omega(n)$. □

References

[AB17] Amir Abboud and Greg Bodwin. The 4/3 additive spanner exponent is tight. *Journal of the ACM (JACM)*, 64(4):1–20, 2017.

[ABF23] Amir Abboud, Karl Bringmann, and Nick Fischer. Stronger 3-sum lower bounds for approximate distance oracles via additive combinatorics. In *Proc. of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*, 2023.

[ABKZ22] Amir Abboud, Karl Bringmann, Seri Khoury, and Or Zamir. Hardness of approximation in P via short cycle removal: cycle detection, distance oracles, and

beyond. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*, pages 1487–1500, 2022.

[ABP18] Amir Abboud, Greg Bodwin, and Seth Pettie. A hierarchy of lower bounds for sublinear additive spanners. *SIAM Journal on Computing*, 47(6):2203–2236, 2018.

[AC13] Ittai Abraham and Shiri Chechik. Dynamic decremental approximate distance oracles with $(1+\epsilon, 2)$ stretch. *CoRR*, abs/1307.1516, 2013.

[ACIM99] Donald Aingworth, Chandra Chekuri, Piotr Indyk, and Rajeev Motwani. Fast estimation of diameter and shortest paths (without matrix multiplication). *SIAM Journal on Computing*, 28(4):1167–1181, 1999. Announced at SODA 1996.

[ACT14] Ittai Abraham, Shiri Chechik, and Kunal Talwar. Fully dynamic all-pairs shortest paths: Breaking the o(n) barrier. In *Proceedings of the 17th International Workshop on Approximation Algorithms for Combinatorial Optimization Problems (APPROX 2014) and the 18th International Workshop on Randomization and Computation (APPROX/RANDOM 2014)*, pages 1–16, 2014.

[AR20] Maor Akav and Liam Roditty. An almost 2-approximation for all-pairs of shortest paths in subquadratic time. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, (SODA 2020)*, pages 1–11, 2020.

[AR21] Maor Akav and Liam Roditty. A unified approach for all pairs approximate shortest paths in weighted undirected graphs. In *Proceedings of the 29th Annual European Symposium on Algorithms (ESA 2021)*, volume 204, pages 4:1–4:18, 2021.

[BC16] Aaron Bernstein and Shiri Chechik. Deterministic decremental single source shortest paths: beyond the $O(mn)$ bound. In Daniel Wichs and Yishay Mansour, editors, *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2016)*, pages 389–397, 2016.

[Ber09] Aaron Bernstein. Fully dynamic $(2+\epsilon)$ approximate all-pairs shortest paths with fast query and close to linear update time. In *Proceedings of the 50th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2009*, pages 693–702. IEEE Computer Society, 2009.

[Ber16] Aaron Bernstein. Maintaining shortest paths under deletions in weighted directed graphs. *SIAM Journal on Computing*, 45(2):548–574, 2016. Announced at STOC 2013.

[Ber17] Aaron Bernstein. Deterministic partially dynamic single source shortest paths in weighted graphs. In *Proceedings of the 44th International Colloquium on Automata, Languages, and Programming, (ICALP 2017)*, pages 44:1–44:14, 2017.

[BGS09] Surender Baswana, Vishrut Goyal, and Sandeep Sen. All-pairs nearly 2-approximate shortest paths in $o(n^2 \operatorname{polylog} n)$ time. *Theoretical Computer Science*, 410(1):84–93, 2009. Announced at STACS 2005.

[BGS21] Aaron Bernstein, Maximilian Probst Gutenberg, and Thatchaphol Saranurak. Deterministic decremental SSSP and approximate min-cost flow in almost-linear time. In *Proceedings of the 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2021)*, pages 1000–1008, 2021.

[BHS03] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Maintaining all-pairs approximate shortest paths under deletion of edges. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2003)*, pages 394–403, 2003.

[BHS07] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Improved decremental algorithms for maintaining transitive closure and all-pairs shortest paths. *Journal of Algorithms*, 62(2):74–92, 2007. Announced at STOC 2002.

[BK07] Piotr Berman and Shiva Prasad Kasiviswanathan. Faster approximation of distances in graphs. In Frank K. H. A. Dehne, Jörg-Rüdiger Sack, and Norbert Zeh, editors, *Proceedings of the 10th International Workshop on Algorithms and Data Structures (WADS 2007*, pages 541–552, 2007.

[BK10] Surender Baswana and Telikepalli Kavitha. Faster algorithms for all-pairs approximate shortest paths in undirected graphs. *SIAM Journal on Computing*, 39(7):2865–2896, 2010. Announced at FOCS 2006.

[BKS12] Surender Baswana, Sumeet Khurana, and Soumojit Sarkar. Fully dynamic randomized algorithms for graph spanners. *ACM Transactions on Algorithms*, 8(4):35:1–35:51, 2012.

[BR11] Aaron Bernstein and Liam Roditty. Improved dynamic algorithms for maintaining approximate shortest paths under deletions. In Dana Randall, editor, *Proceedings of the Twenty-Second Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2011*, pages 1355–1365. SIAM, 2011.

[BRS+21] Arturs Backurs, Liam Roditty, Gilad Segal, Virginia Vassilevska Williams, and Nicole Wein. Toward tight approximation bounds for graph diameter and eccentricities. *SIAM Journal on computing (SICOMP)*, 50(4):1155–1199, 2021.

[CHDKL21] Keren Censor-Hillel, Michal Dory, Janne H Korhonen, and Dean Leitersdorf. Fast approximate shortest paths in the congested clique. *Distributed Computing*, 34(6):463–487, 2021.

[Che18] Shiri Chechik. Near-optimal approximate decremental all pairs shortest paths. In Mikkel Thorup, editor, *Proceedings of the 59th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2018)*, pages 170–181, 2018.

[Chu21] Julia Chuzhoy. Decremental all-pairs shortest paths in deterministic near-linear time. In Samir Khuller and Virginia Vassilevska Williams, editors, *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021)*, pages 626–639. ACM, 2021.

[CS21] Julia Chuzhoy and Thatchaphol Saranurak. Deterministic algorithms for decremental shortest paths via layered core decomposition. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA 2021)*, pages 2478–2496, 2021.

[CZ01] Edith Cohen and Uri Zwick. All-pairs small-stretch paths. *Journal of Algorithms*, 38(2):335–353, 2001. Announced at SODA 1997.

[CZ22] Shiri Chechik and Tianyi Zhang. Nearly 2-approximate distance oracles in subquadratic time. In *Proc. of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022*, pages 551–580. SIAM, 2022.

[DFK+24] Michal Dory, Sebastian Forster, Yael Kirkpatrick, Yasamin Nazari, Virginia Vassilevska Williams, and Tijn de Vos. Fast 2-approximate all-pairs shortest paths. In *Proc. of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 4728–4757. SIAM, 2024.

[DHZ00] Dorit Dor, Shay Halperin, and Uri Zwick. All-pairs almost shortest paths. *SIAM Journal on Computing*, 29(5):1740–1759, 2000. Announced at FOCS 1996.

[DI04] Camil Demetrescu and Giuseppe F. Italiano. A new approach to dynamic all pairs shortest paths. *Journal of the ACM*, 51(6):968–992, 2004. Announced at STOC 2003.

[DI06] Camil Demetrescu and Giuseppe F. Italiano. Fully dynamic all pairs shortest paths with real edge weights. *Journal of Computer and System Sciences*, 72(5):813–837, 2006. Announced at FOCS 2001.

[DKR+22] Mingyang Deng, Yael Kirkpatrick, Victor Rong, Virginia Vassilevska Williams, and Ziqian Zhong. New additive approximations for shortest paths and cycles. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming, ICALP 2022, July 4-8, 2022, Paris, France*, volume 229 of *LIPIcs*, pages 50:1–50:10. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[DP22] Michal Dory and Merav Parter. Exponentially faster shortest paths in the congested clique. *ACM Journal of the ACM (JACM)*, 69(4):1–42, 2022.

[Dür23] Anita Dürr. Improved bounds for rectangular monotone min-plus product and applications. *Information Processing Letters*, page 106358, 2023.

[EFGW21] Jacob Evald, Viktor Fredslund-Hansen, Maximilian Probst Gutenberg, and Christian Wulff-Nilsen. Decremental APSP in unweighted digraphs versus an adaptive adversary. In *Proc. of the 48th International Colloquium on Automata, Languages, and Programming, (ICALP 2021)*, pages 64:1–64:20, 2021.

[EN20] Michael Elkin and Ofer Neiman. Near-additive spanners and near-exact hopsets, a unified view. *Bulletin of EATCS*, 1(130), 2020.

[EP04] Michael Elkin and David Peleg. $(1+\epsilon,\beta)$-spanner constructions for general graphs. *SIAM Journal on Computing*, 33(3):608–631, 2004.

[ES81] Shimon Even and Yossi Shiloach. An on-line edge-deletion problem. *J. ACM*, 28(1):1–4, 1981.

[FGH21] Sebastian Forster, Gramoz Goranci, and Monika Henzinger. Dynamic maintenance of low-stretch probabilistic tree embeddings with applications. In

Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, (SODA 2021), pages 1226–1245, 2021.

[FGNS23] Sebastian Forster, Gramoz Goranci, Yasamin Nazari, and Antonis Skarlatos. Bootstrapping dynamic distance oracles. In *Proceedings of the 31th Annual European Symposium on Algorithms (ESA 2023)*, 2023.

[Fle00] Lisa Fleischer. Approximating fractional multicommodity flow independent of the number of commodities. *SIAM Journal of Discrete Mathematics*, 13(4):505–520, 2000. Announced at FOCS 1999.

[GK07] Naveen Garg and Jochen Könemann. Faster and simpler algorithms for multicommodity flow and other fractional packing problems. *SIAM Journal on Computing*, 37(2):630–652, 2007. Announced at FOCS 1998.

[HK95] Monika Rauch Henzinger and Valerie King. Fully dynamic biconnectivity and transitive closure. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*, pages 664–672. IEEE, 1995.

[HKN14] Monika Henzinger, Sebastian Krinninger, and Danupon Nanongkai. A subquadratic-time algorithm for decremental single-source shortest paths. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2014*, pages 1053–1072. SIAM, 2014.

[HKN16] Monika Henzinger, Sebastian Krinninger, and Danupon Nanongkai. Dynamic approximate all-pairs shortest paths: Breaking the $o(mn)$ barrier and derandomization. *SIAM Journal on Computing*, 45(3):947–1006, 2016. Announced at FOCS 2013.

[HKN18] Monika Henzinger, Sebastian Krinninger, and Danupon Nanongkai. Decremental single-source shortest paths on undirected graphs in near-linear total update time. *Journal of the ACM*, 65(6):36:1–36:40, 2018. Announced at FOCS 2014.

[HKNS15] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. Unifying and strengthening hardness for dynamic problems via the online matrix-vector multiplication conjecture. In *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing (STOC 2015)*, pages 21–30, 2015.

[Kav12] Telikepalli Kavitha. Faster algorithms for all-pairs small stretch distances in weighted graphs. *Algorithmica*, 63(1-2):224–245, 2012. Announced at FSTTCS 2007.

[Kin99] Valerie King. Fully dynamic algorithms for maintaining all-pairs shortest paths and transitive closure in digraphs. In *40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*, pages 81–89. IEEE, 1999.

[KŁ19] Adam Karczmarz and Jakub Łącki. Reliable hubs for partially-dynamic all-pairs shortest paths in directed graphs. In *Proceedings of the 27th Annual European Symposium on Algorithms (ESA 2019)*, pages 65:1–65:15, 2019.

[KŁ20] Adam Karczmarz and Jakub Łącki. Simple label-correcting algorithms for partially dynamic approximate shortest paths in directed graphs. In *Proceedings of the 3rd Symposium on Simplicity in Algorithms (SOSA 2020)*, pages 106–120, 2020.

[Knu17] Mathias Bæk Tejs Knudsen. Additive spanners and distance oracles in quadratic time. In *Proceedings of the 44th International Colloquium on Automata, Languages, and Programming, (ICALP 2017)*, pages 64:1–64:12, 2017.

[KS97] Philip N. Klein and Sairam Subramanian. A randomized parallel algorithm for single-source shortest paths. *J. Algorithms*, 25(2):205–220, 1997.

[ŁN22] Jakub Łącki and Yasamin Nazari. Near-Optimal Decremental Hopsets with Applications. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022.

[M1ę0] Aleksander Mądry. Faster approximation schemes for fractional multicommodity flow problems via dynamic graph algorithms. In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC 2010)*, pages 121–130, 2010.

[PR14] Mihai Patrascu and Liam Roditty. Distance oracles beyond the thorup-zwick bound. *SIAM Journal on Computing*, 43(1):300–311, 2014. Announced at FOCS 2010.

[PRT12] Mihai Patrascu, Liam Roditty, and Mikkel Thorup. A new infinity of distance oracles for sparse graphs. In *Proc. of the 53rd Annual IEEE Symposium on Foundations of Computer Science (FOCS 2012)*, pages 738–747. IEEE Computer Society, 2012.

[Rod23] Liam Roditty. New algorithms for all pairs approximate shortest paths. In Barna Saha and Rocco A. Servedio, editors, *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, pages 309–320. ACM, 2023.

[RZ11] Liam Roditty and Uri Zwick. On dynamic shortest paths problems. *Algorithmica*, 61(2):389–401, 2011. Announced at ESA 2004.

[RZ12] Liam Roditty and Uri Zwick. Dynamic approximate all-pairs shortest paths in undirected graphs. *SIAM J. Comput.*, 41(3):670–683, 2012. Announced at FOCS 2004.

[Som16] Christian Sommer. All-pairs approximate shortest paths and distance oracle preprocessing. In *Proceedings of the 43rd International Colloquium on Automata, Languages, and Programming, (ICALP 2016)*, volume 55, pages 55:1–55:13, 2016.

[SY24] Barna Saha and Christopher Ye. Faster approximate all pairs shortest paths. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 4758–4827. SIAM, 2024.

[Tho04] Mikkel Thorup. Fully-dynamic all-pairs shortest paths: Faster and allowing negative cycles. In *Proceedings of the 9th Scandinavian Workshop on Algorithm Theory (SWAT 2004)*, pages 384–396, 2004.

[TZ01] Mikkel Thorup and Uri Zwick. Compact routing schemes. In Arnold L. Rosenberg, editor, *Proceedings of the Thirteenth Annual ACM Symposium on Parallel Algorithms and Architectures, SPAA 2001*, pages 1–10. ACM, 2001.

[TZ05] Mikkel Thorup and Uri Zwick. Approximate distance oracles. *Journal of the ACM (JACM)*, 52(1):1–24, 2005.

[TZ06] Mikkel Thorup and Uri Zwick. Spanners and emulators with sublinear distance errors. In *Proceedings of the seventeenth annual ACM-SIAM symposium on Discrete algorithm*, pages 802–809, 2006.

[Wil18] R. Ryan Williams. Faster all-pairs shortest paths via circuit complexity. *SIAM Journal on Computing*, 47(5):1965–1985, 2018. Announced at STOC 2014.

[Zwi01] Uri Zwick. Exact and approximate distances in graphs - A survey. In *Proceedings of the 9th Annual European Symposium on Algorithms (ESA 2001)*, volume 2161, pages 33–48, 2001.

A Monotone ES-tree

Algorithm 1 provides pseudocode for the monotone ES-tree algorithm, this is taken from [HKN18].

Algorithm 1: Monotone ES-tree

```
// Internal data structures:
// N(u): for every node u a heap N(u) whose intended use is to store for
   every neighbor v of u in the current graph the value of ℓ(v) + w_H(u,v)
// Q: global heap whose intended use is to store nodes whose levels might
   need to be updated
```

Procedure `Initialize()`
  Compute shortest paths tree from s in H up to depth L;
  foreach $u \in V$ **do**
    Set $\ell(u) = \mathrm{d}_H(s,u)$;
    for *every edge* (u,v) *in* H **do** insert v into heap $N(u)$ of u with key $\ell(v) + w_H(u,v)$;

Procedure `Delete(`u`, `v`)`
  `Increase(`u`, `v`, `∞`)`

Procedure `Increase(`u`, `v`, `$w(u,v)$`)`
  `// Increase weight of edge` (u,v) `to` $w(u,v)$
  Insert u and v into heap Q with keys $\ell(u)$ and $\ell(v)$ respectively;
  Update key of v in heap $N(u)$ to $\ell(v) + w(u,v)$ and key of u in heap $N(v)$ to $\ell(u) + w(u,v)$;
  `UpdateLevels();`

Procedure `Insert(`u`, `v`, `$w(u,v)$`)`
  `// Increase edge` (u,v) `of weight` $w(u,v)$
  Insert v into heap $N(u)$ with key $\ell(v) + w(u,v)$ and u into heap $N(v)$ with key $\ell(u) + w_H(u,v)$;

Procedure `UpdateLevels()`
  while *heap* Q *is not empty* **do**
    Take node u with minimum key $\ell(u)$ from heap Q and remove it from Q;
    $\ell'(u) \leftarrow \min_v(\ell(v) + w_H(u,v))$;
    `//` $\min_v(\ell(v) + w_H(u,v))$ `can be retrieved from the heap` $N(u)$.
      $\arg\min_v(\ell(v) + w_H(u,v))$ `is` u`'s parent in the ES-tree.`
    if $\ell'(u) > \ell(u)$ **then**
      $\ell(u) \leftarrow \ell'(u)$;
      if $\ell'(u) > L$ **then** $\ell(u) \leftarrow \infty$;
      foreach *neighbor* v *of* u **do**
        update key of u in heap $N(v)$ to $\ell(u) + w_H(u,v)$;
        insert v into heap Q with key $\ell(v)$ if Q does not already contain v;

B Maintaining Bunches and Clusters

In this section we give a sketch of the algorithm of Łącki and Nazari [ŁN22] that allows us to maintain bunches and clusters needed for the decremental algorithm in Section 4. One tool used is a decremental algorithm by Roditty and Zwick [RZ12] that maintains Thorup-Zwick clusters up to a bounded depth. This result can be summarized as follows:

Lemma B.1 ([RZ12]). *Let $0 < p \leq 1$ and $d \geq 1$ be parameters. Given a dynamic graph $G = (V, E, w)$, that has edge deletions, weight increases, and edge insertions that do not decrease distances, we can maintain clusters (with sampling probability p) and bunches up to depth d in total update time $\tilde{O}(|E(G)|d/p)$. Moreover, we maintain distances from each node to its cluster and its bunch, and these distances are monotone (non-decreasing).*

This time dependence on d is too expensive for many applications, and thus [ŁN22] improve the time for maintaining *approximate* bunches and clusters by maintaining a hopset, combined with a well-known scaling tool.

Hopsets. Hopsets, originally introduced by Cohen and Zwick [CZ01] in the context of parallel algorithms, are widely used in faster algorithms in decremental settings. We briefly define this concept and sketch its application.

Definition 2. Let $G = (V, E, w)$ be a weighted undirected graph, and let $0 < \epsilon < 1$ be a parameter. A $(\beta, 1+\epsilon)$-hopset is a graph $H = (V, E(H), w_H)$ such that for each $u, v \in V$ we have $d_G(u, v) \leq d^{(\beta)}_{G \cup H}(u, v) \leq (1+\epsilon) d_G(u, v)$, where $d^{(\beta)}_{G \cup H}(u, v)$ denotes the length of the shortest path between u and v in $G \cup H$ that uses at most β hops. We say β is the *hopbound* of the hopset and $1+\epsilon$ is the *stretch* of the hopset.

Theorem B.2 ([ŁN22]). *Let $0 < \rho < 1/2, 0 < \epsilon < 1$ be parameters. We can maintain a $(\beta, 1+\epsilon)$-hopset H for $G = (V, E, w)$ with hopbound $\beta = O(\frac{\log(nW)}{\epsilon})^{2/\rho+1}$ in total update time $\tilde{O}(\frac{\beta \log^2(nW)}{\epsilon} m n^{\rho})$. Moreover, we can assume that the distances incurred by the additional edges of the hopset are monotone. Further we have that H is of size $\tilde{O}(n^{1+\rho})$.*

By setting $\rho = \frac{\log\log n}{\sqrt{\log n}}$, we can maintain a hopset with hopbound $2^{\tilde{O}(\sqrt{\log n})}$ in $2^{\tilde{O}(\sqrt{\log n})}$ amortized time. We often choose this parameter setting when the graph is sparse for balancing the hopset maintenance time with the shortest path computation time. By setting ρ to a constant, we can maintain a hopset of polylogarithmic hopbound in time $O(mn^{\rho})$. We often use this parameter setting when we are maintaining distances from many sources and the graph is slightly denser (e.g. $m = n^{1+\Omega(1)}$).

Scaling. The following scaling technique of Klein and Subramanian [KS97] changes a bounded hop distance problem into a bounded depth distance problem on a *scaled graph*. Similar ideas have been widely used in dynamic algorithms, see e.g., [Ber09,BR11,HKN18,ŁN22]. As we will see, the bounded depth works well together with the cluster and bunch maintaining algorithm of Roditty and Zwick [RZ12]

Lemma B.3 ([KS97]). *Let $G = (V, E, w)$ be a weighted undirected graph. Let $R \geq 0$, $d \geq 1$, $\epsilon > 0$ be parameters. We define a* scaled graph *to be a graph $\textsc{Scale}(G, R, \epsilon, d) := (V, E, \hat{w})$, such that $\hat{w}(e) = \lceil \frac{w(e)d}{R\epsilon} \rceil$. Then for any path π in G such that π has at most d hops and weight $R \leq w(\pi) \leq 2R$ we have: 1) $\hat{w}(\pi) \leq \lceil 2d/\epsilon \rceil$, and 2) $w(\pi) \leq \frac{\epsilon R}{d} \cdot \hat{w}(\pi) \leq (1+\epsilon) w(\pi)$.*

Hence given a hopset, we can 'cover' the entire graph by scaled graphs, by considering separate scaled graphs G_i for the distance scale $[2^i, 2^{i+1}]$ for $i = 1, \ldots, \log(nW)$. In order to combine this with a hopset algorithm, we need to scaled the graph after adding edges corresponding to a sequence of 2^i-*restricted hopsets*, i.e. a graph that satisfies the hopset condition only for pairs within distance at most 2^i. Slightly more formally, we define a scaled graph as follows: $G^j := \textsc{Scale}(G \cup \bigcup_{r=0}^{j} H_r, 2^j, \epsilon, O(\beta)$, where H_r is a 2^r-restricted hopset. The final hopsets of [ŁN22] are the union of all these restricted hopsetes, however for maintaining distances efficiently we need to run the algorithm in Lemma B.1 on all these scaled graphs simultaneously and take the minimum estimate over all the scales. Importantly, to handle edge insertions they use monotone ES trees which implies that the distances obtained from this algorithm is monotone. This is important for us in bounding the running time of our decremental data structures.

The following lemma shows that using the specific hopsets by [ŁN22], we can maintain approximate single-source distances.

Lemma 3.1. *Given a decremental graph $G = (V, E, w)$, and parameters $0 < \epsilon < 1$, $0 < \rho < 1/2$ and the sampling probability $0 < p < 1$ we can maintain for each $v \in V$ an approximate pivot $\tilde{p}(v)$, such that $d_G(v, p(v)) \leq \delta_{\mathbf{B}}(v, \tilde{p}(v)) \leq (1+\epsilon) d_G(v, p(v))$. Moreover, we maintain approximate bunches $\tilde{B}(v)$ that contains $\hat{B}(v) := \{w \in V : (1+\epsilon) d_G(v, w) < d_G(v, p(v))\}$ and we have monotone (non-decreasing over updates) distance estimates $\delta_{\mathbf{B}}$ such that for any $u \in \tilde{B}(v)$ we have $d_G(u, v) \leq \delta_{\mathbf{B}}(u, v) \leq (1+\epsilon) d_G(u, v)$, and with high probability, we have $|\tilde{B}(v)| = O(\frac{\log n \log nW}{p})$. The algorithm has total update time $\tilde{O}(\frac{mn^\rho}{p}) \cdot O(\frac{1}{\rho\epsilon})^{1+2/\rho}$.*

Proof sketch. First, using the algorithm of [ŁN22], we can maintain a $(1+\epsilon, \beta)$-hopset H of size $\tilde{O}(m + n^{1+\rho})$ in update time $\tilde{O}(mn^\rho\beta)$ with hopbound $\beta = \tilde{O}(\frac{1}{\rho\epsilon})^{2\rho}$, which is consisted of a sequence of 2^i-restricted hopsets $H_1, ..., H_{\log nW}$. We then use the framework of [ŁN22] to maintain *approximate* clusters and hence approximate bunches on the scaled graphs. Since for all $u, v \in V$ in $G \cup H$ there is a path with β-hops that with length at most $(1+\epsilon) d_G(u, v)$, they show that it is enough to maintain $O(\beta/\epsilon)$-hop bounded bunches/clusters on scaled graphs $G_1, ..., G_{\log nW}$. We consider $(4\beta/\epsilon)$-*bounded bunches* $B_{G_i}(v) = \{w : d_{G_i}(w, v) < d_{G_i}(v, p_i(v))\}$ where $p_i(v)$ is the closest sampled node in G_i and $d_{G_i}(v, p_i(v)) \leq 4\beta/\epsilon$. The scaling ensures that for each node v there exists $1 \leq i \leq \log nW$ such that $p_i(v)$ satisfies the property $d_{G_i}(v, p_i(v)) \leq 4\beta/\epsilon$. Let d be the distances obtained by scaling back the estimates corresponding to $B_{G_i}(v) \cup p_i(v)$. There exists i such that $\tilde{p}(v) = p_i(v)$ satisfies $d_G(v, p(v)) \leq d(v, \tilde{p}(v)) \leq (1+\epsilon) d_G(v, p(v))$. Also by choosing the appropriate error parameters (to account for the scaling and hopset errors) in Theorem 14 of [ŁN22] we can ensure that for any $u \in \tilde{B}(v)$ we have $d_G(u, v) \leq d(u, v) \leq (1+\epsilon) d_G(u, v)$. Consider any $u \in \hat{B}(v)$. We have $d(u, v) \leq (1+\epsilon) d_G(u, v) \leq d_G(v, p(v)) \leq d(v, \tilde{p}(v))$. Hence we can see that the bunch $B_{G_i}(v) \subseteq \hat{B}(v)$ and thus $\tilde{B}(v) := \cup_{1 \leq i \leq \log nW} B_{G_i}$ contains all the nodes in $\hat{B}(v)$.

Finally since the algorithm of [ŁN22] (that is based on [RZ12]) maintains bunches of size $O(\log n/p)$ on $O(\log nW)$ scaled graphs, we also have the bound on the size of the bunch. □